

Coalition for Content Provenance and Authenticity

Content Credentials

C2PA Technical Specification

2.5, 2026-10-09:

Table of Contents

1. Introduction . . . 2
    1.1. Overview . . . 2
    1.2. Scope . . . 3
    1.3. Technical Overview . . . 3
    1.4. How to Read This Specification . . . 7
    1.5. Claim Generators . . . 7
    1.6. Validators . . . 8
    1.7. Security Reviewers . . . 8
2. Glossary . . . 9
    2.1. Introductory terms . . . 9
    2.2. Assets and Content . . . 10
    2.3. Core Aspects of C2PA . . . 11
    2.4. Additional Terms . . . 14
3. Elements of a Content Credential . . . 16
4. Standard Terms . . . 17
5. Versioning . . . 18
    5.1. Compatibility . . . 18
    5.2. Forward Compatibility . . . 18
    5.3. Version History . . . 19
6. Normative References . . . 20
    6.1. Core Formats . . . 20
    6.2. Schemas . . . 20
    6.3. Digital & Electronic Signatures . . . 20
    6.4. Embeddable Formats . . . 21
    6.5. Other . . . 22
7. Entity Diagram . . . 24
8. Assertions . . . 27
    8.1. General . . . 27
    8.2. Labels . . . 27
    8.3. Versioning . . . 29
    8.4. Multiple Instances . . . 29

8.5. Schema Validation . . . 29
8.6. Assertion Store . . . 30
8.7. Embedded vs Externally-Stored Data . . . 30
8.8. Redaction of Assertions . . . 30
8.9. Specifications of time in assertions . . . 31
9. Unique Identifiers . . . 33
9.1. Uniquely Identifying C2PA Manifests and Assets . . . 33
9.2. Versioning Manifests Due to Conflicts . . . 34
9.3. Identifying Non-C2PA Assets . . . 35
9.4. URI References . . . 35
10. Claims . . . 40
10.1. Overview . . . 40
10.2. Syntax . . . 40
10.3. Creating a Claim . . . 45
11. Manifests . . . 52
11.1. Overview . . . 52
11.2. Manifest Store . . . 52
11.3. Types of Manifests . . . 54
11.4. Ingredient Storage . . . 55
11.5. Serialization . . . 57
11.6. Embedding . . . 62
11.7. Multiple Step Processing . . . 62
11.8. External Manifests . . . 65
12. Cryptography . . . 67
12.1. Hashing . . . 67
12.2. Binding to Content . . . 68
12.3. Digital Signatures . . . 71
13. Trust Model . . . 77
13.1. Overview . . . 77
13.2. Identity of Signers . . . 77
13.3. Validation states . . . 78
13.4. Trust Lists . . . 80
13.5. X.509 Certificates . . . 82

14. Validation . . . 89
14.1. Validation Process . . . 89
14.2. Returning Validation Results . . . 90
14.3. Displaying Manifest Information . . . 93
14.4. Determining the hashing algorithm . . . 93
14.5. Locating the Active Manifest . . . 94
14.6. Locating and Validating the Claim . . . 100
14.7. Validate the Signature . . . 101
14.8. Validate the Time Stamp . . . 101
14.9. Validate the Credential Revocation Information . . . 105
14.10. Validate the Assertions . . . 108
14.11. Validate the Ingredients . . . 126
14.12. Validate the Asset's Content . . . 133
15. Live Media . . . 149
15.1. Introduction . . . 149
15.2. Compatibility With Previous Versions . . . 149
15.3. Architecture . . . 150
15.4. Per-segment C2PA Manifest Box method . . . 151
15.5. Verifiable Segment Info method . . . 153
15.6. Live Media Architecture Diagram . . . 163
15.7. Live Media Generation Process . . . 164
15.8. Live Media Validation Process . . . 166
16. C2PA Standard Assertions . . . 170
16.1. Introduction . . . 170
16.2. Standard C2PA Assertion Summary . . . 170
16.3. Content Binding & Integrity . . . 173
16.4. Provenance & Workflow . . . 224
16.5. Ingredient . . . 263
16.6. Content Description & Enrichment . . . 283
16.7. Trust & Validation Adjuncts . . . 305
16.8. Policy & Disclosure . . . 310
16.9. Assertion-Supporting Structures . . . 315
17. Patent Policy . . . 332

Appendix A: Embedding manifests . . . 333
A.1. Supported Formats . . . 333
A.2. Embedding manifests in multi-part assets . . . 336
A.3. Embedding manifests into non-BMFF-based assets . . . 336
A.4. Embedding manifests into PDFs . . . 356
A.5. Embedding manifests into BMFF-based assets . . . 359
A.6. Embedding manifests into ZIP-based formats . . . 368
A.7. Embedding Manifests into HTML . . . 371
A.8. Embedding Manifests into Unstructured Text . . . 374
A.9. Embedding Manifests into Structured Text . . . 381
A.10. Embedding manifests into WARC files . . . 388
Appendix B: User Experience . . . 393
B.1. Principles . . . 393
B.2. Disclosure Levels . . . 393
B.3. Threats and Security Considerations . . . 393
B.4. Harms, Misuse, and Abuse . . . 396
Appendix C: Implementation Details for `c2pa.metadata` . . . 397
C.1. Completely Supported Schemas . . . 397
C.2. Partially Supported Schemas . . . 397
Appendix D: Standard Validation Status Codes . . . 407
D.1. Success codes . . . 407
D.2. Informational codes . . . 408
D.3. Failure codes . . . 411
Appendix E: Considerations for Deprecation . . . 421
E.1. Status of Constructs . . . 421
Appendix F: Version History . . . 429
F.1. 2.5 - October 2026 . . . 429
F.2. 2.4 - April 2026 . . . 430
F.3. 2.3 - December 2025 . . . 432
F.4. 2.2 - May 2025 . . . 433
F.5. 2.1 - September 2024 . . . 434
F.6. 2.0 - January 2024 . . . 437
F.7. 1.4 - November 2023 . . . 438



This work is licensed under a Creative Commons Attribution 4.0 International License.

THESE MATERIALS ARE PROVIDED “AS IS.” The parties expressly disclaim any warranties (express, implied, or otherwise), including implied warranties of merchantability, non-infringement, fitness for a particular purpose, or title, related to the materials. The entire risk as to implementing or otherwise using the materials is assumed by the implementer and user. IN NO EVENT WILL THE PARTIES BE LIABLE TO ANY OTHER PARTY FOR LOST PROFITS OR ANY FORM OF INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES OF ANY CHARACTER FROM ANY CAUSES OF ACTION OF ANY KIND WITH RESPECT TO THIS DELIVERABLE OR ITS GOVERNING AGREEMENT, WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), OR OTHERWISE, AND WHETHER OR NOT THE OTHER MEMBER HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Chapter 1. Introduction

1.1. Overview

With the increasing velocity of digital content and the increasing availability of powerful creation and editing techniques, establishing the provenance of media is critical to ensure transparency, understanding, and ultimately, trust.

We are witnessing extraordinary challenges to trust in media. As social platforms amplify the reach and influence of certain content via ever more complex and opaque algorithms, mis-attributed and mis-contextualized content spreads quickly. Whether inadvertent misinformation or deliberate deception via disinformation, inauthentic content is on the rise.

Currently, those who wish to include metadata about their work cannot do so in a secure, tamper-evident and standardized way across platforms. Without this information coming from a recognized source, publishers and consumers lack critical context for determining the authenticity of media.

Provenance empowers content creators and editors, regardless of their geographic location or degree of access to technology, to disclose information about how an asset was created, how it was changed and what was changed. Each time an asset is changed, the existing provenance of the asset is preserved, with each new change being added to the provenance. In this way, content with provenance provides indicators of authenticity so that consumers can have awareness of altered content. Such provenance could include what has been changed and the source of those changes. This ability to provide provenance for creators, publishers and consumers is essential to facilitating trust online.

To address this issue at scale for publishers, creators and consumers, the Coalition for Content Provenance and Authenticity (C2PA) has developed this technical specification for providing content provenance and authenticity. It is designed to enable global, opt-in, adoption of digital provenance techniques through the creation of a rich ecosystem of digital provenance enabled applications for a wide range of individuals and organizations while meeting appropriate security requirements.

This specification has been, and continues to be, informed by scenarios, workflows and requirements gathered from industry experts and partner organizations, including the Project Origin Alliance and the Content Authenticity Initiative (CAI). It is also possible that regulatory bodies and governmental agencies could utilize this specification to establish standards for digital provenance.

1.2. Scope

This specification describes the technical aspects of the C2PA architecture; a model for storing and accessing cryptographically verifiable information whose trustworthiness can be assessed based on a defined trust model. Included in this document is information about how to create and process a C2PA Manifest and its components, including the use of digital signature technology for enabling tamper-evidence as well as establishing trust.

Prior to developing this specification, the C2PA created our Guiding Principles that enabled us to remain focused on ensuring that the specification can be used in ways that respect privacy and personal control of data with a critical eye toward potential abuse and misuse. For example, implementers of this specification are strongly encouraged to provide creators and publishers of media assets with the ability to control whether certain provenance data is included.

IMPORTANT

From the overarching goals section of the guiding principles:

> C2PA specifications DO NOT provide value judgements about whether a given set of provenance data is 'good' or 'bad,' merely whether the assertions included within can be validated as associated with the underlying asset, correctly formed, and free from tampering.

It is important that the specification does not negatively impact content accessibility for consumers.

Other documents from the C2PA will address specific implementation considerations such as expected user experiences and details of our threat and harms modelling.

1.3. Technical Overview

The C2PA information comprises a series of statements that cover areas such as asset creation, edit actions, capture device details, bindings to content and many other subjects. These statements, called assertions, make up the provenance of a given asset and represent a series of trust signals that can be used by a human to improve their view of trustworthiness concerning the asset. Assertions are wrapped up with additional information into a digitally signed entity called a claim. This claim is digitally signed by the claim generator on behalf of the signer, using the signer's signing credential, producing the claim signature.

These assertions, claims, and the claim signature are all bound together into a verifiable unit called a C2PA Manifest (see Figure 1, “A C2PA Manifest and its constituent parts”) by a hardware or software component called a claim generator. The set of C2PA Manifests, as stored in the asset’s Content Credential, represent its provenance data.

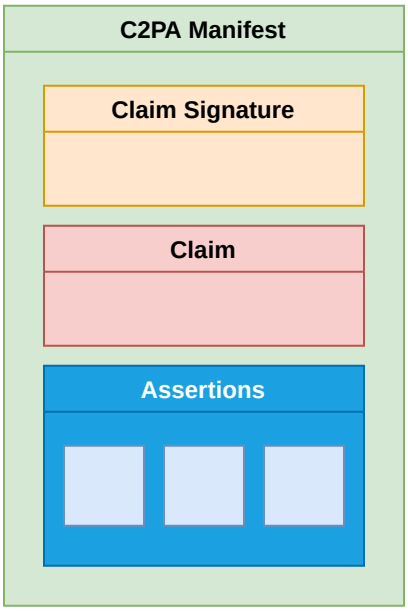


Figure 1. A C2PA Manifest and its constituent parts

1.3.1. Establishing Trust

The basis of making trust decisions in C2PA, our trust model, is the identity of the signer associated with the cryptographic signing key used to sign a claim in a C2PA Manifest. The claim signatures of C2PA Manifests, when combined with trusted time stamps, can undergo the validation process indefinitely to determine if claims were signed while the signing credentials were valid and not revoked.

1.3.2. An Example

A very common scenario will be a user taking a photograph with their C2PA-enabled camera (or phone). In that instance, the camera would create a manifest containing some assertions including information about the camera itself, a thumbnail of the image and some cryptographic hashes that bind the photograph to the manifest. These assertions would then be listed in the Claim, which would be digitally signed and then the entire C2PA Manifest (see Figure 2, “Example C2PA Manifest of a Photograph”) would be embedded into the output JPEG. This C2PA Manifest would remain valid indefinitely.

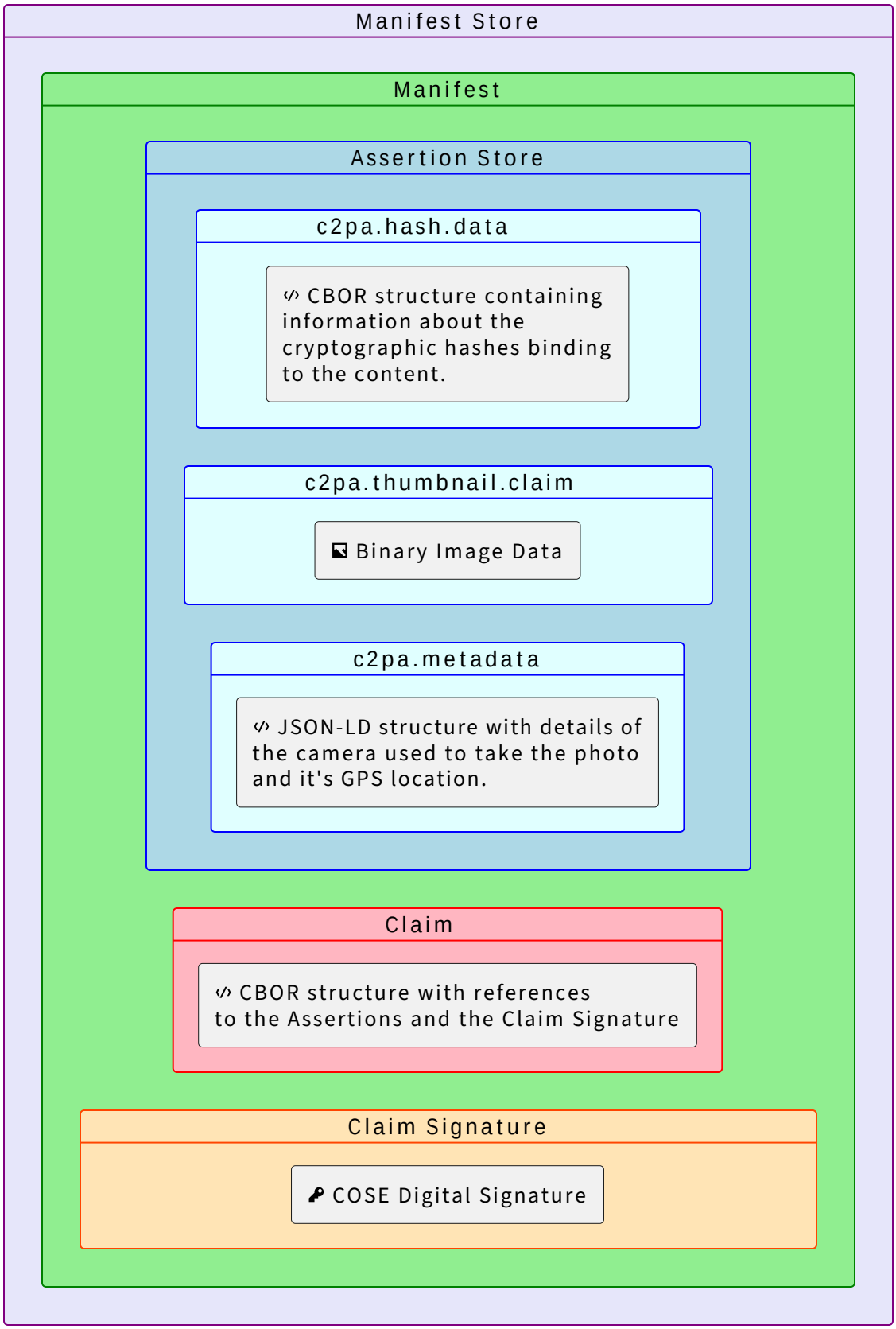


Figure 2. Example C2PA Manifest of a Photograph

A Manifest Consumer, such as a C2PA validator, helps users to establish the trustworthiness of the asset by first validating the digital signature and its associated credential. It also checks each of the assertions for validity and presents the information contained in them, and the signature, to the user in a way that they can then make an informed decision about the trustworthiness of the digital content.

1.3.3. Design Goals

In the creation of the C2PA architecture, it was important to establish some clear goals for the work to ensure that the technology was usable across a wide spectrum of hardware and software implementations worldwide and accessible to all. Those goals can be found in Table 1, “C2PA Design Goals”.

Table 1. C2PA Design Goals

Goal	Description
Privacy	Enable users to control the privacy of their information, including consumption data and information recorded in provenance
Responsibility	Ensure consumers can determine the provenance of an asset
Scalability	Enable creation/consumption/validation of media provenance at the same scale as media creation/consumption on the web
Extensibility	Ensure future metadata and credential providers are able to add their information without requiring input or approval from the C2PA
Interoperability	Ensure that differing implementations are able to operate with each other without ambiguity
Whole Workflow Applicability	Maintain the provenance of the asset across multiple tools, from creation through all subsequent modification and publication/distribution
Technology Minimalism	Create only the minimum required novel technology in the specification by relying on prior, well-established techniques
Security	Design to ensure that consumers can trust the integrity and source of provenance, and ensure the design is reviewed by experts
Content Ubiquity	Enable the inclusion of provenance for all common media types, including documents
Flexible Locality	Enable both online and offline (asset-only) storage and consumption/validation of provenance

Goal	Description
Global Universality	Design for the needs of interested users throughout the world
Accessibility	Ensure that the technology can be used in a way that conforms to recognized accessibility standards, such as WCAG
Harms and Misuse	Design to avert and mitigate potential harms, including threats to human rights and disproportionate risks to vulnerable groups
Evolving	Continuous review of the specification against these goals to ensure that they remain our priority

1.4. How to Read This Specification

Before reading this specification, it's helpful to determine which parts of the specification apply to your role. It's easier to read the spec in relevant parts and in a useful order rather than as a whole from start to finish.

Below are common roles and the sections most relevant to them, listed in the order it's most helpful to read them.

- Section 1.5, "Claim Generators"
- Section 1.6, "Validators"
- Section 1.7, "Security Reviewers"

1.5. Claim Generators

The core implementation flow for Claim Generators is to create assertions, reference them from a claim, include relevant ingredients, bind the claim to the asset, sign the claim, and package everything into a valid C2PA Manifest.

Examples of Claim Generators in the C2PA ecosystem include cameras, recorders, AI asset creators, asset editing software, and transcoders.

See the following reading lists in the Reading Lists by Role annex:

- Must Read to Implement a Claim Generator
- Must Read Only If the Claim Generator Includes Live Media Assets
- Must Read Only If the Claim Generator Supports Ingredients

1.6. Validators

The core implementation flow for validators is to locate the C2PA Manifest Store, identify the active manifest, validate the claim and signature, check signer trust and credential status, validate assertions and ingredients, validate the asset's content binding, and return or display validation results.

Examples of validators in the C2PA ecosystem include photo gallery applications, social media platforms, news publishers, and verification tools.

See the following reading lists in the Reading Lists by Role annex:

- Must Read to Implement Validation
- Must Read Only If Displaying Validation Results to End Users
- Must Read Only If Validating Live Media Assets
- Helpful Context on C2PA Data Structures

1.7. Security Reviewers

Security reviewers inspect the various specifications and guidance documents from the C2PA to determine whether the trust model, cryptographic algorithms, validation process, and security guidance meet current best practices.

See the following reading lists in the Reading Lists by Role annex:

- Must Read to Review C2PA Security
- Helpful Context on C2PA Data Structures
- Must Read Only If Reviewing User-Facing Trust Signals
- Must Read Only If Reviewing Live Media Workflows

Chapter 2. Glossary

2.1. Introductory terms

2.1.1. Actor

A human or non-human (hardware or software) that is participating in the C2PA ecosystem. For example: a camera (capture device), image editing software, cloud service or the person using such tools.

NOTE	An organization or group of actors may also be considered an actor in the C2PA ecosystem.

2.1.2. Claim generator

The non-human (hardware or software) actor that generates the claim about an asset as well as the claim signature, thus leading to the asset's associated C2PA Manifest.

NOTE	The claim generator is a component of a larger system, such as a library, software development kit (SDK), or shared service that a signer relies on to generate and embed claims in assets, but is not the Signer itself and thus does not assume responsibility for the content of the Claim or Assertions.

2.1.3. Signer

The credential holder of a private key that is used to sign the claim. The signer is identified by the subject of the credential.

2.1.4. Manifest consumer

An actor who consumes an asset with an associated C2PA Manifest for the purpose of obtaining the provenance data from the C2PA Manifest.

2.1.5. Validator

A Manifest Consumer whose role is to perform the actions described in validation.

2.1.6. Action

An operation performed by an actor on an asset. For example, "create", "embed", or "apply filter".

2.2. Assets and Content

2.2.1. Digital content

The portion of an asset that represents the actual content, such as the pixels of an image, along with any additional technical metadata required to understand the content (e.g., a colour profile or encoding parameters).

2.2.2. Asset metadata

Non-technical information about the asset and its digital content.

2.2.3. Asset

A file or stream of data containing digital content, asset metadata and optionally, a C2PA Manifest.

> NOTE For the purposes of this definition, we will extend the typical definition of "file" to include cloud-native and dynamically generated data.

2.2.4. Derived asset

A derived asset is an asset that is created by starting from an existing asset and performing actions to it that modify its digital content.

> EXAMPLE: An audio stream that has been shortened or a document where pages have been added.

2.2.5. Asset rendition

A representation of an asset (either as a part of an asset or a completely new asset) where the digital content has had a 'non-editorial transformation' action (e.g., re-encoding or scaling) applied.

EXAMPLE: A video file that is re-encoded for reduced screen resolution or network bandwidth.

2.2.6. Composed asset

A composed asset is an asset that is created by building up a collection of multiple parts or fragments of digital content (referred to as ingredients) from one or more other assets. When starting from an existing asset, it is a special case of a derived asset - however a composed asset can also be one that starts from a "blank slate".

EXAMPLES:

- A video created by importing existing video clips and audio segments into a "blank slate".
- An image where another image is imported and super-imposed on top of the starting image.

2.2.7. Editorial transformation

A type of transformation that alters either the intent or meaning or both of the digital content.

NOTE

Every editorial transformation is a perceptible transformation, but not every perceptible transformation is editorial

2.2.8. Perceptible transformation

A type of transformation that alters how the digital content is perceived by humans under normal conditions.

2.3. Core Aspects of C2PA

2.3.1. Assertion

A data structure which represents a statement either made (or "created") by the signer or simply gathered at claim generation-time, concerning the asset. This data is a part of the C2PA Manifest.

2.3.2. Assertion store

The logical collection of assertions referenced by a claim within a C2PA Manifest, stored together as a single construct. For the full structural description, see Assertion Store.

2.3.3. Claim

A digitally signed and tamper-evident data structure that references a set of assertions, concerning an asset, and the information necessary to represent the content binding. If any assertions were redacted, then a declaration to that effect is included. This data is a part of the C2PA Manifest.

2.3.4. Claim signature

The digital signature on the claim created using the private key owned by a signer. The claim signature is a part of the C2PA Manifest.

2.3.5. C2PA Manifest

The set of information about the *provenance* of an asset based on the combination of one or more assertions (including content bindings), a single claim, and a claim signature. A C2PA Manifest is part of a C2PA Manifest Store.

> NOTE A C2PA Manifest can reference other C2PA Manifests.

2.3.6. C2PA Manifest Store

A collection of C2PA Manifests that can either be embedded into an asset or be external to its asset.

2.3.7. Content Credential

This is the preferred non-technical term for a C2PA Manifest. The C2PA Manifest Store therefore represents the Content Credentials of an asset.

Content Credentials also refers to the overall C2PA technology, and is therefore essentially treated as a plural noun. If a C2PA Manifest is a Content Credential, then multiple C2PA Manifest or the broader, universal concept is Content Credentials.

2.3.8. Active Manifest

The last manifest in the list of C2PA Manifests inside of a C2PA Manifest Store which is the one with the set of content bindings that are able to be validated. For the full structural description, see Manifest Store.

2.3.9. Provenance

The logical concept of understanding the history of an asset and its interaction with actors and other assets, as represented by the provenance data.

2.3.10. Provenance data

The set of C2PA Manifests for an asset and, in the case of a composed asset, its ingredients. For the full structural description, see Manifests.

> NOTE A C2PA Manifest can reference other C2PA Manifests.

2.3.11. Authenticity

A property of digital content comprising a set of facts (such as the provenance data and hard bindings) that can be cryptographically verified as not having been tampered with.

2.3.12. Content binding

Information that associates digital content to a specific C2PA Manifest associated with a specific asset, either as a hard binding or a soft binding. For the full structural description, see Binding to Content.

2.3.13. Hard binding

One or more cryptographic hashes that uniquely identifies either the entire asset or a portion thereof.

2.3.14. Soft binding

A content identifier that is either (a) not statistically unique, such as a fingerprint, or (b) embedded as an invisible watermark in the identified digital content.

2.3.15. Trust signals

The collection of information that can inform a Manifest Consumer's judgment of the trustworthiness of an asset. These are in addition to the signer upon which the fundamental trust model relies.

2.3.16. C2PA Trust List

A C2PA-managed list of X.509 certificate trust anchors that issue certificates to hardware & software signers that use them to sign claims.

2.3.17. C2PA TSA Trust List

A C2PA-managed list of X.509 certificate trust anchors that issue certificates to Time Stamping Authorities (TSAs).

2.4. Additional Terms

2.4.1. Durable Content Credential

A Durable Content Credential is a Content Credential for which there exists one or more soft bindings that enable its discovery in a manifest repository.

2.4.2. Fingerprint

A set of inherent properties computable from digital content that identifies the content or near duplicates of it.

> EXAMPLE: An asset can become separated from its C2PA Manifest due to removal or corruption of asset metadata. A fingerprint of the digital content of the asset could be used to search a database to recover the asset with an intact C2PA Manifest.

2.4.3. Invisible Watermark

Information incorporated in a substantially human imperceptible way into the digital content of an asset which can be used, for example, to uniquely identify the asset or to store a reference to a C2PA Manifest.

2.4.4. Visible Watermark

A perceptible component of the digital content carrying some human consumable information about the provenance of the asset.

2.4.5. Manifest Repository

A repository into which C2PA Manifests and C2PA Manifest Stores can be placed, and which can be searched using a content binding.

Chapter 3. Elements of a Content Credential

This image shows how all these various elements come together to represent the C2PA architecture.

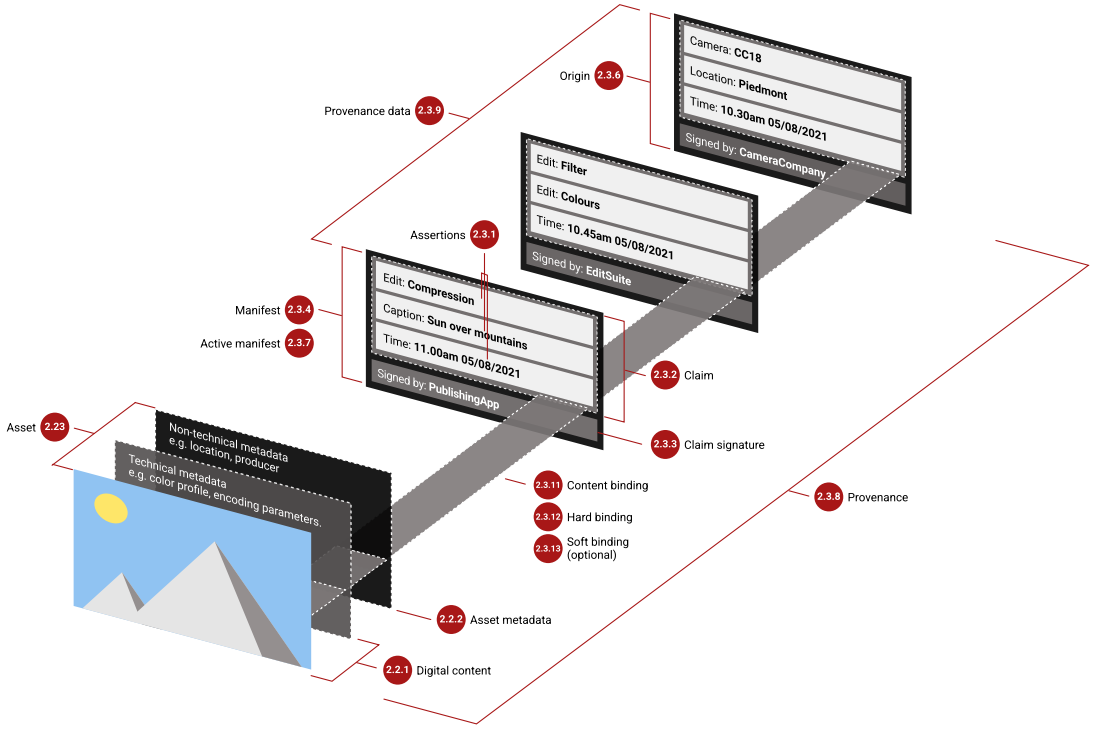


Figure 3. Elements of a Content Credential

Chapter 4. Standard Terms

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14, RFC 2119, and RFC 8174 when they appear in any casing (upper, lower or mixed).

CDDL Rules and Type Definitions in the CDDL schemas for C2PA constructs are normative. Schema validation is not necessary, nor it is recommended. Comments within CDDL schemas are non-normative. If there is a conflict between the CDDL schema and the normative text of this specification, the normative text takes precedence.

Chapter 5. Versioning

5.1. Compatibility

As the Content Credentials specification has evolved, constructs such as box labels, assertions (and their fields), claims and time stamps have also evolved. New assertions have been added, and some existing assertions and the claim have newer versions with additional fields. In addition, some constructs have been deprecated. In this specification, when a construct is marked as deprecated, that means that a claim generator shall not write that construct (or value), but that a validator should read it.

To facilitate testing and diagnosing interoperability issues between claim generators and validators, a claim generator should declare which version of the specification it is using to generate the C2PA Manifest by providing a `specVersion` key in the `claim_generator_info` field of the claim. When a claim generator sets this field, it is declaring that the active manifest of the asset is produced in accordance with that version of the specification and thus, for example, does not contain any constructs that are deprecated in that version of the specification (see Table 21, "Status of constructs" of Appendix E, *Considerations for Deprecation*).

A validator shall be compatible with at least one version of the specification, but may be compatible with additional versions. A validator that is compatible with a specific version of the specification shall support all non-deprecated constructs listed for that version. If the validator encounters a C2PA Manifest that uses constructs from a version of the specification that the validator does not support (either because they are deprecated or unknown), it may ignore the unsupported construct and process the rest of C2PA Manifest as if that construct were not present.

When a claim generator is performing ingredient validation, it should add a `specVersion` key to the `validation-results-map` to declare what version of the specification was used as the basis for the validation procedure.

5.2. Forward Compatibility

C2PA aims to ensure that Content Credentials created under newer versions of the specification remains parseable and validatable when processed by implementations conforming to earlier versions, wherever practical. Implementations of the C2PA specification should therefore be designed to:

- Gracefully handle unknown or unrecognized data (e.g., new fields, structures, or claim types) without causing fatal errors.
- Preserve the integrity of a C2PA Manifest even when parts of it cannot be fully interpreted.

- Indicate partial interpretation to users or downstream systems, rather than rejecting the C2PA Manifest outright.

While this specification strives to maintain both forward and backward compatibility, there may be occasions when breaking changes are necessary to enable significant advances or address fundamental security, trust, or interoperability concerns. In such cases, versioning guidance and migration considerations will be explicitly documented.

5.3. Version History

A detailed version history of this specification can be found in the Version History appendix of this document.

Chapter 6. Normative References

6.1. Core Formats

- CBOR
- JSON
- JSON-LD
- HTML Living Standard
- JPEG universal metadata box format (JUMBF)

6.2. Schemas

- CDDL
- JSON Schema
- Dublin Core Metadata Initiative

6.3. Digital & Electronic Signatures

- Cryptographic Message Syntax (CMS)
- The "data" URL scheme (RFC 2397)
- The Base16, Base32, and Base64 Data Encodings (RFC 4648)
- Internet X.509 PKI Time-Stamp Protocol
- Internet X.509 Public Key Infrastructure: Additional Algorithms and Identifiers for DSA and ECDSA
- JavaScript Object Notation (JSON) Pointer
- Online Certificate Status Protocol (OCSP)
- PKCS #1: RSA Cryptography Specifications Version 2.2
- CBOR Object Signing and Encryption (COSE): Structures and Process
- CBOR Object Signing and Encryption (COSE): Initial Algorithms
- Using RSA Algorithms with COSE Messages
- CBOR Web Tokens (CWT)

- Algorithm Identifiers for Ed25519, Ed448, X25519, and X448 for Use in the Internet X.509 Public Key Infrastructure
- CBOR Object Signing and Encryption (COSE): Header Parameters for Carrying and Referencing X.509 Certificates
- Internet X.509 Public Key Infrastructure: Logotypes in X.509 Certificates
- Internet X.509 Public Key Infrastructure: Algorithm Identifiers for the Module-Lattice-Based Digital Signature Algorithm (ML-DSA)
- Module-Lattice-Based Digital Signature Algorithm (ML-DSA) for JSON Object Signing and Encryption (JOSE) and CBOR Object Signing and Encryption (COSE)
- FIPS 204: Module-Lattice-Based Digital Signature Standard
- JSON Advanced Electronic Signatures (JAdES)
- Procedures for Creation and Validation of AdES Digital Signatures, Part 1 (ETSI EN 319 102-1)
- Electronic Signatures and Trust Infrastructures (ESI); Lists of trusted entities; Data model (ETSI TS 119 602)
- Electronic Signatures and Infrastructures (ESI); Trusted Lists (ETSI TS 119 612)

6.4. Embeddable Formats

- ISO Base Media File Format (BMFF)
- PDF 1.7
- PDF 2.0
- JPEG 1
- JPEG XT, ISO/IEC 18477-3
- JPEG XL, ISO/IEC 18181-2:2024
- ISO/IEC 23009-9, DASH Part 9: Redundant encoding and packaging for segmented live media (REaP)
- PNG
- SVG
- Timed Text Markup Language 2 (TTML2)
- GIF
- ID3

- Standard MIDI Files 1.0
- Digital Negative (DNG) Specification, Version 1.7.1.0
- TIFF/EP
- TIFF v6
- RIFF
- Multi-Picture Format (MPF)
- Exif
- Open Font Format
- OpenType
- Ogg Vorbis
- ISO 28500:2017
- WebVTT
- ISO/IEC 23000-19, Common Media Application Format (CMAF)
- XML Signature Syntax and Processing (W3C XML DigSig Core)
- ONNX Intermediate Representation Specification, Version 10
- SafeTensors Format Specification, Version 0.4
- Protocol Buffers Language Guide (proto3)
- Enhanced RTMP (V2)
- Adobe's Real Time Messaging Protocol (21 December 2012)

6.5. Other

- eXtensible Metadata Platform (XMP)
- JSON-LD serialization of XMP
- IPTC Photo Metadata Standard 2025.1
- Exif metadata for XMP
- Uniform Resource Names (URNs)
- Universally Unique IDentifiers (UUIDs)

- ISO 8601
- RFC 3339
- RFC 7826
- RFC 9110
- RFC 9580
- Media Fragments
- Web Annotation Data Model
- Brotli Compressed Data Format
- Unicode Standard Annex #15: Unicode Normalization Forms (UAX #15)
- RFC 5646, BCP 47
- Unicode Standard, Ch. 23 (Variation Selectors)
- "The Atom Syndication Format", RFC 4287.
- RSS 2.0 Specification
- ISO 14064-1:2018
- ISO/IEC TR 20226:2025
- Subresource Integrity Level 2

Chapter 7. Entity Diagram

Figure 4, “C2PA Entity Diagram” provides a look at how all of the pieces of the C2PA system integrate and relate to each other.

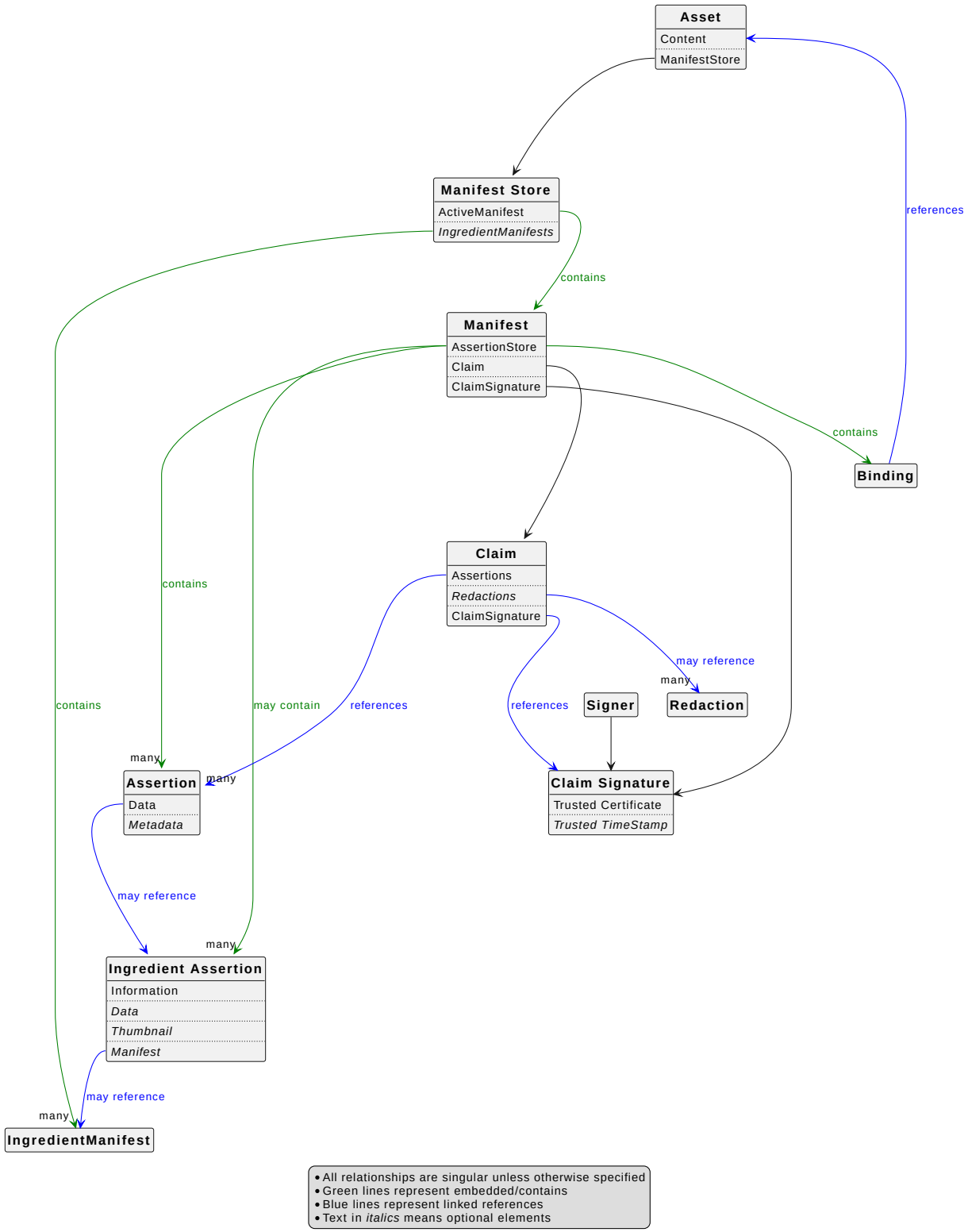


Figure 4. C2PA Entity Diagram

An asset embeds a C2PA Manifest Store which holds one or more C2PA Manifests; the last of these is the active manifest, whose content bindings are validated against the asset's digital content. Within each manifest, the assertion store collects the assertions (including any ingredient assertions and the content binding); the claim

references those assertions by URI and is cryptographically signed by the claim signature. Green lines denote containment (one component is structurally embedded within another); blue lines denote logical references between components.

For the full structural description of each component see Manifests, Assertions, Claims, and Binding to Content.

Chapter 8. Assertions

8.1. General

It is expected that each claim generator, used by actors in the system that creates or processes an asset, will create or assemble one or more assertions about when, where, and how the asset was originated or transformed. An assertion is labelled data, typically (though not required to be) in a CBOR-based structure which represents a declaration about an asset. Some of these assertions will contain human-generated information (e.g., alternate text for accessibility) while others will come from machines (software/hardware) providing the information they generated (e.g., camera type).

Some examples of assertions are:

- metadata (e.g., camera information such as maker or lens);
- actions performed on the asset (e.g., clipping, colour correction);
- thumbnail of the asset or its ingredients;
- content bindings (e.g., cryptographic hashes).

Certain assertions may be redacted by subsequent claims (see Section 8.8, “Redaction of Assertions”), but they cannot be modified once made as part of a claim.

8.2. Labels

8.2.1. Namespacing

String values in C2PA data structures may be organized into namespaces using a period (`.`) as a separator. The C2PA namespace, `c2pa`, shall be the beginning of any string value defined in this specification. In addition to the `c2pa` namespace, the C2PA namespace registry lists other top-level namespaces.

Entity-specific namespaces, meaning any namespace other than `c2pa` or those on the registry, shall begin with the Internet domain name for the entity, beginning with a valid IANA top-level domain, similar to how Java packages are defined (e.g., `com.litware`, `net.fineartschool`).

NOTE

The purpose of the entity-specific namespace is to avoid collisions between labels defined by different entities. For example, if two different entities define a label called `metadata`, the use of that format for entity-specific namespaces allows them to be easily distinguished

> as `com.litware.metadata` and `net.fineartschool.metadata`.

The period-separated components of an entity-specific namespace shall follow the variable naming convention (`[a-zA-Z0-9][a-zA-Z0-9_-]*`) specified in the POSIX or C locale, as defined in the ABNF below (ABNF for Namespaces).

ABNF for Namespaces

```
qualified-namespace = "c2pa" / registered-namespace / entity
registered-namespace = <a namespace listed in the C2PA namespace registry>
entity = entity-component *( "." entity-component )
entity-component = 1( DIGIT / ALPHA ) *( DIGIT / ALPHA / "-" / "_" )
```

8.2.2. Label Naming

Each assertion has a label defined either by the C2PA specifications or an external entity. These labels are strings which are namespaced, as described in the preceding clause or by an entity. The most common labels will be defined in the `c2pa` namespace, but labels may use any namespace that follows the conventions. Labels are also versioned with a simple incrementing integer scheme (e.g., `c2pa.actions.v2`). If no version is provided, it is considered as `v1`. The list of publicly known labels can be found in Chapter 16, *C2PA Standard Assertions*.

> NOTE
>
> Previous versions of this document also provided for namespacing for well-established standards, but that has been superseded by simply having them via entity-specific namespaces (e.g., `org.iso`, `org.w3`).

ABNF for Assertion Labels

```
namespaced-label = qualified-namespace label
qualified-namespace = "c2pa" / entity
entity = entity-component *( "." entity-component )
entity-component = 1( DIGIT / ALPHA ) *( DIGIT / ALPHA / "-" / "_" )
label = 1*( "." label-component )
label-component = 1( DIGIT / ALPHA ) *( DIGIT / ALPHA / "-" / "_" )
```

The period-separated components of a label follow the variable naming convention (`[a-zA-Z][a-zA-Z0-9_-]*`) specified in the POSIX or C locale, with the restriction that the use of a repeated underscore character (`__`) is reserved for labelling multiple assertions of the same type.

8.3. Versioning

When an assertion's schema is changed, it should be done in a backwards-compatible manner. This means that new fields may be added and existing ones may be marked as deprecated (i.e., can be read, but never written). Existing fields shall not be removed. The label would then consist of an incremented version number, for example moving from `c2pa.actions` (deprecated) to `c2pa.actions.v2`.

Since the addition of optional fields can be done while maintaining backwards compatibility, such fields may be added to an existing assertion's schema without a change to the version number.

Deprecated fields for C2PA standard assertions shall be indicated in Chapter 16, *C2PA Standard Assertions*. Claim generators shall not insert data into deprecated assertion fields when creating assertions.

In those situations where a non-backwards compatible change is required, instead of increasing the label's version number, the assertion shall be given a new label.

> NOTE For example, `c2pa.ingredient` could be changed to the fictional `c2pa.component`.

8.4. Multiple Instances

Multiple assertions of the same type can occur in the same manifest, but since assertions are referenced by claims via their label, the assertion labels are required to be unique. This is accomplished by adding a double-underscore and a monotonically increasing index to the label. For example, if a manifest contains a single assertion of type `c2pa.metadata`, then the assertion label will be `c2pa.metadata`. If a manifest contains three assertions of this type, the labels will be `c2pa.metadata`, `c2pa.metadata__1` and `c2pa.metadata__2`.

When a label includes a version number, that version number is part of the label itself. As such, when there are multiple instances, the instance number continues to follow the label - e.g., `c2pa.ingredient.v2__2`.

8.5. Schema Validation

The schemas provided in this document, as well as the machine readable ones that can be downloaded from the C2PA website, should be used by claim generators to ensure that the assertions they create are consistent with the expected structures. These schemas can also be used as aids in understanding the syntax to be read or written. Validators should not validate inputs against these schemas.

8.6. Assertion Store

The set of assertions referenced by a claim in a manifest are collected together into a logical construct that is referred to as the *assertion store*. The assertions and assertion store shall be stored as described in Section 11.5, "Serialization"; in particular, each assertion referenced in a claim's `created_assertions` or `gathered_assertions` (but not `redacted_assertions`) shall be present in the assertion store located in the same C2PA Manifest as the claim.

In each manifest, there is a single assertion store. However, as an asset may have multiple manifests associated with it, each one representing a specific series of assertions, there may be multiple assertion stores associated with an asset.

Each assertion in the assertion store should include a randomly-generated salt in the private `c2sh` box within its JUMBF Description Box, as documented in Section 9.4.2.3, "Hashing JUMBF Boxes", so that subsequent generators can redact assertions securely.

8.7. Embedded vs Externally-Stored Data

Some assertion data, due to its size or an infrequent need for it, may be externally hosted. Such data are not embedded in the assertion store, but instead are referenced by URI. This is accomplished through either a cloud data assertion for remotely-stored C2PA assertions (JUMBF) or an external reference assertion for either remotely stored assertion or non-assertion data. In both cases, the externally hosted data is not retrieved nor validated as part of manifest validation, and is only retrieved and validated when specifically needed by an application according to a different set of validation rules as described in Section 14.10, "Validate the Assertions".

8.8. Redaction of Assertions

Assertions that are present in an asset-embedded manifest may be removed from that asset's manifest when the asset is used as an ingredient. This process is called redaction.

Redaction shall be performed either by removing the entire assertion from the manifest's assertion store or by replacing its content, as described in Section 14.10.3, "Redacted Assertion Representation".

In addition, a record that something was removed shall be added to the claim in the form of a URI reference to the redacted assertion in the `redacted_assertions` field of the claim. It is also strongly recommended that the claim generator should add a `c2pa.redacted` action with a `redacted` field as described in Section 16.4.1.4.8, "Parameters".

When redacting an ingredient assertion that references a C2PA Manifest, the associated manifest shall be removed from the C2PA Manifest Store if no other references to it remain after redacting.

> NOTE
> Because each assertion's URI reference includes the assertion label, it is also known what type of information (e.g., thumbnail, metadata, etc.) was removed. This enables both humans and machines to apply rules to determine if the removal was acceptable.

Unless the assertion redaction is accompanied by changes to the data covered by content bindings, an update manifest should be used to document the redaction, to indicate that digital content was not changed.

Claim generators shall not redact assertions with a label of `c2pa.actions` or `c2pa.actions.v2` as this assertion type represents essential information in understanding the history of an asset. When creating an update manifest, the claim generator shall not redact the hard binding to content assertion that applies to the current asset.

> NOTE
> How a validator handles redacted assertions, including the representation described in Section 14.10.3, "Redacted Assertion Representation", is described in Section 14.10, "Validate the Assertions".

8.9. Specifications of time in assertions

The default specification for a date/time value in an assertion is the date/time format which is serialized in CBOR as tag number `0` (RFC 8949, 3.4.1) and represented in CDDL as type `tdate`.

> NOTE
> CBOR Tag 0 is described in RFC 8949 as "the date-time production in RFC 3339, as refined by Section 3.3 of RFC 4287, representing the point in time described there". This means that the presence of either an explicit timezone or `Z` (for UTC) is required.

Time values in this specification have different evidentiary meanings:

- The standard use of time records the signer's claim of the time in question.
- A claimed time of signing specifically records the time (or interval) of signing, as claimed by the signer. It is not independently authoritative.
- A trusted time stamp contains evidence from a Time-Stamping Authority (TSA) about the data submitted to that TSA. Its meaning and validation are defined by the time stamp profile and do not replace the semantics of other time fields.

Claim generator provided timestamp fields use `generator-lower-bound-time-type` or `generator-upper-bound-time-type`. A value may be either a direct `tdate` or a map containing a `tdate` in `value` and an optional `timeStampType`. A direct `tdate`, or a map without `timeStampType`, shall be interpreted as `absolute`.

`absolute` indicates recorded time. `lowerBound` indicates a lower bound on the recorded time but makes no guarantee about the actual time. `upperBound` indicates an upper bound on the recorded time but makes no guarantee about the actual time. When provided on corresponding start and end fields, lower and upper bounds bracket the range within which the actual process or action occurred.

There is one case, as described when adding a claimed time of signing, where the time is represented as a CBOR tag number 1 (RFC 8949, 3.4.2) (as the number of seconds from 1970-01-01T00:00Z).

Additionally there is the time stamp assertion, which uses the standard time-stamping formats as described in the signing process.

The reason why there are different types of date & time representations is to allow for the most appropriate representation, based on existing standards in use, for each specific use case.

Chapter 9. Unique Identifiers

9.1. Uniquely Identifying C2PA Manifests and Assets

Every C2PA Manifest is uniquely identified and referenced by a Uniform Resource Name RFC 8141, URNs from the `c2pa` URN namespace, and a C2PA asset is uniquely identified by the `c2pa` URN value of its active manifest. The ABNF for the C2PA URN is described by ABNF for C2PA URN.

A `c2pa` URN shall consist of two mandatory and two optional components, in the following order, with colons (`:`) between each section.

- URN identifier (`urn:c2pa`): REQUIRED.
- UUID v4, in string representation (as per RFC 9562, UUIDs, section 4): REQUIRED.
- Claim Generator identifier string : OPTIONAL.
- Version and Reason string (as described below) : OPTIONAL.

When present, the "Claim Generator identifier" string shall consist of no more than 32 characters from the ASCII range (as per RFC 20), but which are not Control Characters (RFC 20, 5.2) or Graphic Characters (RFC 20, 5.3).

When present, the "Version and Reason" string shall consist of a positive integer, followed by an underscore (_) and then another positive integer. The details of each of these values and how they are to be used is described in Versioning Manifests Due to Conflicts. In addition, when a "Version and Reason" string is present, a "Claim Generator identifier" string shall also be present but it may be empty.

ABNF for C2PA URN

```
c2pa_urn = c2pa-namespace UUID [claim-generator [version-reason]]

c2pa-namespace = "urn:c2pa:"

; this definition is taken from RFC 9562
UUID     =  4hexOctet "-"
            2hexOctet "-"
            2hexOctet "-"
            2hexOctet "-"
            6hexOctet
hexOctet = HEXDIG HEXDIG
DIGIT    = %x30-39
HEXDIG   = DIGIT / "A" / "B" / "C" / "D" / "E" / "F"

; ASCII, but not Control Characters or Graphic Characters
```

```
visible-char-except-space = %x21-7E / %x80-FF

; claim-generator-identifier is a string of 0 to 32 visible-char-except-space characters
; this means that an empty string is valid
claim-generator = ":" claim-generator-identifier
claim-generator-identifier = 0*32visible-char-except-space

; version-reason is a string consisting of a positive integer
; followed by an underscore and a positive integer
version-reason = ":" version "_" reason
version = 1*DIGIT
reason = 1*DIGIT
```

Examples:

- `urn:c2pa:F9168C5E-CEB2-4FAA-B6BF-329BF39FA1E4`
- `urn:c2pa:F9168C5E-CEB2-4FAA-B6BF-329BF39FA1E4:acme`
- `urn:c2pa:F9168C5E-CEB2-4FAA-B6BF-329BF39FA1E4:acme:2_1`
- `urn:c2pa:F9168C5E-CEB2-4FAA-B6BF-329BF39FA1E4::2_1`

NOTE

Previous versions of this specification used RFC 9562, UUIDs URN, and had the identifier of the claim generator at the beginning of the URN. However, this was found to be not in compliance with either RFC 9562, UUIDs or RFC 8141, URNs.

This `c2pa` URN identifier is used in various parts of a C2PA-enabled workflow, such as when identifying an asset as an ingredient in a derived or composed asset.

9.2. Versioning Manifests Due to Conflicts

Situations may arise where it is necessary to re-label a C2PA Manifest due to a conflict of identifiers. For example, if a claim generator had already added an ingredient manifest into the asset's C2PA Manifest Store, then later added another ingredient which had a manifest with the same label in its manifest store, but this latter version of the manifest was different due, for example, to a manipulation of one of its assertion values. In such a case, the modified version of the ingredient manifest needs to be copied into the asset's C2PA Manifest Store, and shall be re-labeled.

To re-label a manifest:

- If the current URN does not contain a "Claim Generator identifier string", then the claim generator shall append a `:`.

- In all cases, the claim generator shall append a `:` to the URN followed by a monotonically increasing integer, starting with 1, followed by an underscore (_) and then an integer from the list below representing the reason for the re-labeling.
    - 1: Conflict with another C2PA Manifest

For example, if the claim generator has to re-label a C2PA Manifest for the second time due to a conflict, the appended string would be `:2_1`.

9.3. Identifying Non-C2PA Assets

When working with assets that do not contain a C2PA Manifest, but the asset contains embedded XMP which include values for `xmpMM:DocumentID` and/or `xmpMM:InstanceID` as defined in XMP Specification Part 2, 2.2, those values shall be used as identifiers for the asset.

When working with assets that do not contain a C2PA Manifest and do not contain embedded XMP, the claim generator may use any method of its choosing to provide it with a unique identifier.

9.4. URI References

9.4.1. Standard URIs

All references to information in the manifest, whether stored internally to the asset (i.e., embedded) or stored externally to the asset (e.g., in the cloud), shall be referenced via JUMBF URI references as defined in ISO 19566-5:2023, C.2. These URIs are normally used either as part of a `hashed_uri` or `hashed_ext_uri` data structure.

> NOTE
>
> Section 16.4.1.4.6, “Unique Identifiers for Actions” describes an extension to standard JUMBF URIs to allow for the referencing of individual fields within a CBOR Content Type Box. This extension is only for use in that context, and is not a general extension to JUMBF URIs.

When the reference is to a compressed manifest, the JUMBF URI shall not contain anything about the `brob` box, but the URI to the manifest is treated as if the manifest was not compressed. This means that the URI would include the label of the `c2ma` or `c2um` box, but not the label of the `c2cm` box. In addition, the URI reference to a compressed manifest shall not include the label of the `brob` box - but only the label of the compressed manifest itself.

When resolving an internal JUMBF URI reference, if any label in the path is ambiguous due to multiple child

http(s) headers. If present, this field shall be used as the required format retrieved during any content negotiate/request. Sometimes, it may also be necessary to provide one or more asset types as the value of the `data_types` field for more clarity on the format and usage of that data.

An optional `size` field is also provided to specify the size of the data to be retrieved. This may be useful to a validator as a hint in addition to the hash.

> NOTE
>
> It could be used to provide information about whether to perform downloading or validation or both.

9.4.2.3. Hashing JUMBF Boxes

When creating a URI reference to any JUMBF box (e.g., assertions and data boxes), the hash shall be performed over the contents of the structure's JUMBF superbox, which includes both the JUMBF Description Box and all content boxes therein (but does not include the structure's JUMBF superbox header).

> NOTE
>
> More details on hashing can be found at Section 12.1, "Hashing".

As described in the latest version of JUMBF (ISO 19566-5:2023), and shown in Figure 5, "Example `c2pa.actions` assertion", a new `Private` field can be present as part of any JUMBF Description box. This C2PA specification defines the C2PA salt as a `Private` field whose value is a standard box consisting of:

- a box length (LBox, as a 4-byte big-endian unsigned integer);
- a box type (TBox, 4-byte big-endian unsigned integer, with a value of `c2sh` (for C2PA salt hash));
- and payload data (consisting of randomly-generated binary data of either 16 or 32 bytes in length).

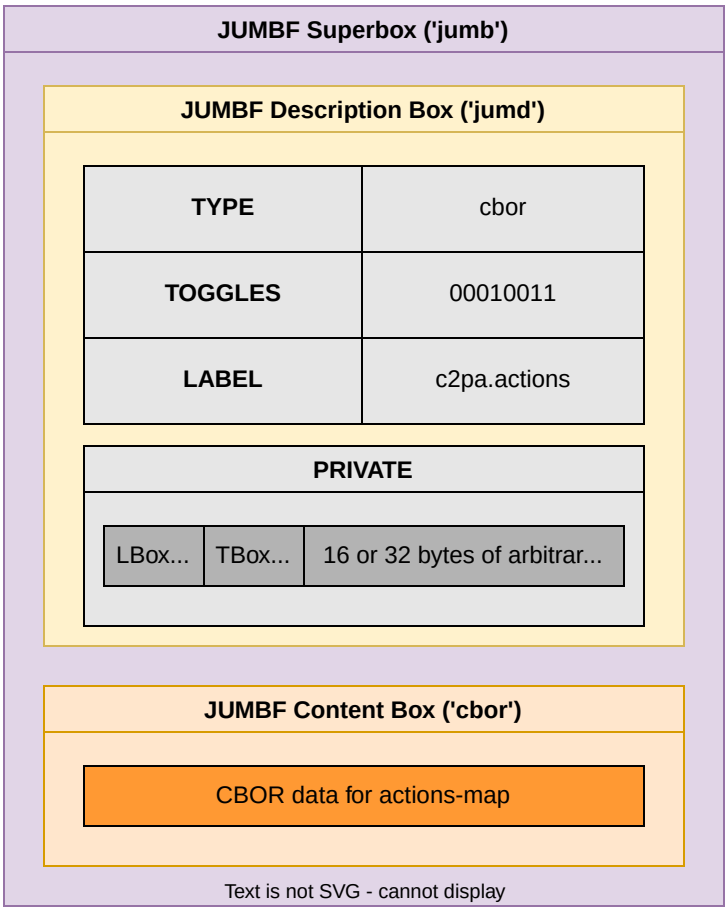


Figure 5. Example `c2pa.actions` assertion

```
  ; List of JUMBF URI references to the assertions of ingredient manifests being
  ; redacted
  ? "redacted_assertions": [1* jumbf-uri-type],

  ; A string identifying the cryptographic hash algorithm used to compute all
  ; data hash assertions listed in this claim unless otherwise overridden, taken
  ; from the C2PA data hash algorithm identifier registry. This provides the
  ; value for the 'alg' field in data-hash and hashed-uri structures contained
  ; in this claim
  ? "alg": tstr .size (1..max-tstr-length),

  ; A string identifying the algorithm used to compute all soft binding
  ; assertions listed in this claim unless otherwise overridden, taken from the
  ; C2PA soft binding algorithm identifier registry."
  ? "alg_soft": tstr .size (1..max-tstr-length),

  ; (DEPRECATED) The version of the specification used to produce this claim
  ; (SemVer)
  ? "specVersion": semver-string,

  ; (DEPRECATED) additional information about the assertion
  ? "metadata": $assertion-metadata-map,
}

; (DEPRECATED) CDDL schema for a claim map in C2PA
claim-map = {
  ; A User-Agent string formatted as per
  ; http://tools.ietf.org/html/rfc7231#section-5.5.3, for including the name and
  ; version of the claims generator that created the claim
  "claim_generator": tstr,
  "claim_generator_info": [1* generator-info-map],

  ; JUMBF URI reference to the signature of this claim
  "signature": jumbf-uri-type,
  "assertions": [1* $hashed-uri-map],
  "dc:format": tstr, ; media type of the asset

  ; uniquely identifies a specific version of an asset
  "instanceID": tstr .size (1..max-tstr-length),
  ? "dc:title": tstr .size (1..max-tstr-length), ; name of the asset

  ; List of JUMBF URI references to the assertions of ingredient manifests being
  ; redacted
  ? "redacted_assertions": [1* jumbf-uri-type],

  ; A string identifying the cryptographic hash algorithm used to compute all
  ; data hash assertions listed in this claim unless otherwise overridden, taken
  ; from the C2PA data hash algorithm identifier registry. This provides the
  ; value for the 'alg' field in data-hash and hashed-uri structures contained
  ; in this claim
  ? "alg": tstr .size (1..max-tstr-length),

  ; A string identifying the algorithm used to compute all soft binding
```

```
  ; assertions listed in this claim unless otherwise overridden, taken from the
  ; C2PA soft binding algorithm identifier registry."
  ? "alg_soft": tstr .size (1..max-tstr-length),

  ; additional information about the assertion
  ? "metadata": $assertion-metadata-map,
}

generator-info-map = {
  ; A human readable string naming the claim generator
  "name": tstr .size (1..max-tstr-length),
  ? "version": tstr, ; A human readable string of the product's version

  ; hashed URI to the icon (either embedded or remote)
  ? "icon": $hashed-uri-map / $hashed-ext-uri-map,

  ; A human readable string of the operating system the claim generator is
  ; running on
  ? "operating_system": tstr,

  ; The version of the specification used to produce this manifest (SemVer)
  ? "specVersion": semver-string,
  * tstr => any
}
```

An example of the `claim-map-v2` structure in CBOR diagnostic notation (RFC 8949, clause 8):

```
{
  "instanceID": "urn:uuid:5E7B01FC-4932-4BAB-AB32-D4F12A8AA322",
  "alg" : "sha256",
  "claim_generator_info" : {
    "name": "Acme C2PA SDK",
    "version": "2.0"
  },
  "signature" : "self#jumbf=c2pa.signature",
  "created_assertions" : [
    {
      "url": "self#jumbf=c2pa.assertions/c2pa.hash.data",
      "hash": b64'U9Gyz05tmpftkoEYP6XYNsMnUbnS/KcktAg2vv7n1n8='
    },
    {
      "url": "self#jumbf=c2pa.assertions/c2pa.thumbnail.claim",
      "hash": b64'G5hfJwYeWTlflxOhmfCO9xDAK52aKQ+YbKNhRZeq92c='
    },
    {
      "url": "self#jumbf=c2pa.assertions/c2pa.ingredient.v3",
      "hash": b64'Yzag4o5jO4xPyfANVtw7ETlbFSWZNfeM78qbSi8Abkk='
    }
  ],
  "redacted_assertions" : [
    "self#jumbf=/c2pa/urn:c2pa:5E7B01FC-4932-4BAB-AB32-
```

```
D4F12A8AA322/c2pa.assertions/c2pa.metadata"
  ]
}
```

10.2.2. Fields

The `alg` field shall contain the hash algorithm used to compute the hash for any hashed-uri, hard binding assertion, or hash assertion that does not explicitly define it. The value of this field, if present, shall be a string that is one of the allowed hash algorithms as defined in Section 12.1, "Hashing". If the `alg` field is not present, then each hashed-uri, each hard binding assertion, and each hash assertion shall explicitly declare which hashing algorithm is to be used.

If present, the value of `dc:title` shall be a human-readable name for the asset. However, instead of setting `dc:title`, claim generators should create a metadata assertion containing the `dc:title` field.

NOTE	The `c2pa.claim` has a `dc:format` field which is no longer present in `c2pa.claim.v2`.

If the asset contains XMP, then the asset's `xmpMM:InstanceID` should be used as the `instanceID`. When no XMP is available, then some other unique identifier for the asset shall be used as the value for `instanceID`.

NOTE	Some field names, such as `dc:title`, have namespace prefixes as their names and definitions are taken directly from the XMP standard. However, their usage in C2PA does not require the use of XMP.

The `signature` field shall be present and it shall contain an absolute URI reference to the claim signature in the same C2PA Manifest.

The `created_assertions` field shall be present and it shall contain one or more URI references to assertions being made by this claim. In a standard manifest, it shall contain, at minimum, a reference to an assertion that represents a hard binding.

NOTE	All `created_assertions` are attributed to the signer as the Trust Model is rooted in the trust of the signer.

When present, the `gathered_assertions` field shall contain one or more URI references to assertions that have been provided to the claim generator by other components in the workflow.

NOTE

By putting an assertion into this list, the claim generator is declaring that the assertion is part of the claim, but it was not sourced from the claim generator and is not attributed to the signer. For example, assertions containing information entered by a human actor or external references would be listed in `gathered_assertions`.

When present, the `redacted_assertions` field shall contain one or more URI references to redacted assertions.

IMPORTANT

Previous versions of this specification had a `specVersion` field in the claim, but that field has been moved to the `claim_generator_info` field as described in Section 10.2.3, "Claim Generator Info". The `specVersion` field in the claim is now deprecated.

10.2.3. Claim Generator Info

10.2.3.1. General

Detailed information about the claim generator shall be present as the value of `claim_generator_info`. A Manifest Consumer shall use the value of `claim_generator_info` in determining information about the claim generator for itself or for presentation in a UX.

NOTE

The `c2pa.claim` has a `claim_generator` field, whose value is a simple string, which is no longer present in `c2pa.claim.v2`.

10.2.3.2. Generator Info Map

When adding a `claim_generator_info` field, its value is a `generator-info-map` object which shall contain a `name` field. It may also contain either a `version` field or an `icon` field or both. In addition, any other field is permitted, using the standard entity-specific namespacing described in Section 8.2.1, "Namespacing". The data in this object shall represent the non-human (hardware or software) actor that actually generated the claim (aka the claim generator itself).

A claim generator may desire to provide a graphical representation of itself, referred here as an `icon`, to a Manifest Consumer that is presenting a user experience. The value of the `icon` field, if present, shall be a hashed URI. This hashed URI shall be to an embedded data assertion whose label is `c2pa.icon` and follows the rules of assertion labels with respect to multiple instances. Manifest Consumers should also support the data box approach recommended by earlier versions of this specification.

> NOTE
>
> As with the assertions array, the hash algorithm used for a hashed URI is determined by the `alg` field present in the hashed URI, or when absent, by an `alg` field in the claim.

A `specVersion` field should be present, and if so, shall contain a SemVer formatted `specVersion` field representing the version of this specification used as the normative reference by the Claim Generator to create this C2PA Manifest (e.g., "2.4.0" for version 2.4). Validators should treat this field as purely informational and should not change their validation logic based on this value.

Example using claim generator info

```
{
    "claim_generator_info" : {
        "name": "Acme C2PA SDK",
        "version": "2.0",
        "icon": {
            "url": "http://cdn.examplephotoagency.com/logo.svg",
            "hash": "5bdec8169b4e4484b79aba44cee5c6bd"
        },
        "specVersion": "2.4.0"
    }
}
```

10.3. Creating a Claim

10.3.1. Creating Assertions

Before the claim can be finalized, all assertions need to be created and stored in a newly created C2PA Assertion Store as described later in this document.

When creating a standard manifest, it may not be possible to know all of the required binding information at the time of claim creation, in which case use the multiple step processing method to setup and then later fill-in the information.

10.3.2. Preparing the Claim

10.3.2.1. Adding Assertions and Redactions

The claim shall contain a `created_assertions` field and may contain a `gathered_assertions` field. The combined values from those two fields represent a list of all of the URI references for all assertions that were added to the assertion store that are being "claimed" by this claim. In a standard manifest, the

`created_assertions` field's value shall include at least one assertion that represents a hard binding.

If any assertions in ingredient claims are being redacted, their URI references shall be added to list which is the value of the `redacted_assertions` field.

10.3.2.2. Adding Ingredients

In many authoring scenarios, an actor does not create an entirely new asset but instead brings in other existing assets on which to create their work - either as a derived asset, a composed asset or an asset rendition. These existing assets are called ingredients and their use is documented in the provenance data through the use of an ingredient assertion.

When an ingredient contains one or more C2PA manifests, those manifests should be inserted into this asset's C2PA Manifest Store to help ensure that the provenance data is kept intact. Such ingredient manifests are added to the JUMBF as described in Section 11.4, "Ingredient Storage". If a manifest with the same unique identifier is already present in the C2PA Manifest Store, the two shall be compared. If they are identical, the new manifest shall be ignored. If they are different, the new manifest shall be added to the store after changing its unique identifier to a new value as described in Chapter 9, *Unique Identifiers*.

If an ingredient's manifest is remote, and the claim generator is unable to retrieve the manifest, it should use an error code of `manifest.inaccessible` to reflect that.

10.3.2.3. Connecting the Signature

The signature cannot be part of the signed payload, but since its label is pre-defined, then the full URI reference is also known. As such, we can include that in the claim by setting the value of the `signature` field of the claim to that URI reference.

> NOTE | This provides the explicit binding of the claim to its signature.

10.3.2.4. Signing a Claim

Producing the signature is specified in Section 12.3, "Digital Signatures".

For both types of manifests, standard and update, the `payload` field of `Sig_structure` shall be the serialized CBOR of the claim document, and shall use detached content mode.

The serialized `COSE_Sign1_Tagged` structure resulting from the digital signature procedure is written into the C2PA Claim Signature box.

10.3.2.5. Time Stamps

10.3.2.5.1. Use of RFC 3161

If possible, the claim generator should use a RFC 3161-compliant Time Stamping Authority (TSA) (RFC 3161) to obtain a trusted time stamp proving that the signature itself actually existed at a certain date and time and incorporate that into the `COSE_Sign1_Tagged` structure as a countersignature.

Claim generators are encouraged to obtain and include time stamps to ensure their manifests will remain valid. As described in Chapter 14, *Validation*, manifests without time stamps cease to be valid when the signing credential expires or becomes revoked. A manifest shall contain only one time stamp.

> NOTE
>
> Previous versions of this specification allowed for multiple time stamps to be included in a manifest.

10.3.2.5.2. Choosing the Payload

A previous version of this specification used the same value for the `payload` field in the time stamp as was used in the `Sig_signature` as described in Section 10.3.2.4, “Signing a Claim”. This payload is henceforth referred to as a "v1 payload" in a "v1 time stamp" and is considered deprecated. A claim generator shall not create one, but a validator shall process one if present.

The "v2 payload", of the "v2 time stamp", is the value of the `signature` field of the `COSE_Sign1_Tagged` structure created as part of Section 10.3.2.4, “Signing a Claim”. A "v2 payload" shall be used by claim generators performing a time stamping operation.

> NOTE
>
> The value of the `signature` field includes the entire serialized `bstr`, including the bytes that indicate the major type and the length (not just the string itself).

10.3.2.5.3. Obtaining the time stamp

All time stamps shall be obtained as described in RFC 3161 with the following additional requirements:

- The `MessageImprint` of the `TimeStampReq` structure (RFC 3161, section 2.4.1) shall be computed by creating the `ToBeSigned` value in RFC 9052, section 4.4, with the following values for elements of `Sig_structure`:
  - The `context` element shall be `CounterSignature`.
  - The `payload` element shall be the value described by Section 10.3.2.5.2, “Choosing the Payload”.

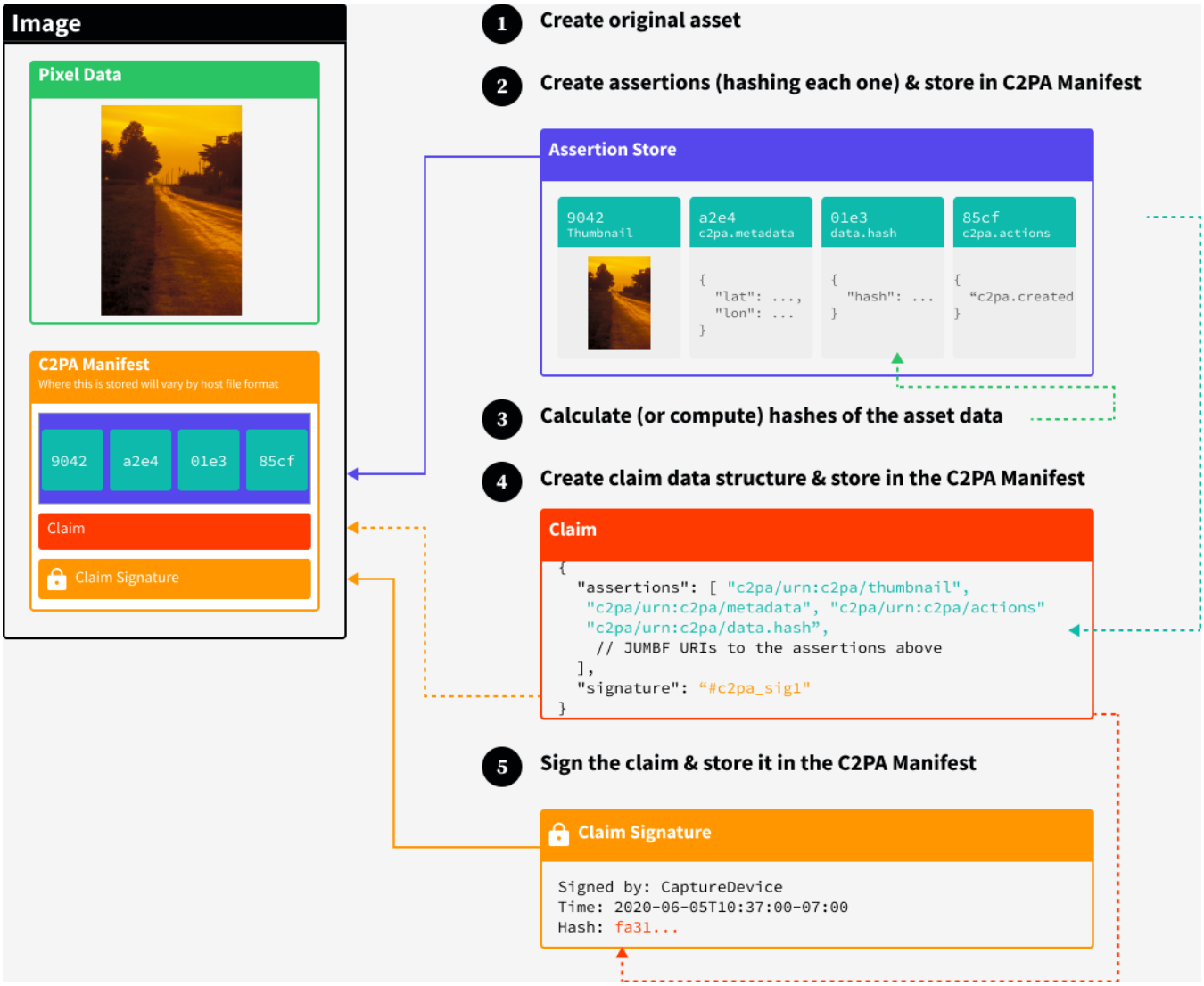


Figure 6. A single claim with assertions

10.3.3.2. Multiple Claims

In this example of creating a second claim for the previous example, one of the original assertions has been redacted from the previous claim. The visual representation for this scenario would look like:

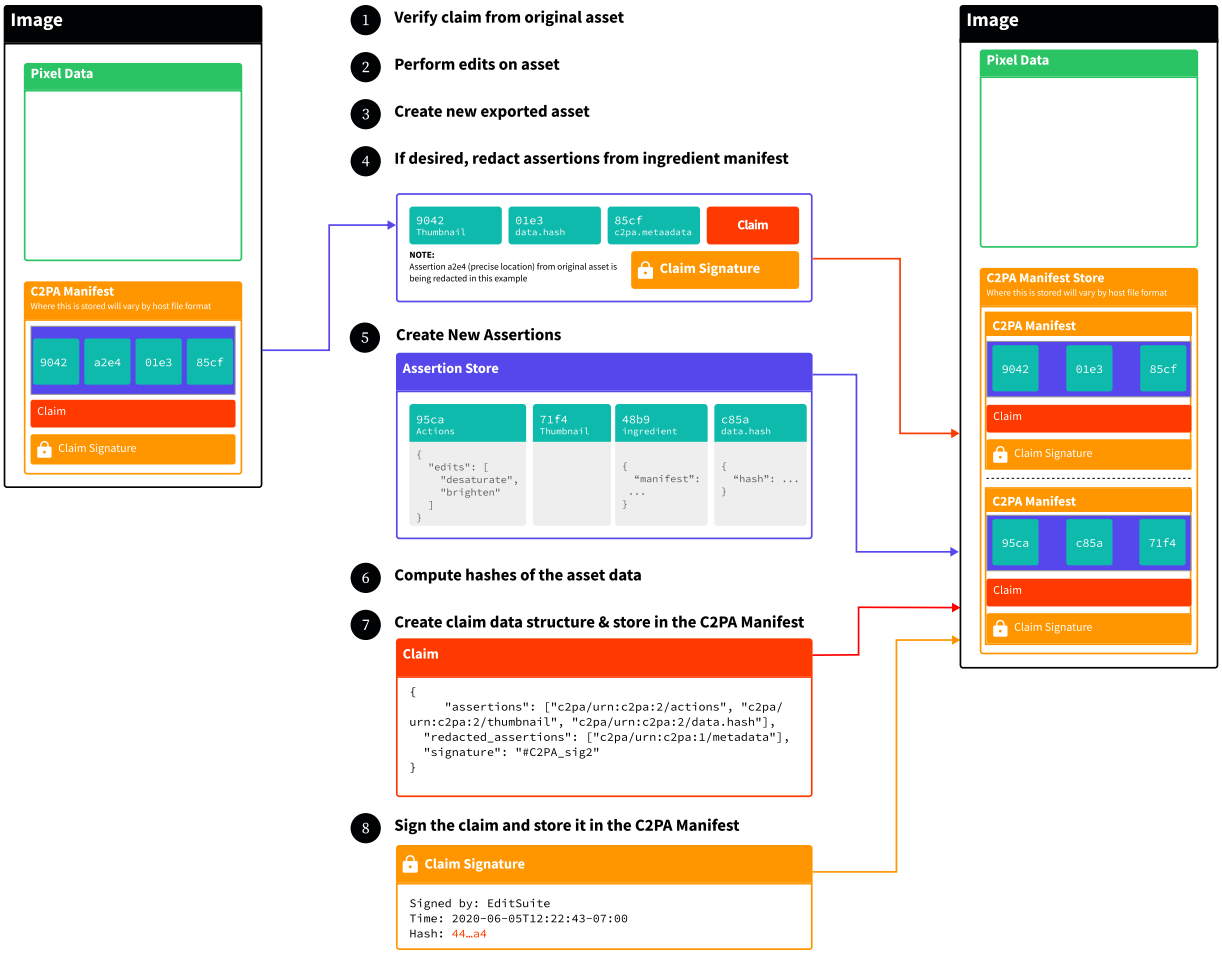


Figure 7. Redacting assertions in a secondary claim

Chapter 11. Manifests

11.1. Overview

Within C2PA, individual statements about an asset are declared as assertions. A C2PA Manifest is a container that bundles together an assertion store holding those assertions, a claim that hashes the assertions and provides information about the claim generator, and a claim signature over that claim. The C2PA Manifest that describes the asset is called the active manifest. Because assets can be built from ingredients that have their own manifests, there may be multiple manifests associated with the asset. These C2PA Manifests are collected into a single container called the C2PA Manifest Store.

A standard C2PA Manifest includes a hard binding that cryptographically associates the C2PA Manifest with the asset.

> NOTE
>
> This means that copying or linking a C2PA Manifest to a different asset is detectable and will fail validation.

The technical details of how the various parts of a C2PA Manifest are serialized can be seen in Section 11.5, “Serialization”.

11.2. Manifest Store

A C2PA Manifest Store contains one or more C2PA Manifests, the last of which is designated the active manifest.

The C2PA Manifest Store is a JUMBF (ISO 19566-5:2023) superbox. See Section 11.5, “Serialization” for more details.

Figure 8, “C2PA Manifest Store” is an example C2PA Manifest Store with a single C2PA Manifest:

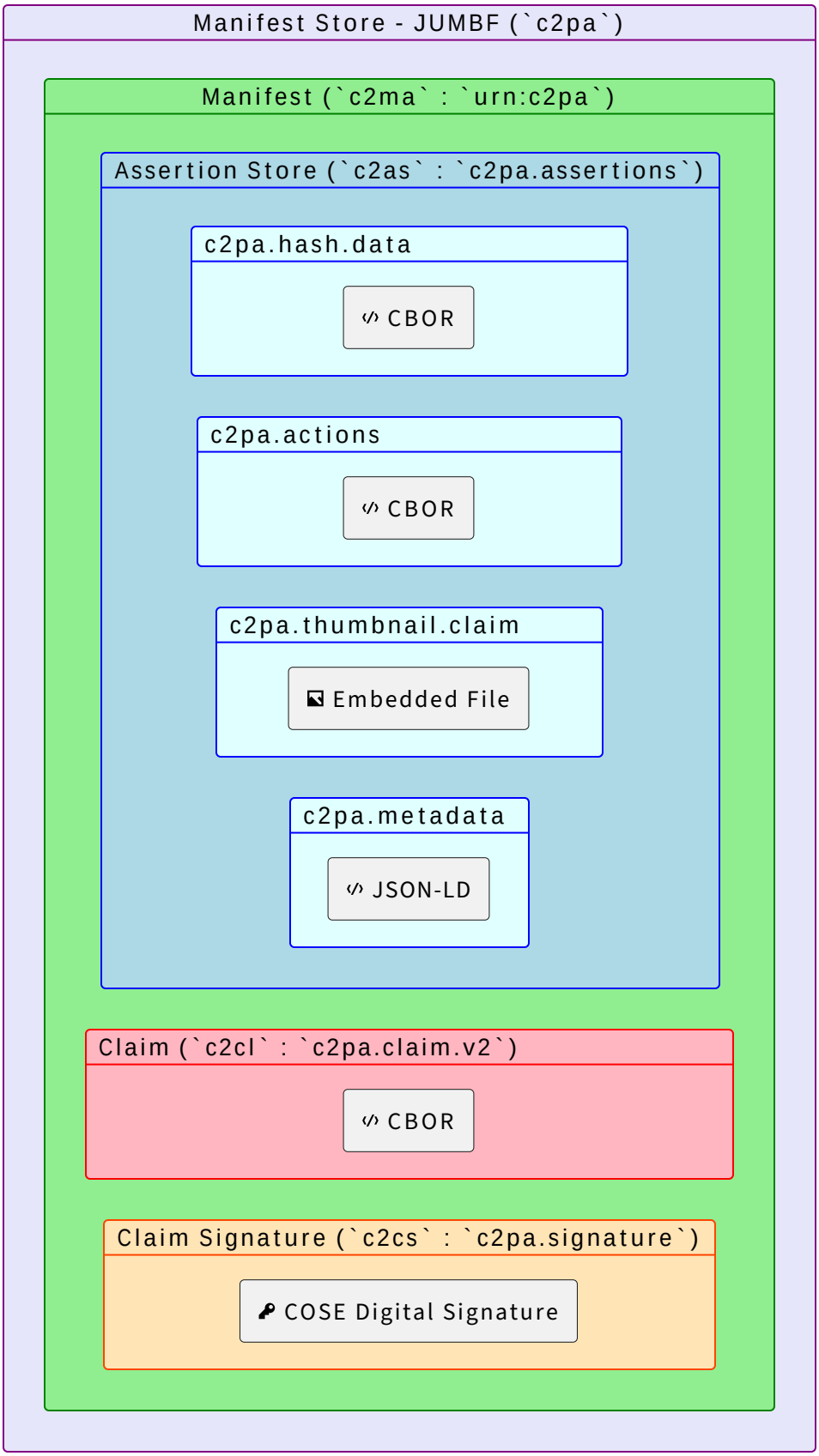


Figure 8. C2PA Manifest Store

11.3. Types of Manifests

11.3.1. Common

All C2PA Manifests shall contain an assertion store, a claim and a claim signature.

11.3.2. Standard Manifests

A standard C2PA Manifest shall contain exactly one hard binding to content assertion.

11.3.3. Update Manifests

In a workflow where additional assertions need to be added to an existing C2PA Manifest, but no changes are made to the digital content or other data covered by content bindings, an Update Manifest should be used.

An Update Manifest shall not contain assertions of types `c2pa.hash.data`, `c2pa.hash.boxes`, `c2pa.hash.collection.data`, `c2pa.hash.segment`, `c2pa.hash.bmff.v3`, or `c2pa.hash.multi-asset`, because the content has not changed and therefore the bindings need not be updated.

In cases where the existing content binding (e.g., a data hash) identifies the C2PA Manifest Store location using a file offset and length, the C2PA Manifest Store shall start at the same file offset after updating, but its length may change.

An Update Manifest may contain assertions of type `c2pa.actions` or `c2pa.actions.v2`, provided that the value of the `action` field of each action present in the `actions` array of these assertions shall only be one of the following values:

- `c2pa.edited.metadata`
- `c2pa.opened`
- `c2pa.published`
- `c2pa.redacted`

An Update Manifest shall not contain a thumbnail assertion.

> NOTE
>
> The reason for these requirements is that an `action` field outside of this list or a thumbnail implies changes to the digital content.

The Update Manifest shall contain exactly one `c2pa.ingredient.v3` assertion that (a) includes both `activeManifest` and `claimSignature` fields with values that are the URI references to the C2PA Manifest and Claim Signature respectively of the asset that is being updated and (b) has the value of `parentOf` for the `relationship` field.

> NOTE
> The ingredient's C2PA Manifest can be either a standard manifest or an update manifest.

11.4. Ingredient Storage

When a C2PA Manifest includes ingredient assertions, and an ingredient contains a C2PA Manifest Store, the C2PA Manifests it contains are normally included in the C2PA Manifest Store of the new asset to ensure that the provenance data is kept intact, as described in Section 10.3.2.2, "Adding Ingredients".

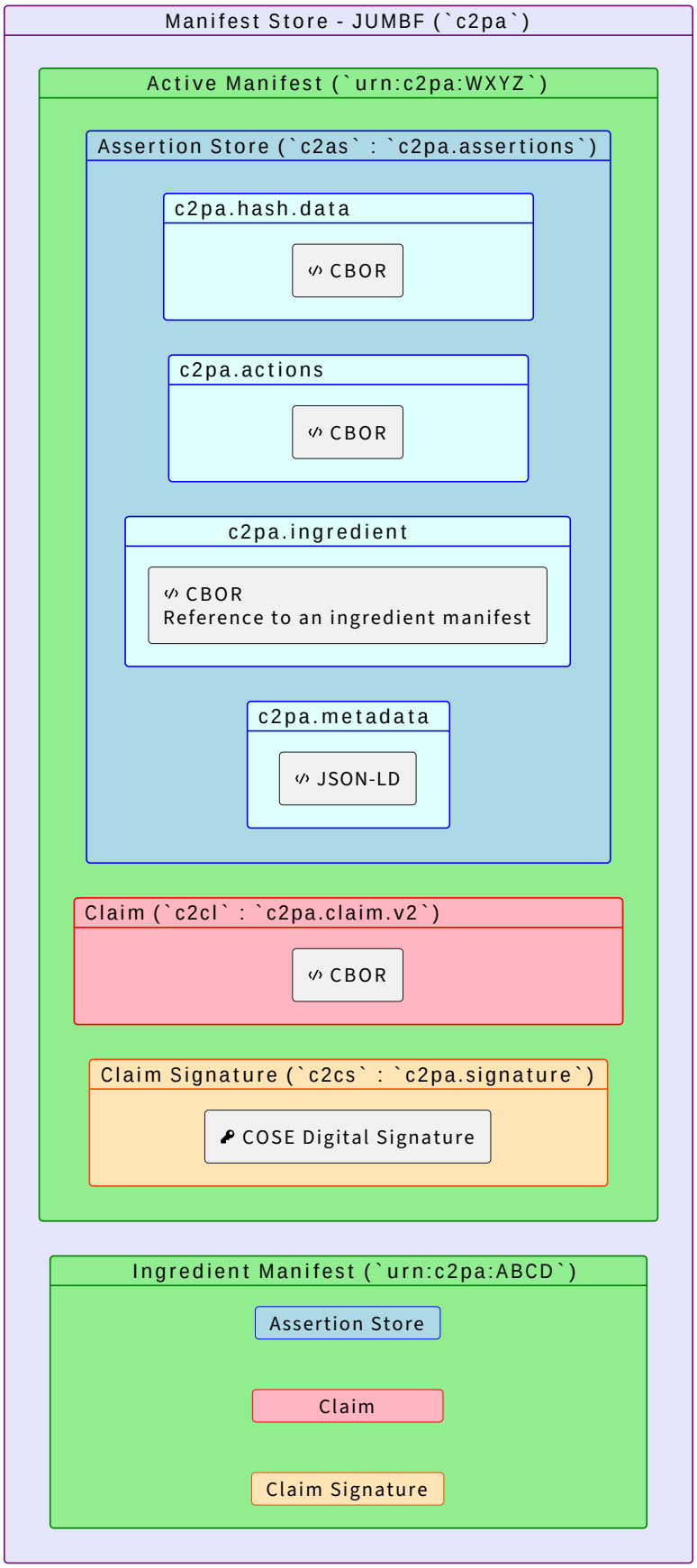


Figure 9. C2PA Manifest Store With an Ingredient

Item	JUMBF type UUID	JUMBF Label	Contents
C2PA Assertion Store	63326173-0011-0010-8000-00AA00389B71 (c2as)	c2pa.assertions	See Assertion Store Contents
C2PA Claim	6332636C-0011-0010-8000-00AA00389B71 (c2cl)	c2pa.claim.v2	A single CBOR Content Type box (cbor) with contents as described in Claim
C2PA Claim Signature	63326373-0011-0010-8000-00AA00389B71 (c2cs)	c2pa.signature	A single CBOR Content Type box (cbor) with contents as described in Claim Signature

The boxes defined in Table 2, “C2PA JUMBF boxes” shall satisfy the requirements in Section 11.5.3.2, “Toggles” and Section 11.5.3.3, “Labels”. Manifest Consumers shall accept c2md superboxes as standard C2PA Manifests, but claim generators shall not create such manifests.

11.5.1. Assertion Store Contents

The C2PA Assertion Store is a JUMBF superbox that shall have the label and type given in Table 2, “C2PA JUMBF boxes”. It shall contain one or more JUMBF superboxes (called C2PA Assertion boxes) whose JUMBF type defines the type of the sub-boxes that contain the assertion data (ISO 19566-5:2023, Annex B). These superboxes shall each have a label as defined in Standard Assertions and shall contain a JUMBF Description Box, one or more JUMBF Content Boxes and possibly a Padding Box (ISO 19566-5:2023, A.4).

The JUMBF Content Type (ISO 19566-5:2023, Annex B) box(es) contained in each assertion superbox should be CBOR Content Type (cbor), JSON Content Type (json), Embedded File Content Type (bfdb & bidb) or UUID Content Type (uuid) though any Content Type defined in JUMBF (ISO 19566-5:2023) and its amendments is permitted. In addition, a JUMBF Protection Box as described in ISO 19566-4:2020 may also be used.

> NOTE
> Custom assertions containing other formats/serializations of data, such as encrypted data, are supported through the use of a UUID Content Box containing the custom UUID followed by the data (ISO 19566-5:2023, B.5).

11.5.2. Compressed Manifests

Both standard and update C2PA Manifests can be compressed, in their entirety. To support this, C2PA defines a new brob JUMBF content box, based on a similar box in JPEG-XL (ISO/IEC 18181-2:2024). The brob box

shall have box ID of `0x62726F62` (`brob`).

For either type of manifest, the value of the `TYPE` field shall be `c2cm`, the value of the label field shall be identical to the label of the compressed manifest superbox, and the contents of the `brob` content box shall be the RFC 7932 (Brotli-compressed) bytes of the entire manifest superbox. See Figure 10, “Example of a compressed manifest” for an example of a compressed standard manifest.

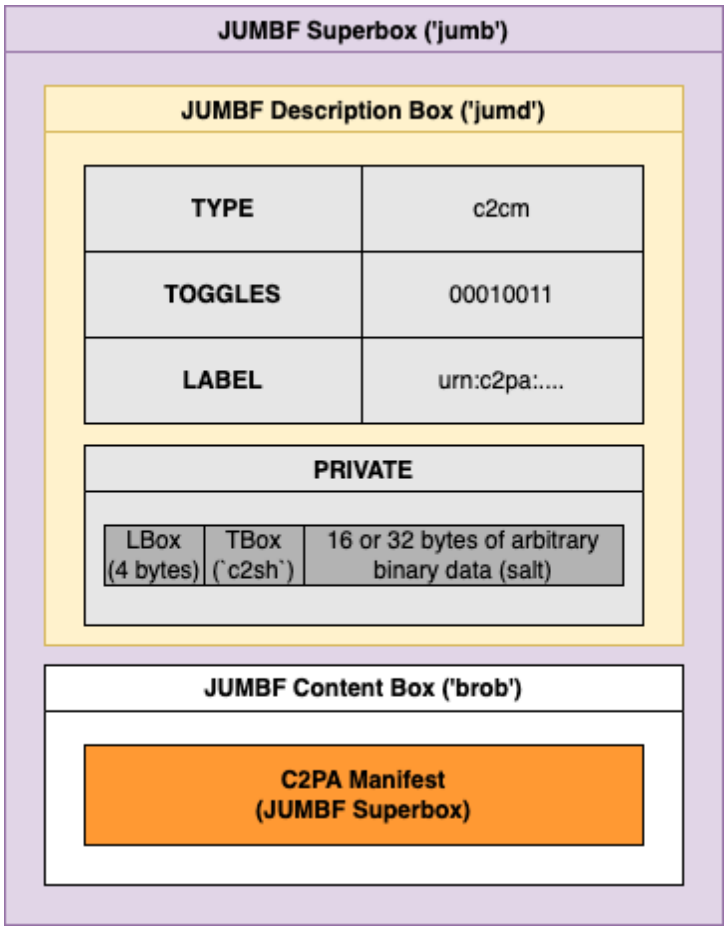


Figure 10. Example of a compressed manifest

Hashing a compressed box is done in the same way as any other box, as described in Section 9.4.2.3, “Hashing JUMBF Boxes”.

NOTE

This implies that given a `hashed_uri` reference from an ingredient assertion to a C2PA Manifest via the `activeManifest` field, the hash is computed using the same process as any other JUMBF superbox: over the JUMBF Description Box and the `brob` box with its compressed payload, but excluding the superbox’s header. The contents of the `brob` box are not decompressed first to compute the hash.

> **IMPORTANT** Any place in this specification that a standard or update manifest is referenced, a compressed standard or update manifest is also valid.

11.5.3. General C2PA JUMBF Requirements

11.5.3.1. Processing Rules

A C2PA Manifest Consumer shall never process an assertion, assertion store, claim, claim signature or C2PA Manifest that is not contained inside of a C2PA Manifest Store. Additionally, when a C2PA Manifest Consumer encounters a JUMBF box or superbox whose JUMBF type UUID it does not recognize, it shall skip over (and ignore) its contents.

> **NOTE** This means that the C2PA Manifest Consumer can process private boxes that it knows about, but ignore ones of which it is unaware.

11.5.3.2. Toggles

All JUMBF Description boxes (ISO 19566-5:2023, A.3) used in a C2PA Manifest require a label and so the *Label Present* toggle (`xxxxxx1x`) shall be set. In addition, because JUMBF URIs are used to refer to boxes throughout the system (e.g., listing assertions, references to ingredients, etc.), the *Requestable* toggle (`xxxxxx11`) shall be set.

When including a salt in a *PRIVATE* box as described in Section 9.4.2.3, “Hashing JUMBF Boxes”, the *Private* toggle (`xxx1xxxx`) shall also be set.

11.5.3.3. Labels

As described in the JUMBF specification (ISO 19566-5:2023, A.3), a label shall be stored as Unicode characters in the UTF-8 encoding and shall be null-terminated. Characters in the ranges U+0000 to U+001F inclusive and U+007F to U+009F inclusive, as well as the specific characters '/', ';', '?', and '#', are not permitted in the label.

Because labels are used as part of JUMBF URIs, the characters U+FEFF, U+FFFF, and U+D800-U+DFFF shall also not be used.

11.5.3.4. Unrecognized Boxes

The C2PA Manifest Store (`c2pa`) and C2PA Manifest (`c2ma`, `c2um`, `c2cm`, and `c2md`) may also contain JUMBF boxes and superboxes that satisfy the requirements in Section 11.5.3.2, “Toggles” and Section 11.5.3.3,

"Labels" and whose JUMBF type UUIDs are not defined in this specification.

NOTE	Allowing other boxes and superboxes enables custom extensions to C2PA as well as enabling the addition of new boxes in future versions of this specification without breaking compatibility.

If the *Requestable* and *Label Present* toggles are both set in the JUMBF Description box of any JUMBF box or superbox, that box or superbox shall be maintained in any updated C2PA Manifest Store.

NOTE	Boxes with those toggles set are intended to be referenced via JUMBF URIs, and their removal could cause downstream workflows to fail.

11.5.4. JUMBF Rationale

In order to support many of the requirements of C2PA, C2PA Manifests needed to be stored (serialized) into a structured binary data store that enables some specific functionality including:

- Ability to store multiple manifests (e.g., parents and ingredients) in a single container.
- Ability to refer to individual elements (both within and across manifests) via URIs.
- Ability to clearly identify the parts of an element to be hashed.
- Ability to store pre-defined data types used by C2PA (e.g., JSON and CBOR).
- Ability to store arbitrary data formats (e.g., XML, JPEG, etc.).

In addition to supporting all of the requirements above, our chosen container format - ISO 19566-5:2023 (JUMBF) - is also natively supported by the JPEG family of formats and is compatible with the box-based model (i.e., ISO/IEC 14496-12, ISO BMFF) used by many common image and video file formats. Using JUMBF enables all the same benefits (and a few extras, such as URI References) while being able to work with classic image formats, such as JPEG/JFIF and PNG as well as 3D and document (e.g., PDF) formats.

NOTE	Since most of the standard assertions, as well the claim signature, are serialized as CBOR, using CBOR for the entire C2PA Manifest was considered but not chosen because CBOR is not a container format. For example, to store a "blob of JSON" inside of CBOR, and know that it is JSON (and not some other format) would necessitate designing a data structure for storing such things. Then the parent structure would need to be defined as to how to carry that structure. This

same concept would also have to be done for each of the native features of JUMBF.

While it would certainly be possible to re-implement all of the required functionality entirely in CBOR, it would be a lot of work and would not fully remove the need for a JUMBF/BMFF parser in all implementations.

11.6. Embedding

A C2PA Manifest Store can be embedded into a variety of file formats covering media types including images, videos, audio, fonts, and documents. Appendix A, *Embedding manifests* provides the technical details on how to embed C2PA Manifests into each specifically supported file format.

NOTE

Many classic image formats such as BMP do not support the embedding of arbitrary data, so the use of an external manifest is required.

11.7. Multiple Step Processing

Some asset file formats require file offsets of the C2PA Manifest Store and asset content to be fixed before the manifest is signed, so that content bindings will correctly align with the content they authenticate. Unfortunately, the size of a manifest and its signature cannot be precisely known until after signing, which could cause file offsets to change.

As an example, in JPEG 1 files, the entire C2PA Manifest Store is required to appear in the file before the image data, and so its size will affect the file offsets of content being authenticated.

To accomplish this, a multiple step approach shall be taken, similar to how signatures in PDF are done.

11.7.1. Create content bindings

When creating a standard manifest, its claim shall include one or more content binding assertions in its list of assertions to ensure that the asset is tamper-evident.

Create the data hash assertion and add it to the assertion store taking into account the following considerations.

In many cases, such as with JPEG 1, it is not possible to hash the asset in its entirety because the manifest will be embedded in the middle of the file, so the size or location of manifest data will not be known at the time the asset hash is computed. This circular dependency is avoided by allowing exclusion ranges to be specified

during hashing. When exclusion ranges are specified, a single hash is performed, but only over the asset ranges that are not in any of the exclusions.

If a manifest is embedded in the center of a JPEG 1 file in an APP11 segment, then the claim creator may exclude the APP11 segment(s) from the hash calculation.

In order to prevent insertion attacks, it is desirable to have only a single exclusion range when possible. When the size or location (or both) of the manifest in the asset is not known, then the `start` and `length` values in the data hash assertion shall both be zero and the size of the `pad` value should be large enough to accommodate writing in the values during the second pass. At least 16 bytes is recommended. The value of the `pad` key shall consist of all 0x00's.

If padding is employed, it is possible that the pad data could be changed without resulting in a validation failure. Claim generators shall ensure that changes to pad data (or any other excluded asset data) cannot change how the asset is interpreted.

NOTE

In the case of JPEG 1 files, this can be achieved either by eliminating padding or by ensuring that the `JFIF APP11/C2PA` segments cannot be shortened or changed to a different segment type. This is accomplished by including all the C2PA manifest segment headers (APP11) and 2-byte length fields in the data-hash-map for all manifest-containing segments. Doing so ensures that any data changed in the exclusion region will not be misinterpreted by JPEG processors.

11.7.2. Create a temporary Claim and Signature

Add the newly created data hash assertion reference to the claim's assertion list providing a temporary hash value, such as empty spaces.

At this point, the temporary claim is complete and can be added to the C2PA Manifest being created.

Since the claim is only temporary at this time, it is not possible to sign it. To ensure the claim signature box contains a valid CBOR structure, create a temporary `COSE_Sign1_Tagged` structure as described in {8152}, section 4.2. The `COSE_Sign1_Tagged` is a tag byte followed by a `COSE_Sign1` structure, which is a four-element CBOR array. Construct the array as follows:

- The first element is the `protected` header bucket ({8152}, section 3). Create an empty bucket by placing a `bstr` of size 0 in this position.
- The second element is the `unprotected` header bucket, which is a CBOR map. Create a map of 1 pair.

Use the string `pad` as the label, and place a `bstr` of the desired padding size filled with zero bytes (0x00) as the value. A 25 kilobyte size is recommended for the initial size of this padding.

- The third element is the `payload`. Place the value `nil` (CBOR major type 7, value 22) here.
- The fourth element is `signature`. Place a `bstr` of size 0 here.

11.7.3. Complete the C2PA Manifest

At this point all of the boxes that comprise the entire C2PA Manifest for the asset are completed and can be (if not already) constructed into its final form. The asset's C2PA Manifest, along with the manifests of any ingredients, are combined together to form the complete C2PA Manifest Store. The active manifest is required to be the last C2PA Manifest superbox in the C2PA Manifest Store superbox. The C2PA Manifest Store can then be embedded into the asset as discussed in Appendix A, *Embedding manifests*.

11.7.4. Going back and filling in

Now that the C2PA Manifest Store has been embedded into the asset, the starting offset and the length of the active manifest can be updated in its data hash assertion. It is necessary that when doing so, you do not change the size of the assertion's box, only its data. This is done by adjusting the value of the `pad` field to be the necessary length to "fill up" the remaining bytes.

NOTE

Preferred/deterministic CBOR serialization of `pad` uses a variable length integer to specify the length of the encoded binary data. When the length goes from zero to 1 byte, or 1 to 2 bytes (etc.), the length of the resulting pad jumps by two bytes. This means that not all paddings can be expressed using a single padding field. For example, 24-byte and 26-byte pads can be created, but a 25-byte pad cannot. If this situation arises, the desired padding can be split between `pad` and `pad2`. For example, to make a 25-byte pad, a claim generator can encode 19 bytes into `pad` (resulting in an encoded length of 20 bytes), and 4 bytes into `pad2` (resulting in 5 bytes.)

Once the data hash assertion has been updated, it can be hashed and the hash written over the empty spaces that were used previously to hold the location.

The claim is now complete, and it can be hashed and signed as described in Section 10.3.2.4, "Signing a Claim", with the resultant signature filling the pre-allocated space. The `pad` header can then be shrunk as required so that the claim signature box remains the same size; because this header is unprotected, changing it does not invalidate the claim signature.

If the serialized `COSE_Sign1_Tagged` structure exceeds the reserved size of the C2PA Claim Signature box, multiple step processing shall be repeated with a larger padding size chosen in Section 11.7.2, "Create a temporary Claim and Signature". Revocation information retrieved during the previous attempt should be reusable if it is still within its validity interval (RFC 6960, section 4.2.2.1), but a new time stamp will be required on the new claim with the file offsets changed as the result of added padding.

A C2PA Manifest may contain assertions defined outside of this specification, and they could depend on file layout. As such, the claim generator may no longer be able to change the file layout and/or offsets in a data hash assertion. In this case, claim generators should use padding prior to assertion creation to ensure that the file layout need not change once the assertion has been finalized.

11.8. External Manifests

In some cases, it may not be possible (or practical) to embed a C2PA Manifest Store in an asset. In those cases, keeping the C2PA Manifests externally to the asset is an acceptable model for providing provenance to assets. The C2PA Manifest should be stored in a location, referred to as a manifest repository, that is easily locatable by a Manifest Consumer working with the asset, such as by reference or URI. As the C2PA Manifest Store is a JUMBF box, it shall be served with the Media Type for C2PA Manifest Stores, `application/c2pa`.

> NOTE
>
> Previous versions of this specification used the media type `application/x-c2pa-manifest-store` for the C2PA Manifest Store. That media type is deprecated.

Some common reasons to use external manifests are:

- Embedding may not be technically possible, for example because the file format has no defined embedding.
- Embedding may not be practical, such as when the size of the C2PA Manifest Store is larger than the asset's digital content.
- Embedding may not be appropriate, such as when it would modify an asset that cannot be modified (e.g., because the asset has already been shared).

11.8.1. Embedding a Reference to an external Manifest

If the asset has embedded XMP, and the C2PA Manifest Store will be stored externally, it is recommended that the claim generator add a `dcterms:provenance` key to the XMP, the value (a URI reference) being where to locate the C2PA Manifest Store.

> NOTE A previous version of this specification also recommended using this method for references to embedded manifests. Now this mechanism is only for external manifests.

Since fonts do not support XMP, an equivalent method for specifying a URI to a remote C2PA Manifest Store is described in this clause on fonts.

Chapter 12. Cryptography

12.1. Hashing

All cryptographic hashes used in the creation and validation of C2PA Manifests (and their contents) as per the technical requirements of this specification shall be generated using one of the hash algorithms as described in this section. This section defines both:

- A list of hash algorithms that generators should use for generating hashes of new content and that validators shall support for validating hashes of existing content (the allowed list);
- A list of hash algorithms that validators should support for validating hashes of existing content but that generators shall not produce (the deprecated list).

> NOTE
> This section does not govern algorithms used for soft bindings as described in Section 16.3.7, “Soft Binding”.

> NOTE
> This section does not govern algorithms used by custom assertions that are defined outside of this specification.

An algorithm shall appear in no more than one list. An algorithm that is instantiated over multiple output lengths (such as the various lengths of SHA2) will each be considered different algorithms, and each instantiation shall be listed separately. If an algorithm does not appear in either list, it is forbidden and shall not be used or supported. Algorithms can be removed from the lists in order to implement forbidding an algorithm. For this reason, implementations shall not support additional algorithms on an optional basis.

Implementers should consult this section in the current version of the specification when releasing software updates and ensure their supported algorithms conform to it.

These lists establish the allowed algorithms for creating hashes and a string algorithm identifier to be used as the algorithm identifier (usually called `alg`) in the corresponding field of C2PA data structures. The outputs of hash functions shall be stored as their binary values encoded into CBOR as byte strings (major type 2) with a declared length. Wherever a field contains the output of a hash function, an algorithm identifier string field shall be present within the same structure, or within an enclosing structure, or in the `claim-map` or `claim-map-v2` structure to declare which algorithm was used. A hash algorithm identifier field should be present in exactly one of these places, but if more than one is present within the structure and its enclosing structures, the nearest identifier shall be used. Nearest is defined first as an identifier that is a sibling field of the hash

value, and then the immediately enclosing structure, up to the root structure. If no identifier is present in any of these places, then the `alg` field from the `claim-map` or `claim-map-v2` structure shall be used.

The allowed list is:

- SHA2-256 ("sha256");
- SHA2-384 ("sha384");
- SHA2-512 ("sha512").

> NOTE
> The SHA-3 family of hash algorithms are not on the allowed list for consistency with the digital signature algorithm allowed list, because COSE has not yet established digital signature algorithms that use a SHA-3 algorithm as the hash algorithm.

The deprecated list is empty.

12.2. Binding to Content

12.2.1. Overview

A key aspect to the standard C2PA manifest is the presence of one or more data structures, called content bindings, that can uniquely identify portions of the asset. There are two types of bindings that are supported by C2PA - hard bindings and soft bindings. A hard binding (also known as a cryptographic binding) enables the validator to ensure that (a) this manifest belongs with this asset and (b) that the asset has not been modified, by determining values that can match only this asset and no other, not even other assets derived from it or renditions produced from it. A soft binding is computed from the digital content of an asset, rather than its raw bits. A soft binding is useful for identifying derived assets and asset renditions.

A single manifest shall not contain more than one assertion defining a hard binding but may contain zero or more assertions defining soft bindings.

12.2.2. Hard Bindings

Every hard binding assertion is built from the hash algorithms defined in Section 12.1, “Hashing”. The specific hard binding assertion used depends on the format and structure of the asset, as described below.

12.2.2.1. Hashing using byte ranges

The simplest type of hard binding that can be used to detect tampering is a cryptographic hashing algorithm, as described in Section 12.1, “Hashing”, over some or all of the bytes of an asset. This approach can be used on any type of asset, but should only be considered for formats that don’t support one of the forms of box-based hashing.

When using this form of hard binding, a data hash assertion is used to define the range of bytes that are hashed (and those that are not). Because a data hash assertion defines a byte range, it is flexible enough to be usable whether the asset is a single binary or represented in multiple chunks or portions. For live media contributed over Enhanced RTMP (V2), a data hash assertion is used to bind a C2PA Manifest to the stream’s codec configuration, as described in Section 15.5.3.3, “Initialization data binding”.

12.2.2.2. Hashing using a general box hash

When an asset’s format is a non-BMFF-based box format, such as JPEG 1, PNG, GIF or others listed here, then a general box hash assertion should be used. This assertion consists of an array of structures, each one listing one or more boxes (by their name/identifier) and a hash that covers that data of those boxes (and any possible data that may be present in the file between them), along with the algorithm used for hashing.

12.2.2.3. Hashing a BMFF-formatted asset

If the asset is based on ISO/IEC 14496-12, ISO BMFF then a hard binding optimized for the box-based format (called BMFF-based hash assertions) may be used instead.

For a monolithic MP4 file asset where the `mdat` box is validated as a unit, the assertion is validated nearly identically to a data hash assertion. It simply uses a box exclusion list instead of byte ranges to define the range of bytes that are hashed (and those that are not).

For fragmented MP4 (fMP4) files, the assertion itself shall be combined with chunk-specific hashing information which is located as specified in Section A.5, “Embedding manifests into BMFF-based assets”.

The details on how to hash individual segments for live media can be found here.

12.2.2.4. Hashing unstructured text assets

For unstructured text assets that have a C2PA Manifest embedded using Unicode Variation Selectors as described in Section A.8, “Embedding Manifests into Unstructured Text”, a data hash assertions shall be used. This assertion provides a hard binding mechanism specifically designed for text content where the C2PA Manifest is embedded as a sequence of non-rendering Unicode characters, and its hashing follows the text-

specific normalization rules defined in Validating a text data hash.

12.2.2.5. Hashing structured text assets

For structured text assets that have a C2PA Manifest associated using ASCII armour-style delimiters as described in Section A.9, “Embedding Manifests into Structured Text”, a data hash assertion shall be used. The exclusion range covers the manifest block so that the hash is computed over the file content without the manifest reference. Details of the hard binding are defined in Section A.9.5, “Hard Binding”.

12.2.2.6. Hashing append-only text assets

For files that are written only by appending complete lines to the end of the file, such as application logs, event logs and JSON Lines streams, as described in Section A.3.17, “Embedding manifests into append-only text files”, a segment hash assertion shall be used. Each such assertion is a hard binding over one bounded segment of the file rather than over the file as a whole, and the segments are linked into a chain of checkpoints so that the complete history of the file can be verified.

12.2.2.7. Hashing a Collection

In workflows where the C2PA Manifest will refer to a collection of assets, instead of a single asset, the collection data hash assertion shall be used as the method to specify the hard bindings for the assets in the collection.

> NOTE
>
> For example, a collection data hash assertion can be used to describe each folder of a training data set for an AI/ML model.

12.2.2.8. Hashing assets with multiple parts

For file formats that aggregate multiple distinct parts, such as CIPA Multi-Picture Format (MPF) or Android Ultra HDR, where the integrity of each part must be independently verifiable, the multi-asset hash assertion shall be used alongside the hard binding assertion for the file as a whole, recording a hash for each part. A multi-asset hash assertion modifies how a hard binding is validated but is not itself considered a hard binding.

12.2.2.9. Binding Non-C2PA Asset Metadata

One of the capabilities of a hard binding is that it can exclude parts of an asset, for various reasons. However, a Claim Generator should, where possible, include asset metadata (i.e., metadata outside a C2PA Manifest such as EXIF or XMP) in the hard binding, in order to protect its integrity throughout the asset’s lifecycle.

NOTE

Only signed C2PA created_assertions are attributed to the signer. Any asset metadata outside of this specific mechanism, whether included in hard bindings or not, is not attributed to the signer by this specification. For example, the signer is not responsible for the accuracy of a GPS location appearing in EXIF metadata, even if that metadata is covered by the hard binding. However, if the GPS EXIF metadata are serialized in a `c2pa.metadata` assertion that is included in `created_assertions`, that is an explicit signal from the signer that it is asserting the GPS location associated with the asset.

Any asset metadata values that are supported by the common metadata assertion, as described in Appendix C, *Implementation Details for `c2pa.metadata`*, and can be asserted by the signer, should be copied into such an assertion and included in the C2PA Manifest.

12.2.3. Soft Bindings

12.2.3.1. General

Soft bindings are described using soft binding assertions such as a fingerprint computed from the digital content or an invisible watermark embedded within the digital content. These soft bindings enable digital content to be matched even if the underlying bits differ.

NOTE

For example, an asset rendition in a different resolution or encoding format.

Additionally, if a C2PA manifest is removed from an asset, but a copy of that manifest remains in a provenance store elsewhere, the manifest and asset may be matched using available soft bindings.

Because they serve a different purpose, a soft binding shall not be used as a hard binding.

12.2.3.2. Referenced List of Soft Binding Algorithms

Soft bindings are generated by an algorithm named in the `alg' field of the soft binding assertion. The algorithm name should be among those algorithms listed in the soft binding algorithm list as supported by this specification.

12.3. Digital Signatures

All claim signatures shall be generated using one of the digital signature algorithms and key types listed as described in this section. This section defines both:

- A list of digital signature algorithms and key types that generators should use for new claim signatures and that validators shall support for validating existing claim signatures (the allowed list);
- A list of digital signature algorithms and key types that validators should support for validating existing claim signatures but that generators shall not use for creating new claim signatures (the deprecated list).

> NOTE
> This section does not govern digital signatures used by custom assertions that are defined outside of this specification. It also does not govern time stamps.

These lists establish the allowed algorithms and key types by referencing an algorithm identifier from the relevant standards that define algorithms for COSE and their mappings to CBOR identifiers, including but not limited to RFC 9052, RFC 9053, RFC 8230, and RFC 9964. These standards also specify the hash algorithm used in the signature scheme. Nothing in Section 12.1, "Hashing" shall apply to this use of hash algorithms; if a digital signature algorithm is present in the digital signature algorithm and key type below, the use of its specified hash algorithm in the signature scheme shall be allowed and followed.

> NOTE
> Parenthetical notes in the lists below are explainers provided only as an aid to the reader.

12.3.1. Signature Algorithms

The allowed list is:

- ES256 (ECDSA with SHA-256);
- ES384 (ECDSA with SHA-384);
- ES512 (ECDSA with SHA-512);
- ESP256 (ECDSA with SHA-256, fully-specified equivalent of `ES256`);
- ESP384 (ECDSA with SHA-384, fully-specified equivalent of `ES384`);
- ESP512 (ECDSA with SHA-512, fully-specified equivalent of `ES512`);
- PS256 (RSASSA-PSS using SHA-256 and MGF1 with SHA-256);
- PS384 (RSASSA-PSS using SHA-384 and MGF1 with SHA-384);
- PS512 (RSASSA-PSS using SHA-512 and MGF1 with SHA-512);
- EdDSA (Edwards-Curve DSA).
  - Ed25519 instance only. No other EdDSA instances are allowed.
- ML-DSA-44 (Module-Lattice-Based Digital Signature Algorithm, parameter set 44, COSE algorithm

identifier −48);

- ML-DSA-65 (Module-Lattice-Based Digital Signature Algorithm, parameter set 65, COSE algorithm identifier −49);
- ML-DSA-87 (Module-Lattice-Based Digital Signature Algorithm, parameter set 87, COSE algorithm identifier −50).

The deprecated list is empty.

NOTE

ML-DSA is a post-quantum digital signature algorithm standardized in FIPS 204 and registered for use with COSE in RFC 9964. ML-DSA signatures are substantially larger than those produced by the classical algorithms above: approximately 2420 bytes for ML-DSA-44, 3309 bytes for ML-DSA-65, and 4627 bytes for ML-DSA-87. Public keys are likewise larger (1312, 1952, and 2592 bytes respectively). Claim generators selecting an ML-DSA parameter set can weigh the resulting impact on C2PA Manifest size against the desired post-quantum security level.

NOTE

ESP256, ESP384, and ESP512 are "fully-specified" algorithm identifiers registered in RFC 9864. Unlike ES256, ES384, and ES512, which do not by themselves indicate a specific elliptic curve, ESP256, ESP384, and ESP512 each unambiguously bind a hash algorithm to a single elliptic curve, matching the pairings required below: ESP256 to P-256, ESP384 to P-384, and ESP512 to P-521. They are otherwise cryptographically equivalent to ES256, ES384, and ES512 respectively.

NOTE

This section identifies the algorithm used for the C2PA claim signature (the COSE signature). It does not limit or identify the algorithm used to sign X.509 certificates. The algorithm a Certificate Authority uses to sign a certificate (`signatureAlgorithm` in the certificate) is a separate matter governed by Section 13.5.1.1, “General Requirements”. A certificate signed by a CA using RSASSA-PKCS1-v1_5 (`sha256WithRSAEncryption` and similar) is permitted to contain a public key whose corresponding private key signs C2PA claims using RSASSA-PSS, as required by this section.

Implementations shall check that keys provided for signing or verification operations are correct for the chosen algorithm, as required by RFC 9053, section 2.1 for ECDSA, RFC 9053, section 2.2 for EdDSA, RFC 8230, section 2 and section 4 for RSASSA-PSS, and RFC 9964, section 7.3 and section 7.4 for ML-DSA.

These requirements are summarized here for convenience:

- ECDSA requires elliptic curve keys on the P-256, P-384, or P-521 elliptic curves.
  - P-256 keys shall be used with `ES256` or `ESP256`, P-384 keys with `ES384` or `ESP384`, and P-521 keys with `ES512` or `ESP512`. Validators may reject other pairings of these curves with ECDSA algorithms.
- Ed25519 requires elliptic curve keys on the edwards25519 elliptic curve.
- RSASSA-PSS requires RSA keys with a modulus length of at least 2048 bits.
- ML-DSA requires a key whose parameter set matches the chosen algorithm identifier: ML-DSA-44 keys with `-48`, ML-DSA-65 keys with `-49`, and ML-DSA-87 keys with `-50`.

Implementations shall refuse to generate or verify signatures with keys that are not correct for the algorithm choice. Implementations may refuse RSA keys with modulus length greater than 16384 bits.

For live media streaming implementations, which are addressed in Chapter 15, *Live Media*, deterministic signature algorithms should be used so that media segment caching is not negatively impacted. For example, workflows that implement ISO/IEC 23009-9 (REaP) or similar mechanisms would be negatively impacted if non-deterministic signatures were used as duplicate media content would be packaged in unique segments, reducing the effectiveness of segment caching.

12.3.2. Use of COSE

The signature for the CBOR-encoded claim is produced by CBOR Object Signing and Encryption (COSE) as described in RFC 9052, sections 4.2 and 4.4.

> NOTE
>
> Payloads can either be present inside a COSE signature, or transported separately ("detached content" as described in RFC 9052, section 4.1). In "detached content" mode, the signed data is stored externally to the `COSE_Sign1_Tagged` structure, and the `payload` field of the `COSE_Sign1_Tagged` structure is always `nil`.

Regardless of whether the payload will be present in or detached from the `COSE_Sign1_Tagged` signature; the contents of the `payload` field of `Sig_structure` in memory, when constructed to compute or verify a digital signature, shall be populated with that external data as described by the particular use of digital signature in this specification. The `payload` field of `Sig_structure` shall never be `nil`.

When computing or verifying the signature of a standard or update manifest, the `payload` field of the `Sig_structure` will contain the contents of the claim JUMBF box, as described in Section 10.3.2.4, “Signing a Claim” and Section 11.5, “Serialization”.

12.3.3. Computing the Signature

The signature is computed or verified as described in RFC 9052, section 4.4. The following additional requirements apply to the construction of `Sig_structure`:

- The value for the `context` element shall be `Signature1` except where a particular use of digital signatures in this specification specifies using `CounterSignature` instead. `Signature` shall not be used.
- The value for the `payload` element will be specified by each use of digital signatures in this specification.
- The `external_aad` element shall be a `bstr` of length zero. External authenticated data shall not be used.
- The `alg` header specifying the signature algorithm shall be present in the `body_protected` element as defined in RFC 9052, section 3.1.

> NOTE
>
> The `alg` header is a standard COSE header, and therefore is always included in the protected header map with the integer `1` as its label, as established in the IANA COSE Header Parameters Registry. The literal string `alg` is never used as the label. The `sign_protected` element is always omitted when using `COSE_Sign1`.

All digital signatures in C2PA structures shall be a `COSE_Sign1_Tagged` structure as defined in RFC 9052, section 4.2. `COSE_Sign1_Tagged` contains a `COSE_Sign1` structure. The following additional requirements apply to the construction of `COSE_Sign1_Tagged`:

- The same `alg` header in the `Sig_structure` above shall be present in the `protected` header bucket.
- The value for the `payload` field and whether the payload is present in the signature or detached will be specified by each use of digital signatures in this specification. When the `payload` is specified as detached, its value here shall be `nil`. Conversely, when the payload is present in the signature, the binary contents of the payload are stored in this field as a `bstr`.

> NOTE
>
> COSE defines `nil` to be major type 7, value 22 in RFC 9052, section 1.3, and uses this value exclusively for detached content. A byte array (major type 2) of length zero cannot be used to indicate detached content.

12.3.4. Adding a claimed time of signing

A claim generator may also wish to establish a "claimed time of signing" by adding an `iat` protected header,

whose value is a `NumericDate`. If present, it shall represent the time at which the signature was generated.

> NOTE
>
> A `NumericDate` is a CBOR numeric date (as described in RFC 8949, section 3.4.2) but with the leading tag 1 (epoch-based date/time) omitted. It is not used anywhere else in this specification.

> NOTE
>
> This recommendation is based on in-process updates to JAdES for providing a claim generator provided timestamp that is not used for certificate validity checking, but could be used in a user experience. It could be useful in scenarios where the claim generator is not able to access a trusted time source, but still wants to provide a time of signing.

12.3.5. Signature Validation

When producing a signature, if the claim generator can also act as a validator, the claim generator should validate that the signing credential is acceptable according to Chapter 13, *Trust Model* and produce a warning if it is not. The claim generator may still allow signing with that credential if so desired. This may be desirable if it is known that the local claim generator's validator has a different configuration than validators used by the expected audience of the asset.

12.3.6. Cryptographic validation

When verifying a signature, an in-memory `Sig_structure` is generated. Its `body_protected` field is populated with the contents of the `protected` header bucket from the `COSE_Sign1_Tagged` structure (RFC 9052, section 4.4). For the `payload` field, if the payload was specified as present in the signature, it is populated from the `payload` field of the `COSE_Sign1_Tagged` structure. If the payload was specified as detached, the `payload` field of the `COSE_Sign1_Tagged` structure will be `nil`. In this case, the contents of the `payload` field of `Sig_structure` shall be populated from the same external source that was used in the generation of the signature. These are defined in the places where the digital signature is used in this specification.

12.3.7. Inclusion of signer icons

A C2PA Manifest Consumer may wish to display an icon or logo for the signer. To locate such a graphic, it shall look inside the embedded certificate for a logotype as defined in RFC 9399. If no logotype is present, the Manifest Consumer may use icons or logos from other sources in an implementation-dependent manner.

Chapter 13. Trust Model

> NOTE
>
> In this section, "user" refers to human actors that are using C2PA-compliant validators in consumption and authoring scenarios.

13.1. Overview

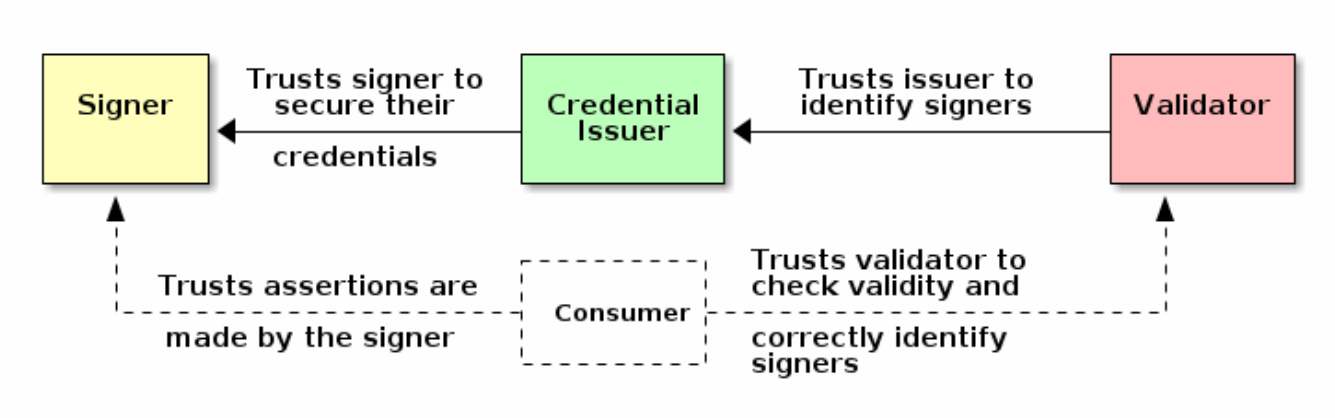


Figure 11. C2PA Trust Model Diagram

Figure 11, “C2PA Trust Model Diagram” shows, in yellow, green and red, the three entities specified in the trust model, which is concerned with trust in a signer’s identity. In dashed lines, below, is the consumer (who is not specified in the trust model), who uses the identity of the signer, along with other trust signals, to decide whether the assertions made about an asset are true.

13.2. Identity of Signers

Identity in the trust model is the means by which a cryptographic signing key (aka credential) is associated with the signer for the basis of making trust decisions based on the claim signature or any structure (including, but not limited to, assertions and claims) signed with that key.

The credential shall be listed in the COSE protected headers of the `COSE_Sign1_Tagged` structure used for digital signatures in all C2PA manifests. Exactly one instance of an identity credential shall appear in the union of the protected and unprotected headers. `COSE_Sign1_Tagged` structures with no credentials, or two or more credentials, shall be rejected. Repeating the same credential more than once, including separately in the protected and unprotected headers, is also an instance of two or more credentials and shall be rejected. An exception is the case where the same X.509 certificate chain is present under both the string label `x5chain` and the integer label `33`, as described in Section 13.5, “X.509 Certificates”; in that case the two

labels shall be treated as a single credential.

> NOTE Older versions of this specification also allowed the credential to appear in the COSE unprotected headers.

How the credential is stored in the header value, how trust chains are constructed are specified, and additional information can be found in Section 13.5, “X.509 Certificates”.

13.3. Validation states

13.3.1. General

A validator is a Manifest Consumer that will make some validation statements about that asset and its associated active manifest. The process for retrieving these statements is described in the validation section. The actor consuming the asset, usually through their user agent and its user interface, then has to interpret those statements to arrive at a set of conclusions of their own about the provenance of the asset they are consuming. These conclusions will be drawn from those statements and the contents of the asset itself.

13.3.2. Manifest States

Based on these statements, a C2PA Manifest may be one of the following:

- Well-Formed
- Valid
- Trusted

> NOTE Any Trusted manifest is also Valid, and any Valid manifest is also Well-Formed.

13.3.3. Asset States

If a validator reports that the portions of the asset that are covered by content bindings have not been modified since the active manifest was produced [Section 14.12, “Validate the Asset’s Content”], and its active manifest is either Valid or Trusted, then the asset itself is a Valid asset.

13.3.4. Well-Formed Manifest

A C2PA Manifest is Well-Formed if validation determines that each of the following is true:

- The manifest's contents abide by the normative requirements of this specification, that are validated via the validation process.
- Only those assertions allowed for the specific type of the manifest are present [Section 14.10.1, "Validate the correct assertions for the type of manifest"].
- The assertions of the manifest meet all the requirements for assertions [Section 14.10.4, "Assertion Validation"].
- Any ingredients present in the manifest meet all the requirements for ingredients [Section 14.11, "Validate the Ingredients"].

13.3.5. Valid Manifest

A C2PA Manifest is Valid if validation determines that each of the following is true:

- The manifest is Well-Formed [Section 13.3.4, "Well-Formed Manifest"].
- The manifest has not been modified since the manifest was signed [Section 12.3.6, "Cryptographic validation"].
- The claim signature receives a success code of `claimSignature.validated` [Section 14.7, "Validate the Signature"].
- Validation of the claim signature validity period receives the success code of `claimSignature.insideValidity` [Section 14.8, "Validate the Time Stamp"].
- The credential of the signer of the C2PA Manifest is not rejected with a failure code of `signingCredential.ocsp.revoked` [Section 14.9, "Validate the Credential Revocation Information"].

If a C2PA Manifest is Valid, then the manifest's claim can be attributed to the claim generator which is identified by the `claim_generator_info` field of the claim [Section 10.2.3, "Claim Generator Info"].

13.3.6. Trusted Manifest

A C2PA Manifest is Trusted if validation determines that each of the following is true:

- The manifest is Valid [Section 13.3.5, "Valid Manifest"].

- The signing credential of the C2PA Manifest receives the success code of `signingCredential.trusted` [Section 14.7, “Validate the Signature”].

13.4. Trust Lists

13.4.1. C2PA Signers

A validator shall maintain the following information for evaluating C2PA signers:

- A list of accepted Extended Key Usage (EKU) values.
- For each accepted EKU value, a list of "trust anchor configurations".

An individual trust anchor configuration shall include the X.509 certificate for that trust anchor. Additionally, the configuration should include a "notBefore" date/time, before which there is no trust for that trust anchor configuration. If the trust for that trust anchor configuration has ended, it should also include a "notAfter" date/time, after which there is no trust for that configuration.

If a "notBefore" date/time is present, the trust anchor configuration shall not be used for claim signatures whose timestamp (or current time, if no timestamp is present) is before that date/time. If a "notAfter" date/time is present, the trust anchor configuration shall not be used for claim signatures whose timestamp (or current time, if no timestamp is present) is after that date/time.

For the `c2pa-kp-claimSigning` (1.3.6.1.4.1.62558.2.1) EKU, the list of trust anchor configurations shall include, but need not be limited to, the signer trust anchor configurations provided by C2PA (i.e., the C2PA Trust List).

NOTE	Some of these lists can be empty.

In addition to the list of trust anchor configurations for the `c2pa-kp-claimSigning` EKU provided in the C2PA Trust List, a validator should allow a user to configure additional trust anchor configurations for that EKU and/or for other EKUs (e.g., `id-kp-emailProtection` (1.3.6.1.5.5.7.3.4) or `id-kp-documentSigning` (1.3.6.1.5.5.7.3.36)). A validator should provide default options or offer lists maintained by external parties that the user may opt into to populate the validator’s trust anchor store for C2PA signers.

NOTE	Previous versions of this specification required the presence of `id-kp-emailProtection` or `id-kp-documentSigning` EKUs, so including at least one of those two EKUs in a signer’s certificate, together with `c2pa-kp-claimSigning`, can

> improve compatibility with older validators.

13.4.2. Time Stamping Authorities

A validator shall maintain a list of trust anchor configurations for Time Stamping Authorities (TSAs), which shall be separate from the lists for C2PA signers.

An individual TSA trust anchor configuration shall include the X.509 certificate for that trust anchor. Additionally, the configuration should include a "notBefore" date/time, before which there is no trust for that trust anchor configuration. If the trust for that trust anchor configuration has ended, it should also include a "notAfter" date/time, after which there is no trust for that configuration.

If a "notBefore" date/time is present, the trust anchor configuration shall not be used for time-stamps asserting times before that date/time. If a "notAfter" date/time is present, the trust anchor configuration shall not be used for time-stamps asserting times after that date/time.

The list shall include, but need not be limited to, the Time Stamping Authority trust anchors provided by the C2PA (i.e., the C2PA TSA Trust List).

In addition to the list of trust anchors provided in the C2PA TSA Trust List, a validator should allow a user to configure additional TSA trust anchor stores, and may provide default options or offer lists maintained by external parties that the user may opt into to populate the validator’s trust anchor store for Time Stamping Authorities.

13.4.3. Private Credential Storage

A validator may also allow the user to create and maintain a private credential store of signing credentials. This store is intended as an "address book" of credentials they have chosen to trust based on an out-of-band relationship. If present, the private credential store shall only apply to validating signed C2PA manifests, and shall not apply to validating time stamps. If present, the private credential store shall only allow trust in signer credentials directly; entries in the private credential store cannot issue credentials and shall not be included as trust anchors during validation.

A validator shall not be pre-configured with any entries in a private credential store.

A validator shall only add entries to a private credential store in response to a user request to trust the credential. Similarly, a validator shall only remove entries from a private credential store in response to a user request to stop trusting the credential.

13.5. X.509 Certificates

X.509 Certificates are stored as defined by RFC 9360 (*CBOR Object Signing and Encryption (COSE): Header Parameters for Carrying and Referencing X.509 Certificates*). For convenience, the definition of `x5chain` is copied below.

IMPORTANT	This specification adds additional requirements beyond those of RFC 9360, which are listed after the quoted text. In particular, this specification requires all intermediate certificate authorities' certificates of the signer's certificate chain to be included in the `x5chain` header, and requires claim generators to always place the `x5chain` header in the protected header bucket.

> x5chain: This header parameter contains an ordered array of X.509 certificates. The certificates are to be ordered starting with the certificate containing the end-entity key followed by the certificate that signed it, and so on. There is no requirement for the entire chain to be present in the element if there is reason to believe that the relying party already has, or can locate, the missing certificates. This means that the relying party is still required to do path building but that a candidate path is proposed in this header parameter.
>
> The trust mechanism MUST process any certificates in this parameter as untrusted input. The presence of a self-signed certificate in the parameter MUST NOT cause the update of the set of trust anchors without some out-of-band confirmation. As the contents of this header parameter are untrusted input, the header parameter can be in either the protected or unprotected header bucket. Sending the header parameter in the unprotected header bucket allows an intermediary to remove or add certificates.
>
> The end-entity certificate MUST be integrity protected by COSE. This can, for example, be done by sending the header parameter in the protected header, sending an 'x5chain' in the unprotected header combined with an 'x5t' in the protected header, or including the end-entity certificate in the external_aad.

> This header parameter allows for a single X.509 certificate or a chain of X.509 certificates to be carried in the message.
>
> - If a single certificate is conveyed, it is placed in a CBOR byte string.
> - If multiple certificates are conveyed, a CBOR array of byte strings is used, with each certificate being in its own byte string.

The validator is only expected to have the certificates for its trust anchors. Therefore, when creating the `x5chain` header as part of signing, the claim generator shall include the signer's certificate and all intermediate certificate authorities in the header's value. The trust anchor's certificate (also called the root certificate) should not be included.

The `subjectPublicKeyInfo` element of the first or only certificate will be the public key used to validate the signature. The `validity` element of the `tbsCertificate` sequence provides the time validity period of the certificate.

A previous version of this specification required claim generators to write the string label `x5chain` only to avoid the unlikely possibility that the integer label `33` would not be standardized.

Integer label `33` has now been standardized, and this specification now adopts it as standard, and deprecates use of the string label. Therefore:

- Claim generators shall use only the integer `33` as the label when inserting this header into a COSE signature. Use of the string label `x5chain` is deprecated (see **Section 18.2, "String Label `x5chain`"**). Claim generators shall place this header only in the protected header bucket of the COSE signature as required above.
- Validators shall accept either the string `x5chain` or the integer `33` as the label for this header. If both labels are present, this shall be treated as a single credential for the purpose of Section 13.2, "Identity of Signers"; validators shall use the header with the integer label `33` and ignore the header with the string `x5chain` as the label. Validators shall accept the header from either the protected or unprotected bucket, to maintain compatibility with previous versions of this specification. In compliance with Section 13.2, "Identity of Signers", if this header appears in both the protected and unprotected buckets with the same label, a validator shall reject the claim signature as malformed due to the presence of multiple credentials.

13.5.1. Certificate Profiles

13.5.1.1. General Requirements

This section defines the requirements to validate that an X.509 certificate is acceptable as a signing credential as described in Section 14.7, “Validate the Signature”.

All C2PA Claim signing certificates shall fulfill the following requirements. The requirements do not apply to TSA, OCSP or CA certificates in certificate chains.

> NOTE
>
> The `signatureAlgorithm` field identifies the algorithm the issuing Certificate Authority (CA) used to sign the certificate itself. This is distinct from the algorithm used for the C2PA claim signature, which is governed by Section 12.3.1, “Signature Algorithms”. RSASSA-PKCS1-v1_5 algorithms (`sha256WithRSAEncryption` and similar) are permitted here as CA certificate signature algorithms but are not permitted for signing C2PA claims.

- The `algorithm` field of the `signatureAlgorithm` field shall be one of the following values:

`ecdsa-with-SHA256`

RFC 5758, section 3.2

`ecdsa-with-SHA384`

RFC 5758, section 3.2

`ecdsa-with-SHA512`

RFC 5758, section 3.2

`sha256WithRSAEncryption`

RFC 8017, appendix A.2.4

`sha384WithRSAEncryption`

RFC 8017, appendix A.2.4

`sha512WithRSAEncryption`

RFC 8017, appendix A.2.4

`id-RSASSA-PSS`

RFC 8017, appendix A.2.3

`id-Ed25519`

RFC 8410, section 3

`id-ml-dsa-44`

RFC 9881, section 3

`id-ml-dsa-65`

RFC 9881, section 3

`id-ml-dsa-87`

RFC 9881, section 3

- If the `algorithm` field of the `signatureAlgorithm` field is `id-RSASSA-PSS`, the `parameters` field is of type `RSASSA-PSS-params`. Its fields shall have the following requirements as defined in RFC 8017, appendix A.2.3:
  - The `hashAlgorithm` field shall be present.
  - The `algorithm` field of the `hashAlgorithm` field shall be one of the following values as defined in RFC 8017, appendix B.1:
    - `id-sha256`.
    - `id-sha384`.
    - `id-sha512`.
  - The `maskGenAlgorithm` field shall be present.
  - The `algorithm` field of the `parameters` field of the `maskGenAlgorithm` field shall be equal to the `algorithm` field of the `hashAlgorithm` field.
- If the `algorithm` field of the `signatureAlgorithm` field is `id-ml-dsa-44`, `id-ml-dsa-65`, or `id-ml-dsa-87`, the `parameters` field shall be absent, as required by RFC 9881, section 3.
- If the `algorithm` field of the `algorithm` field of the certificate's `subjectPublicKeyInfo` is `id-ecPublicKey`, the `parameters` field shall be one of the following named curves from RFC 5480, section 2.1.1.1:
  - `prime256v1`.
  - `secp384r1`.
  - `secp521r1`.
- If the `algorithm` field of the `algorithm` field of the certificate's `subjectPublicKeyInfo` is

`rsaEncryption` or `id-RSASSA-PSS`, the `modulus` field of the `parameters` field shall have a length of at least 2048 bits.

- If the `algorithm` field of the `algorithm` field of the certificate's `subjectPublicKeyInfo` is `id-ml-dsa-44`, `id-ml-dsa-65`, or `id-ml-dsa-87`, the `parameters` field shall be absent, and the `subjectPublicKey` field shall contain an ML-DSA public key encoded as defined in RFC 9881, section 4.

All certificates except those in the private credential store for X.509 certificates shall fulfil the following additional requirements to be acceptable.

- Version shall be `v3` as per RFC 5280, section 4.1.2.1.
- The `issuerUniqueID` and `subjectUniqueID` optional fields of the `TBSCertificate` sequence shall not be present, as per RFC 5280, section 4.1.2.8.
- The Basic Constraints extension shall follow RFC 5280, section 4.2.1.9. In particular, one of the following shall be true:
  - If the certified public key is used to sign certificates, the Basic Constraints extension shall be present with the `cA` boolean asserted.
  - If the certified public key is used to sign C2PA Claims, time stamps, or OCSP responses, the `cA` boolean in the Basic Constraints extension shall not be asserted, and the `keyCertSign` boolean in the key usage extension shall not be asserted. Only end entity certificates shall be used to sign C2PA Claims, time stamps, and OCSP responses.
- The Authority Key Identifier extension shall be present in any certificate that is not self-signed, as per RFC 5280, section 4.2.1.1.
- As prescribed in RFC 5280, section 4.2.1.2, the Subject Key Identifier extension shall be present in any certificate that acts as a CA. It should be present in end entity certificates.
- As prescribed in RFC 5280, section 4.2.1.3, the Key Usage extension shall be present and should be marked as critical. Certificates used to sign C2PA manifests shall assert the `digitalSignature` bit. The `keyCertSign` bit shall only be asserted if the `cA` boolean is asserted in the Basic Constraints extension.
- The Extended Key Usage (EKU) extension shall be present and non-empty in any certificate where the Basic Constraints extension is absent or the `cA` boolean is not asserted, as per RFC 5280, section 4.2.1.12. These are commonly called "end entity" or "leaf" certificates.
  - The `anyExtendedKeyUsage` EKU (2.5.29.37.0) shall not be present.
  - A certificate that signs time stamp countersignatures shall be valid for the `id-kp-timeStamping`

(1.3.6.1.5.5.7.3.8) purpose.

- A certificate that signs OCSP responses for certificates shall be valid for the `id-kp-OCSPSigning` (1.3.6.1.5.5.7.3.9) purpose.
- If a certificate is valid for either `id-kp-timeStamping` or `id-kp-OCSPSigning`, it shall be valid for exactly one of those two purposes, and not valid for any other purpose.
- A certificate should not be valid for any other purposes outside of the purposes listed above, but the presence of any EKUs not mentioned in this profile and not in the list of EKUs in the configuration store shall not cause the certificate to be rejected.

13.5.1.2. Certificate Trust Chain

When validating a certificate as the signing credential, if the certificate is present in the private credential store for X.509 certificates, the certificate is accepted. The private credential store is not consulted when validating time stamps.

If the certificate is not present in the private credential store, or the validator does not implement one, the trust chain shall be built and validated according to the procedure in RFC 5280, section 6 for the particular purpose required (signing, time stamping, or OCSP signing) and for the appropriate trust anchor store for that purpose. Any failure of that validation algorithm shall mean the chain shall be rejected. The private credential store is never included when building certificate chains; certificates in the private credential store cannot act as CAs.

A validator shall ensure a signing certificate is authorized for the purpose for which it is being used, and reject certificates used for an unauthorized purpose. A certificate is authorized for a particular purpose if the purpose's EKU Object Identifier (OID) is present in the Extended Key Usage extension of the certificate (RFC 5280, section 4.2.1.12).

When validating a certificate used to sign a C2PA Claim, the signing certificate shall have at least one of the EKUs for which the validator has an associated list of trust anchors (see Section 13.4.1, "C2PA Signers"), and the validator shall use only the trust anchors it associates with EKUs present in the certificate.

Except for certificates accepted through the private credential store for X.509 certificates, a validator shall verify a certificate's compliance with the Certificate Profile, and reject certificates that do not comply. This includes requiring the presence of the Extended Key Usage extension, as well as a certificate being authorized for no more than one of the three purposes listed in this section: C2PA signing, time stamp signing, or OCSP response signing.

As described in the Certificate Profile, Certification Authority (CA) certificates are not required to have an EKU

Chapter 14. Validation

14.1. Validation Process

14.1.1. Description

Validation of a C2PA Manifest is a multi-step process that involves validating the assertions, claim & associated claim signature contained within it along with (for active manifests, only) validation of any associated hard bindings. This validation process is performed by a validator, which is a hardware or software actor that implements the validation algorithms described in this clause.

14.1.2. Phases of Validation

These phases, which are listed in no particular order, are described in the following clauses:

- Section 14.10, “Validate the Assertions”: Validation of the assertions.
- Section 14.11, “Validate the Ingredients”: Validation of the ingredients, if any.
- Section 14.8, “Validate the Time Stamp”: Validation of the time stamp.
- Section 14.9, “Validate the Credential Revocation Information”: Validation of the credential revocation information.
- Section 14.7, “Validate the Signature”: Validation of the claim signature.
- Section 14.12, “Validate the Asset’s Content”: Validation of the content of the asset.

As described in Section 13.3, “Validation states”, a C2PA Manifest may be considered as Well-Formed, Valid or Trusted based on the results of these steps.

14.1.3. Visual Representation

Figure 12, “Validating a Claim” is a visual representation of the process of validating a C2PA Manifest.

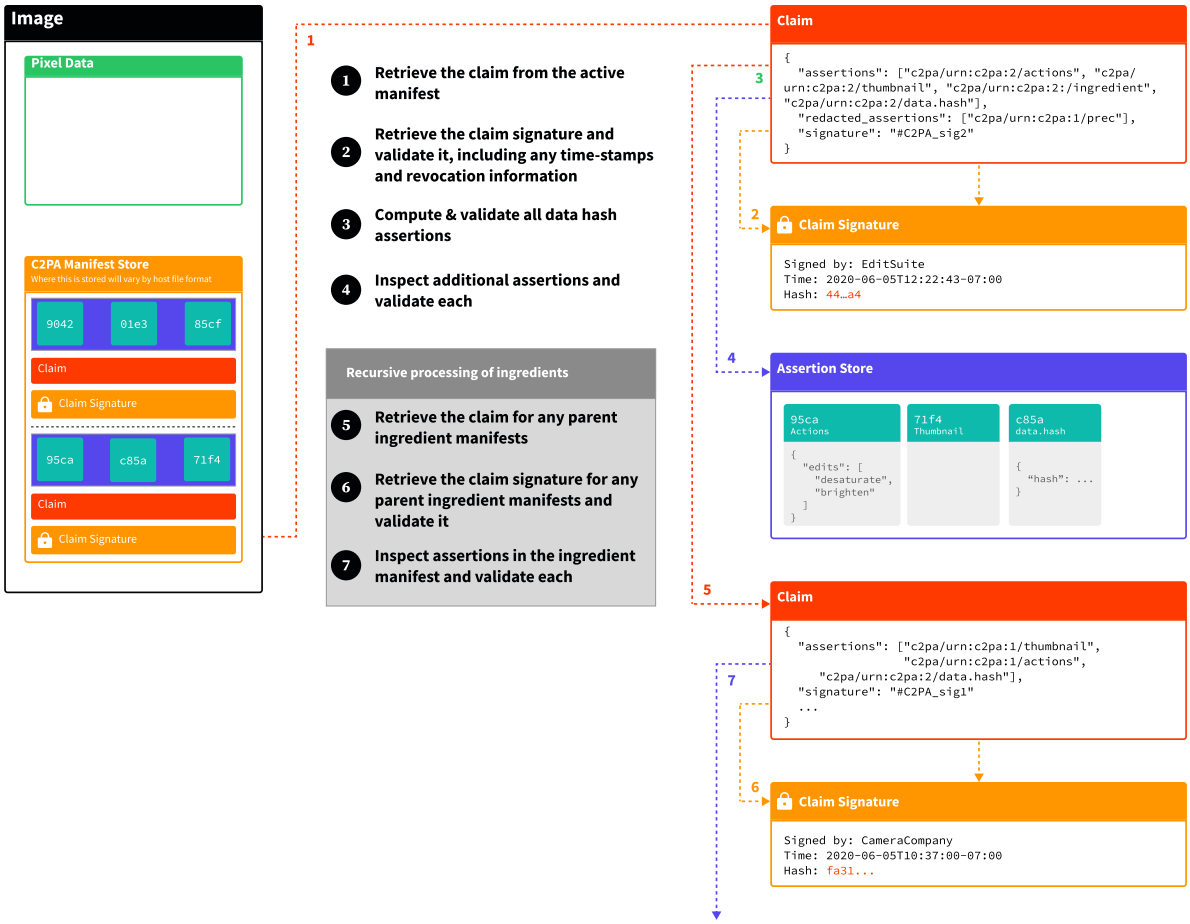


Figure 12. Validating a Claim

> NOTE
>
> If there are any discrepancies between the visual representation and the text, the text is considered authoritative.

14.2. Returning Validation Results

14.2.1. General

The validation algorithm shall return a consolidated set of validation results, per the CDDL Definition schema in Section 14.2.2, “Validation Structures”, for the all manifests in the asset’s C2PA Manifest Store, including the active manifest as well all other manifests in the C2PA Manifest Store that are referenced via ingredient assertions.

Validation results include a standard set of success, informational, and failure codes, as defined in Appendix D, *Standard Validation Status Codes*. Custom status codes are also permitted, when a claim generator has a

```
be empty.
    "failure": [* status-map]           ; an array of validation failure codes. May be
empty.
}

; Objects containing validation results for a manifest and its ingredients

; Definition of a SemVer formatted regex (used in a few places)
semver-string = tstr .regexp "^(0|[1-9]\\d*)\\.(0|[1-9]\\d*)\\.(0|[1-9]\\d*)(?:-((?:0|[1-
9]\\d*|\\d*[a-zA-Z-][0-9a-zA-Z-]*)(?:\\.(?:0|[1-9]\\d*|\\d*[a-zA-Z-][0-9a-zA-Z-
]*))*))?(?:\\+([0-9a-zA-Z-]+(?:\\.[0-9a-zA-Z-]+)*))?$" ; Matches a SemVer version string.

; A trust list used to validate certificates in the trust chain, identified by URI and
type
trust-list-uri-map = {
    "uri": tstr .size (1..max-tstr-length), ; URI identifying the trust list
    "type": tstr .regexp "([\\da-zA-Z_-]+\\.)+[\\da-zA-Z_-]+" ; label identifying the type
of trust list (e.g. a C2PA-defined type such as `c2pa.trustList` or `c2pa.tsaTrustList, or
a custom type using reverse-domain notation)
    ? "identifier": tstr .size (1..max-tstr-length) ; A unique identifier for a specific
instance of the trust list, which may be a version number, a hash, or other unique
identifier
}

validation-results-map = {
    ? "activeManifest": status-codes-map, ; Validation status codes for the ingredient's
active manifest. Present if ingredient is a C2PA asset. Not present if the ingredient is
not a C2PA asset.
    ? "ingredientDeltas": [* ingredient-delta-validation-result-map], ; List of any
changes/deltas between the current and previous validation results for each ingredient's
manifest. Present if ingredient is a C2PA asset.
    ? "specVersion": semver-string, ; The version of the specification against which the
validation was performed (SemVer formatted string)
    ? (
        "trustListUri": tstr .size (1..max-tstr-length) ; DEPRECATED. URI to the trust
list that was used to validate certificates
        //
        "trustListUris": [+ trust-list-uri-map] ; trust list(s) used to validate the
certificates in the trust chain
      )
}

ingredient-delta-validation-result-map = {
    "ingredientAssertionURI": jumbf-uri-type,  ; JUMBF URI reference to the ingredient
assertion
    "validationDeltas": status-codes-map       ; Validation results for the ingredient's
active manifest
}
```

14.3. Displaying Manifest Information

Manifest Consumers should not display data from manifests which are not Valid nor from assets which are not Valid. If the Manifest Consumer chooses to display such data, it shall include as part of the display:

- a warning about the lack of validity,
- a warning that the data shall not be attributed to the manifest's signer, and in the case of an ingredient manifest, additional not to the asset's manifest's signer.

> NOTE
>
> In authoring scenarios, it is desirable to more prominently raise warnings so that a creator can make an informed decision about how to proceed with an asset that is not Valid or that has a flawed provenance history .

14.4. Determining the hashing algorithm

14.4.1. For hard binding assertions

Each hard binding assertion has an `alg` field that specifies the hashing algorithm used to compute the hash of the asset. If the `alg` field is present, it shall be used as the hashing algorithm for the hard binding assertion. If no `alg` field is present in the hard binding assertion, the value of the `alg` field in the Claim shall be used as the hash algorithm. If no `alg` field is present in the Claim, the Claim shall be rejected with a failure code of `algorithm.unsupported`.

14.4.2. For Hashed URIs

Various parts of the C2PA Manifest utilize a `hashed_uri` structure for encapsulating a URI, its hash and (optionally) the algorithm used to compute the hash. If there is an `alg` field in the `hashed_uri` structure, it shall be used as the hashing algorithm. If the `alg` field is not present in the `hashed_uri` structure, the hash algorithm shall be determined by evaluating the nearest enclosing structure that contains an `alg` field. If no `alg` field is found in any of these locations, the value of the `alg` field in the claim shall be used as the hash algorithm. If no `alg` field is present in any of these locations, the claim shall be rejected with a failure code of `algorithm.unsupported`.

14.4.3. For Hashed Ext URIs

Some parts of a C2PA Manifest utilize a `hashed_ext_uri` structure for encapsulating an external URI, its hash and the algorithm used to compute the hash. If there is an `alg` field in the `hashed_ext_uri` structure,

it shall be used as the hashing algorithm. If the `alg` field is not present in the `hashed_ext_uri` structure, the failure code of `algorithm.unsupported` shall be used.

> NOTE
> The `alg` field is mandatory in `hashed_ext_uri`, so no recursive procedure to determine the hash algorithm is necessary.

14.4.4. Algorithm validation

Once the hashing algorithm is determined, it shall be compared to the values in the allowed list or the deprecated list in Section 12.1, "Hashing". If it is not present in either list, the claim shall be rejected with a failure code of `algorithm.unsupported`. If the algorithm is present in the deprecated list, the claim shall be issued an informational code of `algorithm.deprecated`.

14.5. Locating the Active Manifest

14.5.1. General

The last C2PA Manifest superbox in the C2PA Manifest Store superbox shall be considered the active manifest, but locating the C2PA Manifest Store may involve looking in a number of possible locations.

Locating the C2PA Manifest Store is a prerequisite to validation and is not itself associated with a validation status code. If no C2PA Manifest Store can be located by the methods described below, the asset shall be treated as if no manifests were located. Where an embedding method admits only a single C2PA Manifest Store within a given location, more than one in that location shall likewise be treated as if no manifests were located. The per-format considerations that follow apply these rules to specific embedding methods.

> NOTE
> An implementation may provide additional information about why no manifests were located, such as the presence of more than one C2PA Manifest Store. How that information is surfaced is out of scope for this specification.

14.5.2. Embedded

14.5.2.1. General

The C2PA Manifest Store shall be located by the validator embedded inside the asset at the standard locations for embedding manifests. If no embedded C2PA Manifest Store is found, and if an asset was retrieved via an HTTP connection, a validator may look for a `Link` header, as described in the Link Header clause below, to

determine if a C2PA Manifest Store is present.

> NOTE
> Checking the `Link` header, if present, allows a validator to determine if a C2PA Manifest Store is present without having to download the entire asset. This is useful for assets that are large or that are streamed.

If there are multiple C2PA Manifest Stores embedded in an asset, they shall all be considered as invalid and the validation should treat this as if no manifests were located. In the case where this asset is being added as an ingredient, none of these embedded C2PA Manifests shall be included in the ingredient assertion.

If an asset has both an embedded C2PA Manifest Store and a `Link` header indicating the presence of a remote C2PA Manifest Store, the embedded C2PA Manifest Store shall be considered the active manifest and the remote C2PA Manifest Store shall be ignored.

If the embedded C2PA Manifest Store is encrypted, or is otherwise located within an encrypted region of the asset, such that the validator is unable to read its contents, the validator shall reject the manifest with a failure code of `manifest.inaccessible.encrypted`.

> NOTE
> Some file formats provide a mechanism for a C2PA Manifest Store to remain readable even when the rest of the asset is encrypted - for example, the `Identity` crypt filter described in Section A.4, “Embedding manifests into PDFs”. A C2PA Manifest Store embedded using such a mechanism is not considered encrypted for the purposes of this clause.

If the embedded C2PA Manifest Store is readable but its contents cannot be interpreted as a valid C2PA Manifest Store — for example, due to a malformed JUMBF structure, or an unsupported or unrecognized box format — the validator shall reject the manifest with a failure code of `manifest.inaccessible.malformed`.

14.5.2.2. Special Considerations for PDF

PDF files support a technology called "incremental update", where information is appended to the end of the document instead of modifying the original. This requires that PDF files support multiple C2PA Manifest Stores - though there shall only be one per update section.

If there are multiple C2PA Manifest Stores present in a single update section, they shall all be considered as invalid and the validation should treat this as if no manifests were located. However, any C2PA Manifest Stores present in early updates of the PDF or of the original PDF, shall still be considered valid and processed accordingly.

14.5.2.3. Special Considerations for HTML

For HTML-based assets, the C2PA Manifest Store shall be located by following the procedure defined in Section A.7, "Embedding Manifests into HTML". If more than one C2PA manifest element is present in the `head` element — more than one `script` element with `type="application/c2pa"`, both a `script` element and a `link` element referencing a C2PA Manifest Store, or otherwise — the asset shall be treated as if no manifests were located.

14.5.2.4. Special Considerations for Structured Text

For assets consisting of structured text, the C2PA Manifest Store shall be located by following the procedure defined in Section A.9, "Embedding Manifests into Structured Text". The validator shall read the file in binary mode and search for the `-----BEGIN C2PA MANIFEST-----` and `-----END C2PA MANIFEST-----` delimiters. If the delimiters are not found, only one delimiter is found, or more than one manifest block is found, the asset shall be treated as if no manifests were located.

If a manifest block is found, the validator shall extract the manifest reference between the delimiters. The validator shall trim leading and trailing whitespace (U+0020, U+0009, U+000A, U+000D) from the extracted reference before further processing. The trimmed reference shall be resolved as follows:

1. If the manifest reference is a `data:` URI, verify that the media type is `application/c2pa` and the encoding is `base64`. If valid, decode the Base64 data to obtain the C2PA Manifest Store. Otherwise, treat the asset as if no manifests were located.
2. If the manifest reference is a URI, resolve it following the procedure defined in Section 14.5.3, "By Reference or URI". Retrieval of external manifest stores is optional; if the validator does not retrieve the manifest store, the asset shall be treated as if no manifests were located.
3. Otherwise, treat the asset as if no manifests were located.

14.5.2.5. Special Considerations for Append-Only Text Files

For append-only plain-text files and JSONL streams, the C2PA Manifest Store shall be located by following the procedure defined in Section A.3.17, "Embedding manifests into append-only text files". The validator shall:

1. Read the file in binary mode and scan from the end, line by line, for the last complete sentinel line. For plain-text files, this is a line beginning with `-----BEGIN C2PA CHECKPOINT-----` and ending with `-----END C2PA CHECKPOINT-----`; for JSONL files, a line that is a JSON object containing a `c2pa:checkpoint` member.
2. If no such line is found, or if the last candidate line is malformed, the asset shall be treated as if no

manifests were located and the validator shall report `manifest.appendOnlyText.noSentinel`.

3. Extract the manifest reference from the sentinel line, trimming leading and trailing whitespace (U+0020, U+0009, U+000A, U+000D), and resolve it as follows:
    - If the manifest reference is a `data:` URI, verify that the media type is `application/c2pa` and the encoding is `base64`. If valid, decode the Base64 data to obtain the C2PA Manifest Store. Otherwise, treat the asset as if no manifests were located and report `manifest.appendOnlyText.noSentinel`.
    - If the manifest reference is a URI, resolve it following the procedure defined in Section 14.5.3, "By Reference or URI". If the C2PA Manifest Store cannot be retrieved, report `manifest.inaccessible`.

The C2PA Manifest Store located by this procedure is that of the active checkpoint. The remaining checkpoints in the file are located by following the chain, as described in Section 14.12.6, "Validating a segment hash".

14.5.2.6. Special Considerations for Unstructured Text

For assets consisting of unstructured text, the C2PA Manifest Store shall be located by following the procedure defined in Section A.8, "Embedding Manifests into Unstructured Text". If no valid `C2PATextManifestWrapper` is found, or more than one valid `C2PATextManifestWrapper` is found, the asset shall be treated as if no manifests were located.

14.5.2.7. Special Considerations for TIFF

For assets consisting of TIFF based format, the C2PA Manifest Store shall be located in either the first IFD or the last IFD.

14.5.2.8. Special Considerations for WARC

For WARC files, the C2PA Manifest Store shall be located by following the procedure defined in Section A.10, "Embedding manifests into WARC files". The validator shall scan the file for a record whose `WARC-Type` is `c2pa-provenance` and whose `Content-Type` is `application/c2pa`. If no such record is found, the file does not contain a C2PA Manifest Store. If more than one such record is present, the asset shall be treated as if no manifests were located. If exactly one C2PA manifest record is found, the validator shall extract the C2PA Manifest Store from its block.

> NOTE
>
> When a companion CDX or CDXJ index is available, a validator may use it to locate the C2PA manifest record without scanning the entire file.

14.5.3. By Reference or URI

14.5.3.1. By Reference

If there is no embedded C2PA Manifest Store, the following attempts should be made to locate one at a remote location. Once a C2PA Manifest Store is located, the validation process should proceed with that manifest as the active manifest (and therefore not continue looking for additional manifests). The following attempts should be made in order:

- If the asset was retrieved via an HTTP connection, the Link Header clause below describes how to find a manifest via the `Link` header.
- If the asset has any XMP in the standard asset locations (i.e., outside the C2PA Manifest) and that XMP contains a `dcterms:provenance` key, the provided URI should be used to locate the active manifest.
- If the asset is a font with a `C2PA` table and its `activeManifestUriLength` is non-zero, then the indicated URI should be used to locate the active manifest.
- If no C2PA Manifest Store has been located, the validator should look for files at the same path or URI, but with the added filename extension of `.c2pa`. If not found, the validator may also look for a file with the same filename with the extension replaced by `.c2pa`. For example, if the asset is located at `image.jpg`, the validator should look for a C2PA Manifest Store at `image.jpg.c2pa` and then `image.c2pa`.
- If the asset is an entry within an archive or packaging format, such as ZIP or TAR, the validator should look for another entry within that same archive, at the same absolute path as the asset's entry, but with the added filename extension of `.c2pa`. If not found, the validator may also look for a file with the same filename with the extension replaced by `.c2pa`.

If the C2PA Manifest Store has not found by any of the above methods, a validator may look in whatever additional places it deems most appropriate to locate it.

If a manifest was documented to exist in a remote location, but is not present there, or the location is not currently available (such as in an offline scenario), the `manifest.inaccessible` error code shall be used to report the situation.

If a C2PA Manifest Store is retrieved from a remote location but is encrypted such that the validator is unable to read its contents, the validator shall reject the manifest with a failure code of `manifest.inaccessible.encrypted`.

If a C2PA Manifest Store is retrieved from a remote location but its contents cannot be interpreted as a valid C2PA Manifest Store, the validator shall reject the manifest with a failure code of `manifest.inaccessible.malformed`.

Information about the IANA media type for a C2PA Manifest Store can be found in the external manifests section.

14.5.3.2. By `Link` header

If the asset was retrieved via an HTTP connection, the validator should look in the header of the HTTP response for a `Link` header, as defined in RFC 8288, containing a parameter of `rel=c2pa-manifest`. If present, a C2PA Manifest Store can be retrieved from that URI reference. The URI will be a standard `http` or `https` URI, such as https://c2pa.org/image.c2pa.

It is also possible to use the link relation to refer to the C2PA Manifest Store that is embedded inside an asset through the use of a JUMBF URI fragment. The URI would include a JUMBF URI fragment, to the C2PA Manifest Store superbox https://c2pa.org/image.jpg#jumbf=c2pa. References to specific C2PA Manifests within the C2PA Manifest Store are not permitted and the validator shall ignore any `childlabel` portion of the JUMBF URI fragment.

> NOTE
>
> HTTP refers to the *Hypertext Transfer Protocol* defined in RFC 9110, not the specific URL scheme `http://`.

14.5.4. Decompression

As described previously, both standard and update manifests can be compressed. When a compressed manifest is encountered, a validator shall decompress it before proceeding with the standard validation process. If the data contained in the `brob` box of a compressed manifest is not either a standard or update manifest or if the decompression fails, the validator shall reject the manifest with a failure code of `manifest.compressed.invalid`.

14.5.5. Validating a Match

A validator may wish to validate that the located C2PA Manifest Store is indeed the one associated with asset.

If the C2PA Manifest Store was located then the hard binding assertion present in its active manifest shall be used to validate that it is the matching manifest and whether the asset has been modified without manifest updates. If the hard binding does not match, it is unknown if that is because of (a) modification of the asset or (b) the wrong C2PA Manifest Store was located. Accordingly, the validator shall treat this as a non-matching hard binding and reject the manifest with a failure code of `assertion.dataHash.mismatch` if a data hash assertion is used, `assertion.boxesHash.mismatch` if a general box hash assertion is used, `assertion.collectionHash.mismatch` if a collection data hash assertion is used,

`assertion.segmentHash.mismatch` if a segment hash assertion is used, or `assertion.bmffHash.mismatch` if a BMFF hash assertion is used.

14.6. Locating and Validating the Claim

14.6.1. Locating

Once the manifest to be validated has been located (hereafter referred to as the "current manifest"), the claim is found by locating, within the current manifest, the JUMBF Superbox with a label of `c2pa.claim.v2` (or `c2pa.claim` for files with older claim structures) and a JUMBF type UUID of `6332636C-0011-0010-8000-00AA00389B71` (`c2cl`). Note that the JUMBF type UUID is the same for both the new (with `c2pa.claim.v2` label) and old (with `c2pa.claim` label) claim formats. There shall only be one such box in the current manifest. If more than one is located, the C2PA Manifest shall be rejected with a failure code of `claim.multiple`.

14.6.2. Validating

If the content of the claim is not well-formed CBOR, the claim shall be rejected with a failure code of `claim.cbor.invalid`.

> NOTE Well-formed CBOR is defined in RFC 8949, Appendix C.

For a "c2pa.claim.v2", the following fields are expected to be present in the CBOR object. If any are absent, then the claim shall be rejected with a failure code of `claim.malformed`.

- `instanceID`
- `signature`
- `created_assertions`
- `claim_generator_info`

If the `claim_generator_info` field does not contain a `name` field, the claim shall be rejected with a failure code of `claim.malformed`.

If there is an `icon` field in the `generator-info-map` referenced by the `claim_generator_info` field of the `claim-map` or `claim-map-v2`, then its value shall be validated as described in Section 14.10.4.3, “Validation of References”.

14.7. Validate the Signature

The validator shall retrieve the URI reference for the signature from the value of the claim's `signature` field and resolve the URI reference to obtain the COSE signature. If the `signature` field is not present, or the URI cannot be resolved, or the URI does not resolve to a location within the same C2PA Manifest box (as the claim), then the claim shall be rejected with a failure code of `claimSignature.missing`.

For all types of C2PA Manifests, the validation of the credential used in the signature shall be performed according to Chapter 13, *Trust Model*.

If the credential is not acceptable per the requirements of the credential's type, then the claim shall be rejected with a failure code of `signingCredential.invalid`. If the signature algorithm is not on the allowed or deprecated list in Section 12.3, "Digital Signatures", then the claim shall be rejected with a failure code of `algorithm.unsupported`.

Before a chain of trust to a trust anchor is established, the validator shall first determine if a valid time stamp is present. Then, a chain of trust should be verified from the credential to a trust anchor configuration in one of the applicable trust anchor lists. If a "notBefore" date/time is present in a trust anchor configuration, that configuration shall not validate claim signatures whose time stamp (or current time, if no time stamp is present) is before that date/time. If a "notAfter" date/time is present, that configuration shall not validate claim signatures whose time stamp (or current time, if no time stamp is present) is after that date/time.

If a chain of trust cannot be verified, the claim shall be rejected with a failure code of `signingCredential.untrusted`; otherwise, the claim signature shall be assigned a success code of `signingCredential.trusted`.

If the claim has not yet been rejected, validation shall proceed according to the specified procedure in Section 12.3, "Digital Signatures". If validation of the signature fails, then the claim shall be rejected with a failure code of `claimSignature.mismatch`. Otherwise, the claim signature shall be assigned a success code of `claimSignature.validated`.

For the remainder of this chapter, headers refer to the union of the set of protected and unprotected header parameters in the COSE signature. Unless otherwise specified in Section 12.3, "Digital Signatures" or Section 13.5, "X.509 Certificates", a header may appear in either bucket. COSE headers are described in RFC 9052, section 3.

14.8. Validate the Time Stamp

14.8.1. Obtaining the TimeStampToken

14.8.1.1. Embedded in the Claim Signature

If either the `sigTst` or `sigTst2` header is present, then the `tstTokens` array is expected to contain a single `tstToken`. If the header contains more than one `tstToken`, the validator shall issue a `timeStamp.malformed` informational code and ignore the time stamps.

A validator that supports `sigTst` shall perform the following procedures to validate the time stamp response:

- Retrieve the `val` property from the `tstToken`, which shall be an RFC3161-compliant `TimeStampResp` (time stamp response).
- Check the value of the `status` field `PKIStatusInfo`, which is the value of the `status` field of `TimeStampResp`.
    - If it contains any value other than `0` (`granted`) or `1` (`grantedWithMods`), the validator shall issue a `timeStamp.malformed` informational code and ignore that time stamp.
    - If it is either `0` (`granted`) or `1` (`grantedWithMods`), continue with the rest of the time stamp validation process as described below.
- Retrieve the value of the `timeStampToken` field of the `TimeStampResp` for use in the remainder of the validation process.

A validator for `sigTst2` shall retrieve the `val` property from the `tstToken`, which shall be an RFC3161-compliant `timeStampToken` (TimeStampToken, TST).

14.8.1.2. Referenced by a time stamp assertion

If a validator has already located a TimeStampToken in a `sigTst` or `sigTst2` header, that passes validation (as per Section 14.8.2, “Validating the TimeStampToken”), then it shall skip this step. When no such header exists or the TimeStampToken located there did not pass validation, this step shall be followed.

If a validator had previously located any time stamp assertions, which were then maintained in a mapping of C2PA Manifest identifiers to TimeStampTokens, then the validator shall check if the current C2PA Manifest’s identifier is present in the mapping. If it is, the validator shall use the TimeStampToken associated with the identifier in the mapping for the TimeStampToken in the validation process described in Section 14.8.2, “Validating the TimeStampToken”. If more than one TimeStampToken for that identifier is found in the mapping, the validator shall try each one until one successfully passes validation (and then should ignore the others). If the identifier does not appear in the mapping, no error is raised, as it simply means that there is no

TimeStampToken associated with this C2PA Manifest in the current context.

14.8.2. Validating the TimeStampToken

All validators shall continue the process as follows:

- If the signature algorithm in the `timeStampToken` is not on the allowed or deprecated list in Section 12.3, "Digital Signatures", then the validator shall issue a `timeStamp.untrusted` informational code and ignore the time stamp.
- Validate the signature in the `timeStampToken`, as described in RFC 5652, Section 5.6. If the signature is not valid, then the validator shall issue a `timeStamp.mismatch` informational code and ignore the time stamp.
- If the `timeStampToken` does not contain a `messageImprint` field, the validator shall issue a `timeStamp.malformed` informational code and ignore the time stamp.
- If the message imprint hash algorithm is not on the allowed or deprecated list in Section 12.1, "Hashing", then the validator shall issue a `timeStamp.untrusted` informational code and ignore the time stamp.
- Validate that the value of the `messageImprint` field (in the `timeStampToken`), matches either the claim (v1, `sigTst`) or `COSE_Sign1_Tagged` structure's `signature` field (v2, `sigTst2`) of the C2PA Manifest being validated, as described in Section 10.3.2.5.2, "Choosing the Payload". If the values do not match, the validator shall issue a `timeStamp.mismatch` informational code and ignore the time stamp.
- Validate that the `certificates` field of the `timeStampToken` is present, the TSA's certificate can be found in the provided set of certificates in this field, and it is possible to build a trust chain from the TSA's certificate to an entry in C2PA TSA Trust List (or other list of trust anchors present in the validator for this purpose). If the certificate cannot be located or a trust chain cannot be constructed, the validator shall issue a `timeStamp.untrusted` informational code and ignore the time stamp. If the validator determines that a certificate is invalid it may additionally return the informational code `timeStamp.credentialInvalid`.
- Validate that the attested time, as found in the `genTime` field (in the `timeStampToken`), falls within the validity period of the TSA's signing certificate and all CA certificates up to the trust anchor. If it does not, the validator shall issue a `timeStamp.outsideValidity` informational code and ignore the time stamp.
- If the time stamp validation does not stop or fail due to any of the above conditions, then the validator shall issue the success codes of `timeStamp.trusted` and `timeStamp.validated`.
- If the validator issued both `timeStamp.trusted` and `timeStamp.validated` success codes, then

14.8.3. Validating the "claimed time of signing"

A validator may choose to validate the "claimed time of signing" as attested by the value present in the `iat` protected header. If the `iat` header is present, the validator may validate that the attested time falls within the validity period of the signer's certificate and all CA certificates up to the trust anchor, and not later than the time attested by any associated trusted time stamp. If the validator does the validation of this value, and it falls inside the validity period, the validator shall return the `timeOfSigning.insideValidity` informational code, but if it falls outside the validity period, then the validator shall return the `timeOfSigning.outsideValidity` informational code.

14.9. Validate the Credential Revocation Information

Discovery of the revocation status of the signer's certificate and all CA certificates that are part of the trust chain is important to determining whether a given certificate is considered valid and trusted.

For CA certificates, the validator should determine revocation status as indicated in the Authority Information Access (AIA) extension as described in RFC 5280, section 4.2.2.1. The validator should make use of relevant OCSP responses included in the C2PA Manifest if the AIA extension indicates that OCSP is available.

If the validator determines that a CA certificate was revoked at the time indicated in a trusted time stamp, or at the current time if no trusted time stamp is present, then the claim signature shall be rejected with a failure status of `signingCredential.untrusted`.

For the signer's certificate, the validator shall use the following process.

- If a certificate does not support revocation status, or the certificate issuer did not provide a method to query its revocation status, the validator shall treat the credential as not revoked.
- If the claim generator "stapled" OCSP responses in the `rVals` header of the `COSE_Sign1` structure, the validator shall decode and validate the stapled OCSP responses as described in Section 14.9.1, "Determining revocation through OCSP responses in the C2PA Manifest Store".
- If subsequent claim generators added certificate status assertions in other C2PA Manifests in the C2PA Manifest Store, the validator shall use those OCSP response(s) in the validation process described in Section 14.9.1, "Determining revocation through OCSP responses in the C2PA Manifest Store". If more than one OCSP response for the certificate is found, the validator shall try each one until one successfully passes validation (and then should ignore the others).

If no revocation information was found in the C2PA Manifest Store, and the validator is online, then the validator should attempt to determine the revocation status of the certificate by querying the OCSP

responder as described in Section 14.9.2, “Determining revocation from online OCSP response”.

14.9.1. Determining revocation through OCSP responses in the C2PA Manifest Store

A validator shall decode OCSP responses per the requirements of RFC 6960, in particular requirements 1 through 4 of section 3.2. If an OCSP response is accepted, and if all of the following requirements are met, then this establishes that the relevant certificate was not revoked at the time of signing.

- The claim signature has an attested time provided by a valid signed time stamp.
- There exists a `SingleResponse` in the `responses` array of the `tbsResponseData` field of the OCSP response such that all of the following conditions hold:
  - The current time is no earlier than `thisUpdate`.
  - The attested time from the time stamp:
    - is earlier than `thisUpdate`, or
    - falls within the `(thisUpdate,nextUpdate)` interval, if `nextUpdate` is present, or
    - falls within the `(thisUpdate,producedAt + 24 hours)` interval where `producedAt` is the field in the containing `ResponseData`, if `nextUpdate` is not present.
  - The `certStatus` field of the `SingleResponse` is `good`.
- The OCSP signer of the response is an "authorized responder" as defined by RFC 6960, section 4.2.2.2.

If the above conditions are met for any OCSP response in the C2PA Manifest Store, then the validator shall either issue a `signingCredential.ocsp.notRevoked` success code, indicating that the certificate is considered not revoked at the time of signing, or (for additional assurance) perform an online OCSP check as described in Section 14.9.2, “Determining revocation from online OCSP response”.

Otherwise, if an OCSP response in the C2PA Manifest Store meets all of the above conditions except that the `certStatus` field is `revoked`, the certificate shall be considered revoked and the claim shall be rejected with a `signingCredential.ocsp.revoked` failure code.

14.9.2. Determining revocation from online OCSP response

If, for a given certificate, no OCSP response in the C2PA Manifest Store satisfies the conditions in Section 14.9.1, “Determining revocation through OCSP responses in the C2PA Manifest Store”, or if the claim signature does not have a time stamp, the validator may choose to query the OCSP responder, per RFC 6960, with the

responder `accessLocation` found via RFC 6960, section 3.1.

> NOTE
>
> Querying the credential status method can reveal to an observer the identity of the asset being validated, and so this query is optional.

If the validator chooses not to perform an online OCSP check, it shall issue a `signingCredential.ocsp.skipped` informational code.

If the validator attempts to query the OCSP responder but is unable to receive a response, the validator shall issue a `signingCredential.ocsp.inaccessible` informational code.

If a response is received and accepted per the requirements 1 - 4 of RFC 6960, section 3.2, it shall establish the signer's certificate was not revoked at the time of signing if either of the following requirements is fulfilled:

- The claim signature has a valid time stamp, and the attested time falls within the `(thisUpdate,nextUpdate)` interval of the response, or
- The claim signature does not have a valid time stamp but the current real-world time falls within the `(thisUpdate,nextUpdate)` interval of the response,

And both of the following requirements are fulfilled:

- The `certStatus` field of the response is `good`, or `revoked` but with a `revocationReason` of `removeFromCRL`, and
- The OCSP signer of the response is an "authorized responder" as defined by RFC 6960, section 4.2.2.2.

If the `certStatus` field of the response is `revoked` but with a `revocationReason` that is not `removeFromCRL`, it shall establish the signer's certificate was not revoked at the time of signing if both of the following requirements are met:

- The manifest has a valid time stamp, and the attested time falls within the `(thisUpdate,nextUpdate)` interval of the response, and
- The `revocationTime` in the response is after the attested time stamp.

If the above conditions are met, the certificate shall be considered not revoked at the time of signing, and the validator shall issue a `signingCredential.ocsp.notRevoked` success code.

Otherwise:

- If the `certStatus` field of the response is `unknown`, a `signingCredential.ocsp.unknown`

informational code shall be recorded.

- Else, the certificate shall be considered revoked and the claim shall be rejected with a `signingCredential.ocsp.revoked` failure code.

14.10. Validate the Assertions

14.10.1. Validate the correct assertions for the type of manifest

14.10.1.1. General

Depending on the type of manifest, there are assertions that are either required or forbidden. A validator shall check for required and not-permitted assertions.

14.10.1.2. Standard Manifest Assertions

If it is a standard manifest:

1. Validate that there is exactly one hard binding to content assertion - either a `c2pa.hash.data`, a `c2pa.hash.boxes`, a `c2pa.hash.collection.data`, a `c2pa.hash.segment`, `c2pa.hash.bmff.v2` (deprecated), or a `c2pa.hash.bmff.v3`. If no such assertion is present, the manifest shall be rejected with a failure code of `claim.hardBindings.missing`. If there is more than one such assertion, the manifest shall be rejected with a failure code of `assertion.multipleHardBindings`.
2. Validate that there are zero or one `c2pa.ingredient` assertions whose `relationship` is `parentOf`. If there is more than one, the manifest shall be rejected with a failure code of `manifest.multipleParents`.
3. Validate that at least one actions assertion is present in either `created_assertions` or `gathered_assertions`. If there is none, the manifest shall be rejected with a failure code of `claim.actions.missing`.
4. Validate that a `c2pa.created` or `c2pa.opened` action is present in either the first actions assertion in `created_assertions` or the first actions assertion in `gathered_assertions`, and not both. If none is present, or if more than one is present, the manifest shall be rejected with a failure code of `assertion.action.malformed`.

14.10.1.3. Update Manifest Assertions

If it is an update manifest:

1. Validate that exactly one ingredient assertion is present and that its `relationship` is `parentOf`. If there is not (i.e., either it is missing, there are more than one, or the value of `relationship` is not `parentOf`), the manifest shall be rejected with a failure code of `manifest.update.wrongParents`.
2. Validate that there are no `c2pa.hash.data`, `c2pa.hash.boxes`, `c2pa.hash.collection.data`, `c2pa.hash.segment`, `c2pa.hash.bmff.v2` (deprecated), `c2pa.hash.bmff.v3`, or thumbnail assertions. If there are, the manifest shall be rejected with a failure code of `manifest.update.invalid`.
3. Validate that there are no `c2pa.hash.multi-asset` assertions. If there are, the manifest shall be rejected with a failure code of `manifest.update.invalid`.
4. If there are one or more `c2pa.actions` or `c2pa.actions.v2` assertions, validate that the `action` field of each action found in the `actions` array of any such assertion is one of the supported values specified in Update Manifests. If it is not, the manifest shall be rejected with a failure code of `manifest.update.invalid`.

14.10.2. Preparing the list of redacted assertions

A validator, when processing a claim, shall gather the set of redacted assertions for each ingredient's manifest (if present) based on each JUMBF URI listed in its `redacted_assertions` field. A claim's `redacted_assertions` field shall never include a JUMBF URI to any of its own assertions.

> NOTE
>
> Assertions can be redacted from ingredient assets at any point in the final asset's provenance history, and not necessarily by the claim generator that first uses an ingredient asset as an ingredient.

For more details, refer to the Section 14.11.3.2, "Performing explicit validation" section.

14.10.3. Redacted Assertion Representation

A redacted assertion may be represented in the assertion store either by being removed entirely, or by having its labelled assertion container retained with the JUMBF Content boxes within it replaced by a single UUID Content box whose `ID` field has a value of `CAA98EEE-9D4D-F80E-86AD-4DFFCA263973` (called the C2PA Redaction UUID) and whose `DATA` field contains only zeros (binary `0x00` values).

A validator shall accept either representation as a validly redacted assertion, as described in Section 14.10.4, “Assertion Validation”.

14.10.4. Assertion Validation

14.10.4.1. General

Each assertion in the `created_assertions` and `gathered_assertions` fields of the claim (and in the `assertions` field of a v1 claim) is a `hashed_uri` structure. For each assertion, the validator shall first determine if the URI reference in the `url` field is in the list of redacted assertions.

> NOTE
> Even though the assertions listed in the `gathered_assertions` field were not created by the claim generator, they are still part of the Claim and are therefore also validated according to this validation algorithm.

If it is in the list of redacted assertions, then if the assertion’s label is `c2pa.actions` or `c2pa.actions.v2`, the claim shall be rejected with a failure code of `assertion.action.redacted`. Otherwise, the redacted assertion (represented as described in Section 14.10.3, “Redacted Assertion Representation”) is considered valid, and validation continues based on the type of assertion.

For all other assertions (not found in the list of redacted assertions), resolve the URI reference in the `url` field to obtain its data. If the URI does not refer to a location within the same C2PA Manifest (a `self#jumbf` location), the claim shall be rejected with a failure code of `assertion.outsideManifest`. If the URI cannot be resolved and the data retrieved, the claim shall be rejected with a failure code of `assertion.missing`.

Follow the procedure in Section 14.4, “Determining the hashing algorithm” to determine the hash algorithm and any possible failure codes. Compute the hash of the assertion using that algorithm and the procedure described in Section 9.4.2.3, “Hashing JUMBF Boxes”, and compare the computed hash value with the value in the `hash` field. If they do not match, the claim shall be rejected with a failure code of `assertion.hashedURI.mismatch`. Otherwise, a success code of `assertion.hashedURI.match` shall be recorded.

If the content of a standard assertion is not well-formed CBOR or is non-conforming JSON, the claim shall be rejected with a failure code of `assertion.cbor.invalid` or `assertion.json.invalid`.

> NOTE
> Well-formed CBOR is defined in RFC 8949, Appendix C.

> NOTE
> RFC 8259, Clause 2, defines the grammar that JSON data conforms to.

If an assertion that is present in the assertion store is not referenced by an element of either the `created_assertions` or `gathered_assertions` arrays in the claim (or the `assertions` array in the v1 claim), the claim shall be rejected with a failure code of `assertion.undeclared`.

For each URI in the claim's `redacted_assertions` array, if the URI points into the claim's own manifest, the claim shall be rejected with a failure code of `assertion.selfRedacted`. A claim is not permitted to redact its own assertions.

For those assertions that support assertion metadata (via a `metadata` field), if the `metadata` field is present, the validator shall not perform any validation on the contents of that structure, except for the following check: if both `processStart` and `processEnd` are present, validate that the time value represented by `processStart` is less than or equal to the time value represented by `processEnd`. If not, the claim shall be rejected with a failure code of `assertion.metadata.malformed`.

> NOTE
> These fields are checked to be consistent with a similar check performed on `whenStart` and `whenEnd` for action assertions.

14.10.4.2. Specific Assertion Validation

For each assertion, the validator shall check the assertion's label and if it is listed below, the validator shall perform the specific validation steps for that assertion type. If the assertion's label is not listed below, then that type of assertion does not require any additional validation steps beyond those already described.

- `c2pa.cloud-data`, Section 14.10.4.2.1, "`c2pa.cloud-data` validation"
- `c2pa.external-reference`, Section 14.10.4.2.2, "`c2pa.external-reference` validation"
- `c2pa.actions` or `c2pa.actions.v2`, Section 14.10.4.2.4, "`c2pa.actions` validation"
- `c2pa.linked-resource-integrity`, Section 14.10.4.2.3, "`c2pa.linked-resource-integrity` validation"
- `c2pa.metadata`, Section 14.10.4.2.5, "`c2pa.metadata` validation"
- `c2pa.session-keys`, Section 14.10.4.2.6, "`c2pa.session-keys` validation"
- `c2pa.time-stamp`, Section 14.10.4.2.7, "`c2pa.time-stamp` validation"
- `c2pa.alternative-content-representation`, Section 14.10.4.2.8, "`c2pa.alternative-content-representation` validation"

> NOTE
>
> Ingredient assertions (`c2pa.ingredient` or `c2pa.ingredient.v2` or `c2pa.ingredient.v3`) are subject to additional validation at a different point in the validation process (see Section 14.11, "Validate the Ingredients").

If the value of any field of a standard assertion is a `hashed_uri` or `hashed_ext_uri`, the validator shall perform the steps described in Section 14.10.4.3, "Validation of References", except for the `activeManifest` field in `c2pa.ingredient.v3` and the entries of the `externalResources` array in `c2pa.linked-resource-integrity`, for which special validation behavior is specified in Section 14.11.3, "Ingredient Assertion Validation" and Section 14.10.4.2.3, "`c2pa.linked-resource-integrity` validation" respectively.

14.10.4.2.1. **`c2pa.cloud-data`** validation

If the assertion's label is `c2pa.cloud-data`:

1. Check that the assertion contains the following fields: `label`, `size`, and `location`. If any of those fields are missing, the claim shall be rejected with a failure code of `assertion.cloud-data.malformed`.
2. If the `label` field of the external assertion is any of the following values, the claim shall be rejected with a failure code of `assertion.cloud-data.hardBinding`.
    a. `c2pa.hash.bmff.v2` (deprecated)
    b. `c2pa.hash.bmff.v3`
    c. `c2pa.hash.boxes`
    d. `c2pa.hash.collection.data`
    e. `c2pa.hash.data`
    f. `c2pa.hash.multi-asset`
    g. `c2pa.hash.segment`
3. If the manifest is an update manifest and the `label` field of the external assertion is `c2pa.actions` or `c2pa.actions.v2`, the claim shall be rejected with a failure code of `assertion.cloud-data.actions`.
4. If the `label` field of the external assertion is any of the following values, the claim shall be rejected with a failure code of `assertion.cloud-data.malformed`.
    a. `c2pa.actions` (deprecated)

   b. `c2pa.actions.v2`
   c. `c2pa.cloud-data`
   d. `c2pa.ingredient` (deprecated)
   e. `c2pa.ingredient.v2` (deprecated)
   f. `c2pa.ingredient.v3`

5. The `location` field shall be validated according to Section 14.10.5.2, "Validation of External References".
6. If the data for the external assertion is retrieved (from the specified `location`), the label of the JUMBF box retrieved from that location shall match the value of the `label` field in the `c2pa.cloud-data` assertion. If it does not, the claim shall be rejected with a failure code of `assertion.cloud-data.labelMismatch`.

14.10.4.2.2. **`c2pa.external-reference`** validation

If the assertion's label is `c2pa.external-reference`:

1. Check that the assertion contains a `location` field with a `url` field. If it is missing, the claim shall be rejected with a failure code of `assertion.external-reference.malformed`.
2. If the `location` field contains one of `alg` or `hash` but not both, the claim shall be rejected with a failure code of `assertion.external-reference.malformed`.
3. If the `location` field contains both `alg` and `hash` (hashed variant) and the data is retrieved from the URI in the `location` field, the validator shall compute the hash of the retrieved data and compare it against the hash in the `location` field. If they do not match, the validator shall report a failure code of `assertion.external-reference.hashMismatch`.
4. If the `label` field is present and its value is any of the following, the claim shall be rejected with a failure code of `assertion.external-reference.malformed`:
   a. `c2pa.actions` (deprecated)
   b. `c2pa.actions.v2`
   c. `c2pa.cloud-data`
   d. `c2pa.external-reference`
   e. `c2pa.hash.bmff.v2` (deprecated)
   f. `c2pa.hash.bmff.v3`

g. `c2pa.hash.boxes`
h. `c2pa.hash.collection.data`
i. `c2pa.hash.data`
j. `c2pa.hash.multi-asset`
k. `c2pa.hash.segment`
l. `c2pa.ingredient` (deprecated)
m. `c2pa.ingredient.v2` (deprecated)
n. `c2pa.ingredient.v3`

5. If the `label` field is present and the data is retrieved from the URI in the `location` field, the label of the retrieved JUMBF box shall match the value of the `label` field. If it does not, the validator shall report a failure code of `assertion.external-reference.labelMismatch`.

14.10.4.2.3. **`c2pa.linked-resource-integrity`** validation

If the assertion's label is `c2pa.linked-resource-integrity`:

1. Check that the assertion appears in the claim's `created_assertions` array. If it does not, the claim shall be rejected with a failure code of `assertion.linkedResourceIntegrity.notCreated`.
2. Check that the assertion's `resources` field is an object containing either a non-empty `externalResources` array, or a non-empty `relativeResources` array together with an `alg` field. If the `resources` field is missing or matches neither of these shapes, the claim shall be rejected with a failure code of `assertion.linkedResourceIntegrity.malformed`.
3. Every success, informational, or failure code reported under this procedure for a specific entry in the `externalResources` or `relativeResources` array shall include a `resourceUrl` field in its status result, set to the value of that entry's `url` or `uri` field.
4. For each well-formed entry in the `externalResources` or `relativeResources` array, the validator shall report exactly one terminal code for that entry, as determined by the steps below: an informational code of `assertion.linkedResourceIntegrity.skipped` if a validator does not attempt to retrieve its data; otherwise, if a validator attempts to retrieve its data, exactly one of a success code of `assertion.linkedResourceIntegrity.match`, or an informational code of `assertion.linkedResourceIntegrity.inaccessible`, `assertion.linkedResourceIntegrity.sizeMismatch`, `assertion.linkedResourceIntegrity.formatMismatch`, or `assertion.linkedResourceIntegrity.hashMismatch`. None of these codes, when reported

under this procedure, shall cause the claim to be rejected.

5. If `resources` contains an `externalResources` array:

    a. For each entry in the `externalResources` array, check that `url`, `alg`, and `hash` fields are all present. If any are missing, the claim shall be rejected with a failure code of `assertion.linkedResourceIntegrity.malformed`.

    b. For each entry in the `externalResources` array whose data a validator chooses to retrieve, the validator shall perform the resolution, size-comparison, content-type-comparison, algorithm-determination, and hash-comparison steps described in Section 14.10.5.2, "Validation of External References", except that:

        i. If the URI cannot be resolved or the data otherwise cannot be obtained, including when the validator aborts the attempt by applying a countermeasure described in Section 14.10.5.3, "Retrieval Security", the validator shall report an informational code of `assertion.linkedResourceIntegrity.inaccessible` as the terminal code for that entry.

        ii. If the size-comparison step would return a failure code of `assertion.hashedURI.mismatch`, the validator shall instead report an informational code of `assertion.linkedResourceIntegrity.sizeMismatch` for that entry.

        iii. If the content-type-comparison step would return a failure code of `assertion.hashedURI.mismatch`, the validator shall instead report an informational code of `assertion.linkedResourceIntegrity.formatMismatch` for that entry.

        iv. If the hash-comparison step would return a failure code of `assertion.hashedURI.mismatch`, the validator shall instead report an informational code of `assertion.linkedResourceIntegrity.hashMismatch` for that entry.

        v. Otherwise, the validator shall report a success code of `assertion.linkedResourceIntegrity.match` for that entry.

6. If `resources` contains a `relativeResources` array:

    a. For each entry in the `relativeResources` array, check that `uri` and `hash` fields are both present. If either is missing, the claim shall be rejected with a failure code of `assertion.linkedResourceIntegrity.malformed`.

    b. The shared `alg` field shall be validated as described in Section 14.4.4, "Algorithm validation". If it is not present in either the allowed list or the deprecated list, the claim shall be rejected with a failure code of `algorithm.unsupported`. If it is present in the deprecated list, the validator shall report an informational code of `algorithm.deprecated`.

c. Before use, a validator shall percent-decode the value of each entry's `uri` field and validate the decoded URI, ensuring that no path segment of the decoded URI (i.e., a substring delimited by `/`) is exactly `..`. If this is found, the manifest shall be rejected with a failure code of `assertion.linkedResourceIntegrity.invalidURI`. A path segment that is exactly `.` is permitted, as it does not enable traversal outside the base location.

d. For each entry in the `relativeResources` array whose data a validator chooses to retrieve:

   i. If the URI cannot be resolved or the data otherwise cannot be obtained, the validator shall report an informational code of `assertion.linkedResourceIntegrity.inaccessible` for that entry.

   ii. If the `size` field is present and the size of the retrieved data does not equal its value, the validator shall report an informational code of `assertion.linkedResourceIntegrity.sizeMismatch` for that entry.

   iii. If the `dc:format` field is present and the media type of the retrieved data does not match its value, the validator shall report an informational code of `assertion.linkedResourceIntegrity.formatMismatch` for that entry.

   iv. Otherwise, the hash of the retrieved data shall be computed using the determined algorithm over the decoded content of the resource, after any transfer or content encoding has been removed but before any character decoding, parsing, normalization, or other transformation. If the resultant hash does not match the value of the `hash` field, the validator shall report an informational code of `assertion.linkedResourceIntegrity.hashMismatch` for that entry.

   v. Otherwise, the validator shall report a success code of `assertion.linkedResourceIntegrity.match` for that entry.

14.10.4.2.4. **`c2pa.actions`** validation

If the assertion's label is `c2pa.actions` or `c2pa.actions.v2`:

The `when`, `whenStart`, and `whenEnd` fields are claim generator provided timestamps annotate the time within which an action occurred. Validators shall apply the syntax and internal-consistency checks below but shall not compare these fields with an external time source.

1. Ensure that it has an `actions` field. If not, the claim shall be rejected with a failure code of `assertion.action.malformed`.
2. For each action in the `actions` list:

   a. If the assertion's label is `c2pa.actions.v2` and `when` is present together with `whenStart` or

whenEnd, the claim shall be rejected with a failure code of assertion.action.malformed.

b. If the assertion's label is c2pa.actions.v2 and both whenStart and whenEnd are present, validate that the time value represented by whenStart is less than or equal to the time value represented by whenEnd. If not, the claim shall be rejected with a failure code of assertion.action.malformed.

c. If the action field is either c2pa.created or c2pa.opened, then the claim shall be rejected with a failure code of assertion.action.malformed unless all of the following are true:

   i. the assertion is the first actions assertion in the created_assertions or gathered_assertions array (of a v2 claim), or the first actions assertion in the assertions array of a v1 claim, and

   ii. the action is the first element in the actions array in this assertion.

d. If the action field is c2pa.opened, c2pa.placed, or c2pa.removed:

   i. If the action has no parameters field, or that field's value is empty, the claim shall be rejected with a failure code of assertion.action.ingredientMismatch.

   ii. If the action's parameters field contains no ingredients field (or ingredient field for c2pa.actions), the claim shall be rejected with a failure code of assertion.action.ingredientMismatch.

   iii. If the value of the ingredients field is not an array with at least one element, the claim shall be rejected with a failure code of assertion.action.ingredientMismatch.

   iv. Check references to ingredient assertions:

      A. For c2pa.opened: Check that the ingredients field (or ingredient field for c2pa.actions) contains exactly one valid hashed URI, which resolves either to an ingredient assertion in the current manifest whose relationship field is parentOf, or to a redacted ingredient assertion in the current manifest (in the list of redacted assertions). If not, then the claim shall be rejected with a failure code of assertion.action.ingredientMismatch.

      B. For c2pa.placed: Check that the ingredients field (or ingredient field for c2pa.actions) contains one or more valid hashed URIs, each of which resolves either to an ingredient assertion in the current manifest whose relationship field is componentOf, or to a redacted ingredient assertion in the current manifest (in the list of redacted assertions). If not, then the claim shall be rejected with a failure code of assertion.action.ingredientMismatch.

      C. For c2pa.opened or c2pa.placed, in a v2 claim: Check that each resolved (non-

redacted) ingredient assertion is listed in the same field of the claim (`created_assertions` or `gathered_assertions`) as this actions assertion. If not, then an informational code of `assertion.action.ingredientLocationMismatch` shall be returned.

D. For `c2pa.removed`: Check that the `ingredients` field (or `ingredient` field for `c2pa.actions`) contains one or more valid hashed URIs, each of which resolves either to an ingredient assertion in another manifest whose `relationship` field is `componentOf`, or to a redacted ingredient assertion in another manifest (in the list of redacted assertions). If not, then the claim shall be rejected with a failure code of `assertion.action.ingredientMismatch`.

E. For `c2pa.deleted`: If the `ingredients` field is present, check that it contains one or more valid hashed URIs, each of which resolves either to an ingredient assertion in another manifest whose `relationship` field is `componentOf`, or to a redacted ingredient assertion in another manifest (in the list of redacted assertions). If not, then the claim shall be rejected with a failure code of `assertion.action.ingredientMismatch`.

e. If the `action` field is `c2pa.transcoded` or `c2pa.repackaged`:

i. If the `ingredients` field (or `ingredient` field for `c2pa.actions`) is present, check that each element of that field is a valid hashed URI that resolves either to an ingredient assertion in the current manifest whose `relationship` field is `parentOf`, or to a redacted ingredient assertion in the current manifest (in the list of redacted assertions). If not, then the claim shall be rejected with a failure code of `assertion.action.ingredientMismatch`.

f. If the `action` field is `c2pa.redacted`:

i. Check the `redacted` field that is a member of the `parameters` object for the presence of a JUMBF URI. If the JUMBF URI is not present, or cannot be resolved to an assertion, the claim shall be rejected with a failure code of `assertion.action.redactionMismatch`.

g. If there is a `softwareAgent` field in the `action-common-map-v2` or one or more `softwareAgents` listed in the `softwareAgents` field of the `actions-map-v2`:

i. If there is an `icon` field in the `generator-info-map`, then it shall be validated as described in Section 14.10.4.3, “Validation of References”.

h. If the action’s `parameters` field exists and contains a `relatedAssertions` field:

i. If the value of `relatedAssertions` is not an array with at least one element, the claim shall be rejected with a failure code of `assertion.action.malformed`.

ii. For each element of the `relatedAssertions` array:

A. Validate the hashed URI as described in Section 14.10.4.3, “Validation of References”.

B. Resolve the hashed URI to locate the referenced assertion.

C. If the referenced assertion cannot be resolved, or does not resolve to an assertion (which may or may not be redacted) within the current manifest, the claim shall be rejected with a failure code of `assertion.action.malformed`.

D. If the referenced assertion is of type `c2pa.ingredient`, `c2pa.ingredient.v2`, `c2pa.ingredient.v3`, `c2pa.actions`, or `c2pa.actions.v2`, the claim shall be rejected with a failure code of `assertion.action.malformed`.

E. If the referenced assertion has been redacted (it can be found in the list of redacted assertions), the informational code `assertion.redacted` shall be recorded.

i. If the `action` field is either `c2pa.watermarked` [DEPRECATED] or `c2pa.watermarked.bound`:

i. Verify that there is also a `c2pa.soft-binding` assertion present in the manifest. If there is not, the claim shall be rejected with a failure code of `assertion.action.softBindingMissing`.

j. For each template in the `templates` list:

i. If there is an `icon` field in the `action-template-map-v2`, then it shall be validated as described in Section 14.10.4.3, “Validation of References”.

14.10.4.2.5. `c2pa.metadata` validation

No assertion-specific validation is required for `c2pa.metadata`.

Previous versions of this specification required validators to check metadata against the allowed list. While generators are still required to limit provided metadata to this set of fields, validators should not reject assertions containing unlisted fields, to support forward compatibility as the allowed list evolves.

14.10.4.2.6. `c2pa.session-keys` validation

If the assertion’s label is `c2pa.session-keys`, the validator shall ensure that the `signerBinding` signature is correct (which indicates that the session key private key is known to the Signer). This is done by verifying that the `signerBinding` signature successfully validates (using the session key), and that the payload of the signature is the Signer’s end-entity certificate . Session keys are only valid if the `signerBinding` signature is successfully validated.

14.10.4.2.7. `c2pa.time-stamp` validation

If the assertion's label is `c2pa.time-stamp`, the validator shall ensure that the assertion is well-formed CBOR consisting of a single map (Major type 5) with at least one key/value pair. If this is not the case, the claim shall be rejected with a failure code of `assertion.timestamp.malformed`.

Since validation of the time stamp token is performed as described in Section 14.8.2, "Validating the TimeStampToken", the validator needs to store the time stamp token (and its associated C2PA Manifest identifier) for later use.

14.10.4.2.8. `c2pa.alternative-content-representation` validation

If the assertion's label is `c2pa.alternative-content-representation`, a validator shall perform the following steps:

1. If the `type` field is `exif.originalPreservationImage`, the validator shall proceed to the original preservation image representation validation defined in `original-preservation-image-representation validation`.

original-preservation-image-representation validation

1. Check that no more than one `c2pa.alternative-content-representation` assertion where the `type` field is `exif.originalPreservationImage` is present in the active manifest. If more than one is found, the active manifest shall be rejected with a failure code of `assertion.alternativeContentRepresentation.malformed`.
2. The `original-preservation-image-params` contains either the `multiAssetPartIndex` field or the `embeddedOriginalPreservationImage` field. If both fields are present, or neither is present, the assertion shall be rejected with a failure code of `assertion.alternativeContentRepresentation.malformed`.
3. If the `multiAssetPartIndex` field is present and parameters is present in the active manifest:
    a. If the active manifest does not include a `c2pa.hash.multi-asset` assertion, the assertion shall be rejected with a failure code of `assertion.alternativeContentRepresentation.malformed`.
    b. The `multiAssetPartIndex` field shall be a valid zero-based index for an element within the parts array of the located c2pa.hash.multi-asset assertion. If the index is out of bounds (i.e., greater than or equal to the total length of the parts array), the assertion shall be rejected with a failure code of `assertion.alternativeContentRepresentation.malformed`.
4. If the `embeddedOriginalPreservationImage` field is present:

a. Resolve the URI reference in the `embeddedOriginalPreservationImage` field to obtain the data of this embedded data assertion.

b. The `embeddedOriginalPreservationImage` field shall be a hashed-uri-map structure, and its hash field shall be present. If this hash field is missing, the assertion shall be rejected with a failure code of `assertion.alternativeContentRepresentation.malformed`.

c. Compute the hash of the resolved embedded data using the determined algorithm. If the computed hash does not match the hash value provided in the `embeddedOriginalPreservationImage` field, the assertion shall be rejected with a failure code of `assertion.alternativeContentRepresentation.hashMismatch`.

5. If no failure codes are encountered during the validation of the `c2pa.alternative-content-representation` assertion, a success code of `assertion.alternativeContentRepresentation.match` shall be recorded.

14.10.4.3. Validation of References

Some C2PA standard assertions support referencing other boxes in the C2PA Manifest via the use of a `hashed_uri` and `hashed_ext_uri`. For example, there can be various references in actions, ingredient, linked resource integrity, and thumbnail assertions.

For all `hashed_uri` and `hashed_ext_uri` fields in standard assertions, except for the `activeManifest` field in `c2pa.ingredient.v3` (for which special validation behaviour is specified in Section 14.11.3, “Ingredient Assertion Validation”) and the entries of the `externalResources` array in `c2pa.linked-resource-integrity` (for which special validation behaviour is specified in Section 14.10.4.2.3, “`c2pa.linked-resource-integrity` validation”), the validator shall perform the following validation: . For a `hashed_ext_uri` whose resource the validator chooses to retrieve, the validator shall perform the steps described in Section 14.10.5.2, “Validation of External References”. . For a `hashed_uri`, the validator shall perform the steps described below.

The destination of a `hashed_uri` is found in its `url` field. If the field is not present or the destination cannot be located (i.e., that data isn’t present where it is supposed to be) then it shall be treated as a validation failure with code `hashedURI.missing`.

If the destination can be located, then proceed as follows: . Follow the procedure in Section 14.4, “Determining the hashing algorithm” to determine the hash algorithm and any possible failure codes. . Ensure that the `hash` field is present in the `hashed_uri` structure. If it is not, the claim shall be rejected with a failure code of `hashedURI.mismatch`. . Compute the hash of the assertion using the determined hash algorithm and the procedure described in Section 9.4.2.3, “Hashing JUMBF Boxes”. . Compare the computed

hash value with the value in the `hash` field. If they do not match, the claim shall be rejected with a failure code of `hashedURI.mismatch`.

14.10.5. External Data Validation

14.10.5.1. General

A cloud data assertion or a hashed external reference assertion contains URI references to (and hashes of) externally hosted data. Those references are not retrieved and validated as part of standard validation. A validator shall first successfully validate a claim before attempting to retrieve the external data referenced. A validator shall not attempt to retrieve external data from a rejected claim. As the retrieval of external data is optional, the inability to retrieve or validate external data shall not cause a claim to be rejected.

If a validator chooses to retrieve any of the external data in a cloud data or hashed external reference assertion, the validator shall perform the steps described in Section 14.10.5.2, “Validation of External References”, and may apply the countermeasures described in Section 14.10.5.3, “Retrieval Security”. If successful, retrieved externally hosted assertions (identified by a `label` field) shall be validated as if the same assertion were embedded.

14.10.5.2. Validation of External References

The following procedure shall be used to validate the external data referenced in a cloud data or hashed external reference assertion. Network retrieval performed as part of this procedure may apply the countermeasures described in Section 14.10.5.3, “Retrieval Security”.

1. Resolve the URI reference in the `url` field and retrieve its data. If the `url` field is not present, the validator aborts the attempt by applying a countermeasure described in Section 14.10.5.3, “Retrieval Security”, or the URI otherwise cannot be resolved and the data retrieved, the validator shall abort the attempt to retrieve the external data and shall not perform the remaining steps.
2. Determine the retrieved representation of the response as described in Section 14.10.5.3.5, “Retrieved representation”. If the length of the retrieved representation is not equal to the value of the `size` field, the validator shall return a failure code of `assertion.hashedURI.mismatch` to the application and not provide the retrieved data.
3. Validate that the media type of the retrieved representation, as declared in the `Content-Type` header of the HTTP response, is equal to the declared content type. If they do not match, the validator shall return a failure code of `assertion.hashedURI.mismatch` to the application and not provide the retrieved data. The declared content type is determined by:

a. For external data, the content type is determined by the `dc:format` field of the `hashed_ext_uri` or `unhashed_ext_uri` structure. If the `dc:format` field is absent, content type validation is always successful.

b. For a cloud data assertion, if the `dc:format` field is not present in its `location` field, the media type shall default to `application/jumbf`. The `content_type` field in the cloud data assertion, if present, shall be ignored.

4. For a `hashed_ext_uri`, determine the hash algorithm to be used as specified in Section 14.4.3, "For Hashed Ext URIs" or possible failure codes.

   a. Compute the hash of the retrieved representation using the determined hash algorithm and the procedure described in Section 9.4.2.3, "Hashing JUMBF Boxes".

   b. Compare the computed hash value with the value in the `hash` field. If the `hash` field is not present or they do not match, the validator shall return a failure code of `assertion.hashedURI.mismatch` to the application and not provide the retrieved data.

   c. Otherwise, the validator shall record a success code of `assertion.hashedURI.match` and provide the retrieved data to the application.

14.10.5.3. Retrieval Security

A validator performing network retrieval under Section 14.10.5.2, "Validation of External References" (or any other clause that references this section) may apply the countermeasures described in this section before and during each retrieval attempt, regardless of which assertion type initiated the retrieval.

Where network retrieval is performed on the validator's behalf by a platform that the validator does not control, and the validator is therefore unable to apply one or more of the countermeasures described in this section, the validator may perform the retrieval through a component that can apply them, such as a server-side service operated for the validator, or may decline to perform the retrieval.

> NOTE
>
> A validator that runs as a web page within a web browser is an example. Such a validator typically cannot observe the addresses to which a host name resolves, cannot enforce its own redirect limit on cross-origin redirects, and cannot prevent the browser from attaching browser-managed proxy credentials. Because retrieval of external data is optional, not performing the retrieval does not cause a claim to be rejected (see Section 14.10.5, "External Data Validation").

14.10.5.3.1. Permitted schemes and identifiers

A validator may limit automated retrieval performed under this procedure to the `https` URI scheme, and may decline to dereference other schemes (such as `file` or `ftp`) and non-network identifiers.

> NOTE
>
> Unencrypted `http` is discouraged by standard industry practice.

A validator may reject a URI whose authority component contains a `userinfo` subcomponent (i.e. `user:password@host`) before a retrieval attempt is made.

14.10.5.3.2. Credential handling

A validator may omit ambient credentials — such as cookies, `Authorization` headers, proxy credentials, or client certificates — from a retrieval request unless the operator has explicitly configured those credentials for the specific origin (scheme, host, and port) being retrieved. A validator may also withhold credentials attached for one origin from a different origin reached by way of a redirect (see Section 14.10.5.3.4, “Redirect handling”).

14.10.5.3.3. Address validation and connection binding

Before opening a connection, a validator may resolve the target host to one or more candidate IP addresses and check each candidate against an address policy. Under such a policy, unless the operator has explicitly configured the validator to permit a particular address or range, a validator may disallow a candidate address that is a loopback address (IPv4 `127.0.0.0/8` or IPv6 `::1`), a link-local address (IPv4 `169.254.0.0/16` or IPv6 `fe80::/10`), a private-use address (IPv4 `10.0.0.0/8`, `172.16.0.0/12`, or `192.168.0.0/16`), a unique-local address (IPv6 `fc00::/7`), or any other address not routable on the public internet. If every candidate address is disallowed, the validator may abort the retrieval.

> NOTE
>
> Operator configuration allows a validator deployed within an enterprise, where assets and their C2PA Manifests reference resources on a private network, to retrieve those resources, while validators that process content from untrusted sources retain the restriction. The restriction exists because the URI being retrieved is supplied by the C2PA Manifest author rather than by the operator of the validator.

Because checking a resolved address is not sufficient on its own, a validator that applies an address policy may also establish the connection to one of the specific candidate addresses that passed the check, and may verify that the address of the peer the connection was actually made to is the same address that was checked, before sending any request data. Such a validator may treat any address resolved by a separate, later name resolution — whether performed by the validator itself, an underlying network library, or an intermediary such

as a forward or transparent proxy — as a candidate address subject to the same policy, applied to the address actually connected to rather than to a previously checked hostname.

Where the operator has explicitly configured the validator to perform retrieval through a proxy that resolves names on the validator's behalf, the validator may delegate the name resolution, address checking, and connection binding described in this section to that proxy, which can apply an equivalent address policy to the address to which it actually connects. The address of such a proxy, which can be a loopback or private-use address, is not a candidate address and is not subject to the address policy described in this section.

NOTE	Some tunnelling services and local resolvers return synthetic addresses drawn from private-use or other non-publicly-routable ranges in place of the target's actual address. An operator using such a service can permit the relevant range through the operator configuration described above.

NOTE	Retrieving external data can disclose, to network observers and to the operators of the DNS resolvers and servers involved, the hostname being requested and therefore the validator's interest in a particular asset. Using an encrypted DNS transport, or retrieving through an operator-configured proxy that performs name resolution remotely, reduces this disclosure. Because retrieval of external data is optional, a validator can also choose not to retrieve it (see Section 14.10.5, "External Data Validation").

14.10.5.3.4. Redirect handling

A validator may limit the number of HTTP redirects it will follow for a single validation operation (see Section 14.10.5.3.6, "Resource limits"), and may refuse to follow a redirect from an HTTPS URI to an HTTP URI. If the redirect limit is exceeded or a downgrade redirect is encountered, the validator may abort the retrieval.

Before a redirected request is sent, a validator may apply the countermeasures in Section 14.10.5.3.1, "Permitted schemes and identifiers", Section 14.10.5.3.2, "Credential handling", and Section 14.10.5.3.3, "Address validation and connection binding" to the redirect's target URI, exactly as it would to the initial request, rather than sending the redirected request and evaluating them against the response it returns.

14.10.5.3.5. Retrieved representation

For the purposes of the declared-size, media-type, and hash checks in Section 14.10.5.2, "Validation of External References", the retrieved representation of a response consists of the response body bytes after removing any HTTP transfer coding (e.g. chunked transfer encoding) and any `Content-Encoding` (e.g. `gzip`, `br`, `deflate`), and before any character-set decoding, parsing, normalization, or other transformation

of those bytes. Declared size comparison, media type validation, and hash computation shall all be performed against this same representation.

14.10.5.3.6. Resource limits

For the purposes of this section, a *retrieval attempt* is a single request-response exchange, including a request made to a redirect target, and a *validation operation* is the complete process of resolving, retrieving, decoding, and validating the data referenced by one `hashed_ext_uri`, `unhashed_ext_uri`, or cloud data `location`, including all redirects followed for that reference.

A validator may enforce, for each validation operation:

- a maximum number of redirects, as described in Section 14.10.5.3.4, “Redirect handling”;
- a maximum number of encoded bytes read from the network before decoding;
- a maximum number of decoded representation bytes, as described in Section 14.10.5.3.5, “Retrieved representation”, enforced separately from the encoded-byte limit;
- a maximum decompression nesting depth, when the response declares more than one chained `Content-Encoding`; and
- a single deadline for the operation, covering DNS resolution, redirect handling, connection establishment, data transfer, content decoding, and validation of the retrieved data.

Across all validation operations performed while validating a single claim, a validator may also enforce a maximum total number of retrieval attempts and a maximum number of validation operations performed concurrently.

If a limit that the validator enforces is exceeded, the validator may abort the retrieval and discard any partial data.

> NOTE
>
> The `size` field in `c2pa.cloud-data` and `hashed_ext_uri` structures provides an a priori bound on decoded representation bytes that a validator can apply as a read limit before or during retrieval, avoiding the need to buffer the entire response before checking its length.

14.11. Validate the Ingredients

14.11.1. Explanation

A validator shall perform the validation steps for the asset being presented and its active manifest. If any of the steps conclude the active manifest is invalid, that manifest shall be rejected with the indicated failure code.

An asset's active manifest may list one or more ingredients, through the use of ingredient assertions. Some of those ingredients may have their own manifests associated with them, and some of those manifests may themselves have ingredients and ingredient manifests.

14.11.2. Processing Ingredient Manifests

14.11.2.1. Standard Manifests in an ingredient

When processing a standard manifest, a validator shall validate each ingredient (regardless of the value of its `relationship` field), as described below.

14.11.2.2. Update Manifests in an ingredient

For update manifests, the `parentOf` ingredient of the update manifest shall be validated as described below.

14.11.2.3. Time Stamp Manifests in an ingredient

> IMPORTANT
>
> This feature has been deprecated in favour of the time stamp assertion. The information below is retained for historical purposes.

Any time stamp manifests found in an ingredient shall be ignored.

14.11.3. Ingredient Assertion Validation

14.11.3.1. Validation Overview

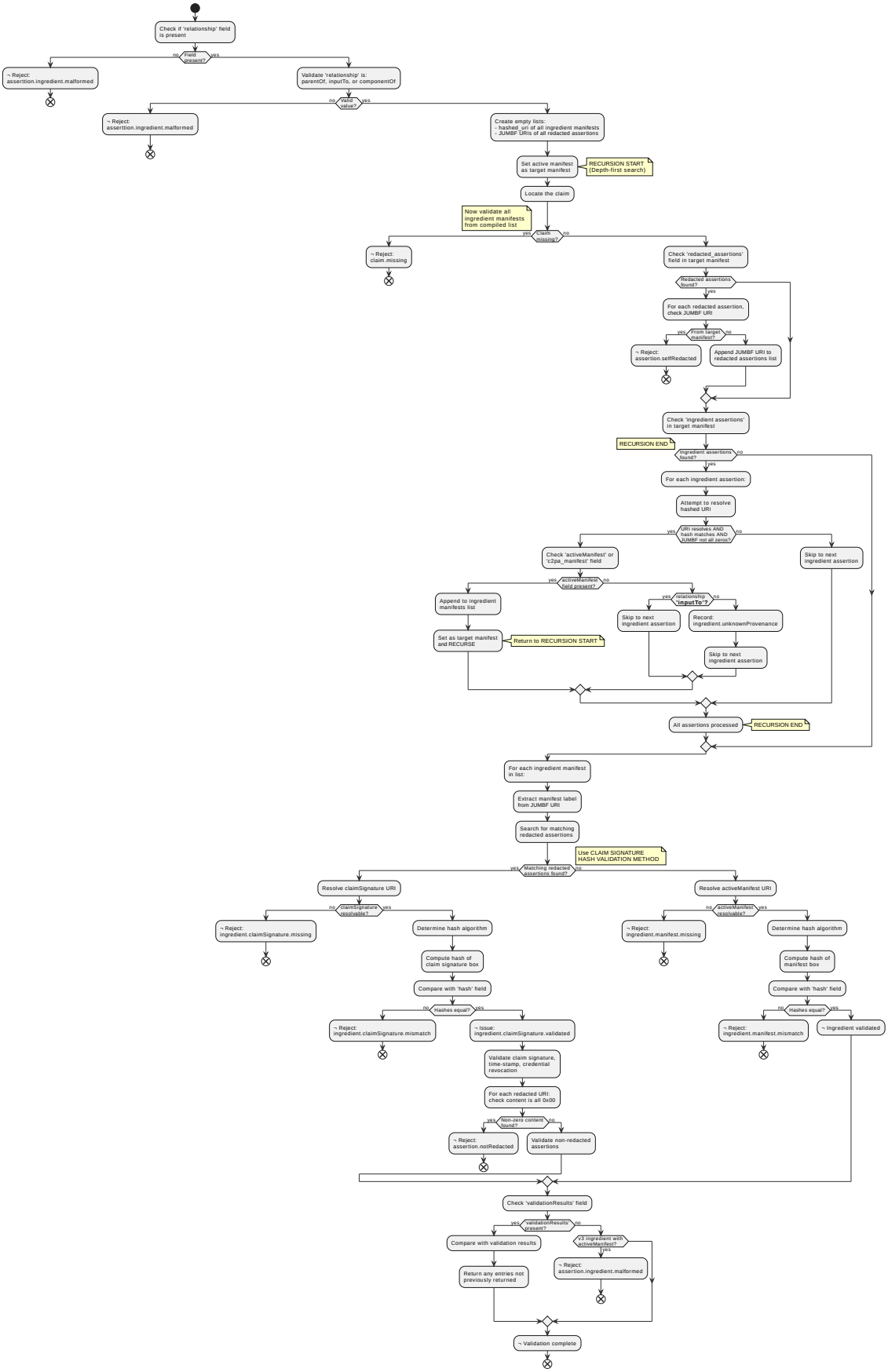


Figure 13. Ingredient Validation

iii. If the ingredient assertion has a `validationResults` field, return each status code within it as part of the validation results.

iv. Skip to the next ingredient assertion until they are all exhausted. At that point, return from the current recursion level.

7. If the claim of the target manifest does not include ingredient assertions, return from the current recursion level.

8. End recursion.

Having compiled a list of all ingredient manifests and a list of all redacted assertions, the validator shall perform the following validation algorithm:

1. For each ingredient manifest in the list of all ingredient manifests:

   a. Extract the manifest label from the ingredient manifest JUMBF URI from each tuple

   b. Search the list of all redacted assertions for assertions with a matching manifest label

   c. If one or more matching redacted assertions are found:

      i. Validate the ingredient using the claim signature hash validation method, described in Section 14.11.3.3.1, “Claim Signature Hash Validation Method”.

   d. If no matching redacted assertions are found:

      i. Validate the ingredient using either the manifest hash validation method, described in Section 14.11.3.3.2, “Manifest Hash Validation Method”, or the claim signature hash validation method, described in Section 14.11.3.3.1, “Claim Signature Hash Validation Method”.

   e. If the ingredient assertion contains a `validationResults` field:

      i. For each entry in the value of the `validationResults` field, if an equivalent entry was not returned as part of the validation process, return it as part of the validation results.

      ii. If there are any entries returned as part of the validation process that are not present in the `validationResults` field, return it as part of the validation results.

   f. If no `validationResults` field is present and the ingredient assertion is a v3 ingredient assertion with the `activeManifest` field present, then return the failure code

> NOTE
>
> Ignoring additional C2PA Manifests supports compatibility with custom assertions and future constructs that may reference C2PA Manifests in ways that the validator does not recognize.

14.11.3.3.1. Claim Signature Hash Validation Method

This method includes a full validation of the ingredient's claim, like that performed for the active manifest, except that content bindings are not evaluated:

1. Resolve the URI reference in the `url` value of the `claimSignature` field to obtain the ingredient's claim signature box. If the URI reference cannot be resolved, or the `claimSignature` field is not present, the ingredient claim is rejected with a failure code of `ingredient.claimSignature.missing`.
2. Determine the hash algorithm identifier (or possible failure code) by following the procedure in Section 14.4, "Determining the hashing algorithm".
3. Compute the hash of the ingredient claim signature box using that algorithm and the procedure described in Section 9.4.2.3, "Hashing JUMBF Boxes".
4. Compare the computed hash with the value in the `hash` field.
    a. If the hashes are not equal or the `hash` field is not present:
        i. Reject the claim with a failure code of `ingredient.claimSignature.mismatch`.
    b. If the hashes are equal, issue a `ingredient.claimSignature.validated`.
        i. Validate the claim signature, time stamp, and credential revocation information as per Section 14.7, "Validate the Signature", Section 14.8, "Validate the Time Stamp", and Section 14.9, "Validate the Credential Revocation Information".
        ii. For each URI in the list of redacted assertions with a matching manifest label, if the referenced assertion is present and any JUMBF Content box or padding box within it contains anything other than zero or more `0x00` bytes (see Section 14.10.3, "Redacted Assertion Representation"), the claim shall be rejected with a failure code of `assertion.notRedacted`.
        iii. Validate each non-redacted assertion per Section 14.10, "Validate the Assertions", except for the hard binding assertions, which cannot be validated for ingredients.

NOTE	The reason for this is that if redactions affect the referenced manifest, it is possible that the hash for this field would not match.

NOTE	When assertions are redacted in an ingredient manifest that is referenced via either of the deprecated ingredient assertions (`c2pa.ingredient` or `c2pa.ingredient.v2`), validation of that assertion will fail (as described in Section 14.11.3, "Ingredient Assertion Validation"), because only `c2pa.ingredient.v3` assertions have a `claimSignature` field and can therefore use the claim signature hash validation method, which is necessary for validating manifests with redacted assertions.

14.11.3.3.2. Manifest Hash Validation Method

An ingredient manifest that has not been changed due to redaction can be validated faster if the current validator trusts the previous claim generator's validation results:

1. Resolve the URI reference in the `url` value of the `activeManifest` field to obtain the ingredient's manifest box. If the `url` field is not present or the URI reference cannot be resolved, the ingredient claim is rejected with a failure code of `ingredient.manifest.missing`.
2. Determine the hash algorithm identifier (or possible failure code) by following the procedure in Section 14.4, "Determining the hashing algorithm".
3. Compute the hash of the ingredient manifest box using that algorithm and the procedure described in Section 9.4.2.3, "Hashing JUMBF Boxes".
4. Compare the computed hash with the value in the `hash` field.
    a. If the hashes are not equal or the `hash` field is not present:
        i. Reject the claim with a failure code of `ingredient.manifest.mismatch`.
    b. If the hashes are equal, the ingredient is fully validated and a `ingredient.manifest.validated` success code is issued.

14.12. Validate the Asset̃s Content

The asset's content shall be validated using the hard binding in the active manifest if the active manifest is a standard manifest. If the active manifest is an update manifest, the hard binding shall be found in the

If a validator encounters exclusion ranges other than for just the C2PA Manifest Store and appropriate padding (e.g., zero'd data), such as for all or part of the asset metadata (as described in Section 12.2.2.9, "Binding Non-C2PA Asset Metadata"), the informational code `assertion.dataHash.additionalExclusionsPresent` shall be issued.

> NOTE
>
> The combination of exclusion ranges and padding values, especially padding needed to support multi-pass processing workflows, may enable an attacker to replace parts of that padding with arbitrary data that could impact the consumption of the asset without invalidating the hash.

The validator shall ignore the presence and contents of `pad` and `pad2` fields.

If no error conditions were encountered, the validator shall add the success code `assertion.dataHash.match` to the list it eventually returns.

If the hash computed over all the asset's data (minus any exclusion ranges) does not match the value of the `hash` field in the `c2pa.hash.data`, then the validator shall look for presence of a multi-asset hash assertion. If one is present, it shall be validated as described in Section 14.12.4, "Validating a multi-asset hash", but if one is not present, the manifest shall be rejected with a failure code of `assertion.dataHash.mismatch`.

14.12.1.2. Hashing of JPEG 1 files

In JPEG 1 files, the file format extras described above would include any `APP11` markers and their respective segment length bytes for `APP11` segments. Because the segment lengths are inside the exclusion range, a validator shall match the total length of the exclusion range with that of the total length of all `APP11` segments representing the C2PA Manifest to ensure that the length was not tampered with.

> NOTE
>
> A JPEG 1 file can contain `APP11` segments for reasons other than C2PA (e.g., JPEG 360 or JPEG Privacy and Security) and those are not included in these calculations.

14.12.1.3. Hashing of Unstructured Text

14.12.1.3.1. Validating a text data hash

For assets consisting of unstructured text with an embedded `C2PATextManifestWrapper`, validators shall perform the following when validating a `c2pa.hash.data` hard binding:

14.12.1.4. Hashing of Structured Text

For structured text assets, the data hash shall be computed over the raw bytes of the file with the manifest block excluded via the exclusion range in the data hash assertion. The validator shall follow the general data hash validation procedure defined in Section 12.2.2, “Hard Bindings”.

NOTE

Structured text hashing operates on the raw bytes of the file without Unicode normalization. Structured text files are byte-stable on disk, unlike clipboard-portable unstructured text. Applying NFC normalization would create false hash mismatches for files containing legitimate NFD content (e.g., filenames or identifiers in YAML or source code). The byte boundaries of the manifest block are determined by the ASCII delimiters, making exclusion range computation unambiguous.

14.12.1.5. Hashing of ONNX files

For ONNX files, the data hash shall be computed over the raw bytes of the serialized `ModelProto` with the Base64-encoded `c2pa:manifest` value bytes excluded via the exclusion range in the data hash assertion. The validator shall follow the general data hash validation procedure defined in Section 12.2.2, “Hard Bindings”.

A validator shall verify that exactly one `StringStringEntryProto` entry with a `key` of `c2pa:manifest` exists in the `ModelProto.metadata_props` list. If more than one such entry is encountered, the asset shall be treated as if no manifests were located.

The validator shall verify that the exclusion range corresponds to the byte offset and length of the Base64-encoded manifest value within the serialized file.

14.12.1.6. Hashing of SafeTensors files

For SafeTensors files, the data hash shall be computed over the raw bytes of the complete file (8-byte header length field + JSON header + tensor data) with the Base64-encoded `c2pa:manifest` value bytes excluded via the exclusion range in the data hash assertion. The validator shall follow the general data hash validation procedure defined in Section 12.2.2, “Hard Bindings”.

A validator shall verify that exactly one `c2pa:manifest` key exists in the `__metadata__` object of the JSON header. If more than one such key is encountered, the asset shall be treated as if no manifests were located.

The validator shall verify that the 8-byte header length field is consistent with the actual byte length of the

JSON header. If the header length field does not match the actual header byte length, the validator shall reject the asset with a failure code of `assertion.dataHash.malformed`.

14.12.2. Validating a BMFF-hash

For any portions of an asset rendered for presentation to a user, including but not limited to audio, video, or text, the corresponding hard binding corresponding to the rendered content shall be validated in accordance with Section 12.2.2, “Hard Bindings”. If the standard hard binding does not validate, and a multi-asset hash assertion is present, it shall be validated as described in Section 14.12.4, “Validating a multi-asset hash”. If at any time content fails to be validated, the validator shall clearly signal to the user that some of the content does not match the claim, and if possible, should indicate what part of the content did not validate. If any content is absent for which content bindings exist, discovery of this absence is also a validation failure. The validator shall continue to report validation has failed, even if later portions of the content validate correctly.

For content that is not wholly available before rendering begins, such as during adaptive bitrate streaming (ABR) and progressive download, absence of not-yet-available portions of content is not considered a validation failure. As the content becomes available, the validator shall validate each portion of the content before it is rendered as previously described. In addition, the validator shall validate that the sequence of said content is the same as when the manifest was produced. Unless the player has explicitly signalled the validator that a discontinuity is expected (e.g., when the consumer performs a manual seek operation via the UI), the validator shall clearly signal to the user that an unexpected discontinuity has occurred whenever the sequence does not match. This includes validating that the `location` values for a given Merkle tree start at zero and increments by one for each following chunk; equivalently, the `location` value always indicates which chunk is being rendered.

For content that is to be validated during playback via progressive download, the leaf nodes of the merkle tree may align to synchronization points of the video track in the `'mdat'` (e.g., the RAP points random access points). When the `'variableBlockSizes'` are setup to achieve such alignment, validation during linear playback or seeking to desired playback time can be both achieved via the same sequence. The desired blocks shall be fetched, validated and the tracks within them selected for rendering.

For content that is intentionally not being rendered as the claim generator originally intended, such as during fast-forward, rewind, or playback at a different speed, the validator may not be able to validate the content. In this case, the validator shall clearly signal to the user that the content cannot be validated during the corresponding operation.

For content with C2PA ContentProvenanceBox with `box_purpose` set to `update` presence, the active manifest is first searched in the C2PA ContentProvenanceBox with `box_purpose` set to `update` then in the C2PA ContentProvenanceBox with `box_purpose` set to `original`. If the active manifest is in the C2PA

14.12.2.1. Non-fragmented asset using Merkle tree

If the `bmff-hash-map.merkle` field is present, the validator shall validate the Merkle tree. If the `merkle-map.fixedBlockSize` and `merkle-map.variableBlockSizes` fields are not present, the whole payload of the `mdat` is treated as a single leaf node for hash calculation. If the `fixedBlockSize` is present and if `variableBlockSizes` is not present, the payload of the `mdat` is divided into fixed-length blocks, and each block is treated as a leaf node. If the final block exceeds the end of the mdat payload, the size of the last block should be set to extend only to the end of the mdat payload. If the `variableBlockSize` is present and if `fixedBlockSizes` is not present, the payload of the `mdat` is divided into sizes defined by the array of `variableBlockSizes`. If the number of elements is not equal to `count` or sum of the values is not equal to size of payload of mdat, then the manifest shall be rejected with a failure code of `assertion.bmffHash.malformed`. If the `merkle-map.fixedBlockSize` and `merkle-map.variableBlockSizes` fields are present, then the manifest shall be rejected with a failure code of `assertion.bmffHash.malformed`.

- If `merkle-map.count` is equal to the number of elements of `merkle-map.hashes`, and if the hash of a leaf node doesn't match the corresponding element of `merkle-map.hashes`, then the manifest shall be rejected with a failure code of `assertion.bmffHash.mismatch`.
- If `merkle-map.count` is larger than the number of elements of `merkle-map.hashes`, and if the auxiliary `uuid` C2PA box doesn't exist as described in Section A.5.3, "Auxiliary `'c2pa'` Boxes for Large and Fragmented Files", then the manifest shall be rejected with a failure code of `assertion.bmffHash.malformed`. If the hash calculated from the auxiliary `uuid` C2PA box and leaf node doesn't match the corresponding element of `merkle-map.hashes`, then the manifest shall be rejected with a failure code of `assertion.bmffHash.mismatch`.
- If `merkle-map.count` is smaller than the number of calculated leaf nodes (either the number of auxiliary uuid C2PA boxes or the number of leaf node `merkle-map.hashes`), then the manifest shall be rejected with a failure code of `assertion.bmffHash.malformed`.

14.12.2.2. Fragmented asset using Merkle tree

If the `bmff-hash-map.merkle` field is present, the validator shall validate the Merkle tree. If the auxiliary `uuid` C2PA box doesn't exist as described in Section A.5.3, "Auxiliary `'c2pa'` Boxes for Large and Fragmented Files", then the manifest shall be rejected with a failure code of `assertion.bmffHash.malformed`. If the hash calculated from the auxiliary `uuid` C2PA box and leaf node doesn't match the corresponding element of `merkle-map.hashes`, then the manifest shall be rejected with a failure code of `assertion.bmffHash.mismatch`

14.12.3. Validating a general box hash

Once a standard manifest (and its bindings) has been located, the list of boxes to be validated shall be extracted from the `boxes` field of the `box-map` structure stored in the `c2pa.hash.boxes` assertion. If no such field is present, then the manifest shall be rejected with a failure code of `assertion.boxesHash.malformed`.

The boxes shall appear in the asset in the same order that they appear in the `boxes` array, including the box containing the C2PA Manifest. If there are any other boxes present in the asset, then the manifest shall be rejected with a failure code of `assertion.boxesHash.unknownBox`. If the boxes appear out of order, then the manifest shall be rejected with a failure code of `assertion.boxesHash.mismatch`.

The hash algorithm for each `box-hash-map` shall be determined by its `alg` field, or if that is not present, by the `alg` field in the containing `box-map`, or if that is not present, by the `alg` field in the Claim. If the hash algorithm is not specified in any of these places, or does not appear in the allowed or deprecated list in Section 12.1, “Hashing”, then the manifest shall be rejected with a failure code of `algorithm.unsupported`.

Refer to `boxIndex` to determine which box the exclusion ranges indicated in the `exclusions` field correspond to. If the box indicated by `boxIndex` does not exist, the manifest shall be rejected with a failure code of `assertion.boxesHash.malformed`. The `start` that indicates the beginning of an exclusion range specifies an offset measured from the start of the box indicated by `boxIndex`. If the ending byte offset of one exclusion range (`start` + `length`) is greater than the starting byte offset of the next exclusion range in the array, or a `start` or `length` value is negative, then the manifest shall be rejected with a failure code of `assertion.boxesHash.malformed`.

Follow the procedure in Section 14.4, “Determining the hashing algorithm” to determine the hash algorithm and any possible failure codes. The hash shall be computed over the bytes of the asset, except for the ranges specified in the `exclusions` field and any boxes where `excluded` field is `true`. If the end of an exclusion range falls beyond the end of the box that contains the exclusion range, then the manifest shall be rejected with a failure code of `assertion.boxesHash.mismatch`.

The validator shall ignore the presence and contents of `pad` and `pad2` fields.

The combination of exclusion ranges and padding values, especially padding needed to support multi-pass processing workflows, may enable an attacker to replace parts of that padding with arbitrary data that could impact the consumption of the asset without invalidating the hash. For this reason a validator shall ensure that the data contained within the exclusion range including a C2PA Manifest Store consists only of the C2PA Manifest Store and appropriate padding (e.g., zero’d data) in clearly marked pad fields or free/skip boxes.

its data. If the resultant hash does not match the value of the `hash` field, then the manifest shall be rejected with a failure code of `assertion.collectionHash.mismatch`. Otherwise, the validator shall add the success code `assertion.collectionHash.match` to the list it eventually returns.

If there are any files listed in the collection data hash assertion that are not found by the validator, then the manifest shall be rejected with a failure code of `assertion.collectionHash.incorrectFileCount`.

14.12.5.2. Extras for ZIP

In a ZIP file with an associated C2PA Manifest, the collection data hash contains the additional `zip_central_directory_hash` field. As described earlier, this field contains a hash of every "central directory header" in the ZIP Central Directory as well as the "end of central directory record" (which is the last part of a ZIP file). The hash algorithm used for this field is the same as the one used for the `hash` field in the `c2pa.hash.collection.data` assertion.

When validating a ZIP file, the validator shall check that the `zip_central_directory_hash` field is present and that the hash of the ZIP Central Directory and "end of central directory record" matches its value. If the hash does not match, then the manifest shall be rejected with a failure code of `assertion.collectionHash.mismatch`.

14.12.5.3. Hashing of WARC files

For WARC files, the collection data hash covers every record other than the C2PA manifest record, and each record's hash is computed over its uncompressed bytes, as described in Section A.10.3, "Hashing".

A WARC file has no central directory that fixes the complete set of records present, so the collection data hash on its own binds only the records it lists. A validator shall reject the manifest with a failure code of `assertion.collectionHash.mismatch` if any two records in the file, other than the C2PA manifest record, share the same `WARC-Record-ID`. A validator shall then verify that the correspondence between the file's remaining records and the entries of the `uris` array, formed by matching each entry's `uri` field against a record's `WARC-Record-ID`, is a bijection: every record corresponds to exactly one entry, and every entry corresponds to exactly one record. A record covered by no entry, an entry covered by no record, or any record or entry covered by more than one counterpart, shall be rejected with a failure code of `assertion.collectionHash.mismatch`. This is the WARC counterpart of the ZIP Central Directory check above: without it, a record appended after signing, or a signed record duplicated under its own identifier, would leave the manifest valid.

Where a record is stored compressed, the validator shall decompress it before computing its hash. To bound

the cost of decompressing untrusted input, the validator shall enforce an implementation-defined maximum decompressed record size, shall reject any record whose decompressed size exceeds that maximum, and shall reject any record whose decompressed block length does not equal its declared `Content-Length`. A record failing either check shall be rejected with a failure code of `assertion.collectionHash.mismatch`.

> NOTE
>
> A record stored with a high compression ratio can expand to a size far larger than its stored form. Bounding the decompressed size, and checking the decompressed block length against the declared `Content-Length`, prevents a maliciously crafted record from forcing unbounded work during validation.

14.12.6. Validating a segment hash

14.12.6.1. General

Validation of a segment hash assertion (`c2pa.hash.segment`) that has been located in a standard manifest, as described in Section 14.5.2.5, “Special Considerations for Append-Only Text Files”, shall be performed over the file read in binary mode.

If the `version` field is absent, or contains a value that the validator does not recognise, then the manifest shall be rejected with a failure code of `assertion.segmentHash.malformed`. If the `hash`, `segmentStart` or `segmentEnd` field is absent, or `segmentEnd` is less than `segmentStart`, then the manifest shall be rejected with a failure code of `assertion.segmentHash.malformed`. If a `predecessorSentinel` field is present but does not contain all of the `start`, `length` and `hash` fields, then the manifest shall be rejected with a failure code of `assertion.segmentHash.malformed`. If the `sentinelLength` field is absent while the sentinel line carrying this manifest references the C2PA Manifest Store by an external URI, or is present while that sentinel line carries the C2PA Manifest Store inline, then the manifest shall be rejected with a failure code of `assertion.segmentHash.malformed`.

Follow the procedure in Section 14.4, “Determining the hashing algorithm” to determine the hash algorithm and any possible failure codes. Once determined, hashing shall be processed as specified in Section 14.4.4, “Algorithm validation”.

The validator shall verify that the byte offset at which the sentinel line carrying this manifest begins is equal to the value of the `segmentEnd` field. If it is not, then the manifest shall be rejected with a failure code of `assertion.segmentHash.mismatch`.

The hash shall be computed over the bytes `[segmentStart, segmentEnd)` of the file. If the resultant hash does not match the value of the `hash` field, then the manifest shall be rejected with a failure code of

Chapter 15. Live Media

15.1. Introduction

The C2PA Live Media specification is designed to support the unique requirements of live media workflows, streaming workflows and dynamic packaging, where content is created and distributed in real-time, or packaged in a just-in-time manner. This specification outlines how to manage assertions related to tracks (video, audio, subtitles, etc) in live sessions, ensuring the integrity and authenticity of the content as it is produced and shared.

NOTE	Live Media workflows can include both live streaming scenarios (e.g., live sports events, news broadcasts) and dynamic packaging scenarios (e.g., on-the-fly packaging of content for adaptive streaming).

NOTE	Live Media isn't constrained to only live video streaming, it can also be used for other types of live media streaming (e.g., live audio streaming), so long as the packaging is BMFF or the content is delivered via Enhanced RTMP (V2).

This version of the specification applies to content packaged using the ISO/IEC 14496-12, ISO BMFF standard, while being agnostic of manifest and delivery protocols. It also applies to ISO/IEC 23000-19 content, which is a constrained subset of ISO BMFF optimized for adaptive streaming. It also applies to content contributed over Enhanced RTMP (V2), a message-based streaming protocol that does not package content using ISO BMFF, via the Section 15.5, "Verifiable Segment Info method".

NOTE	It does not support MPEG-2 — Part 1: Systems (MPEG-2 Transport Streams).

15.2. Compatibility With Previous Versions

Beginning with version 2.5 of the C2PA specification, "live video" was renamed to "live media" to reflect the broader scope of this specification. The term "live video" may still be used in some contexts, but it is now considered a subset of live media.

IMPORTANT	Various identifiers, including CDDL rule and file names, assertion labels, and validation failure codes, that were originally defined with "livevideo" nomenclature have been retained (without renaming) for compatibility.

15.3. Architecture

15.3.1. C2PA Manifest transport

The live media architecture builds upon this specification's support for BMFF-based asset embedding, adding mechanisms to support real-time content validation. The two key parts of that are (1) any hashing of content shall be done using a standard `bmff-hash-map` (where the target of the hash is a segment) or, for content contributed over Enhanced RTMP (V2), as described in Section 15.5.3, "Carriage over Enhanced RTMP (E-RTMP)", and (2) anytime that a C2PA Manifest Store is included in a live media stream, it shall be incorporated into a `uuid` box as described in Section A.5.1, "The `'uuid'` Box for C2PA", or for content contributed over Enhanced RTMP (V2), it shall be delivered in a `c2paManifest` [name, value] pair in a `VideoPacketType.Metadata` or `AudioPacketType.Metadata` message.

For live media or dynamically packaged content, the `uuid` box containing a C2PA Manifest Store may also be present in any segment. This box will be referred to as the "C2PA Manifest Box" in the context of live media.

Dynamically packaged content is prepared "just-in-time" for streaming. That is, the content segments and manifest describing the content, are prepared once a request is received for the content and not before (which would be the case for statically packaged content). In this specification the terms "live media" and "dynamically packaged content" are used interchangeably.

> NOTE
>
> Live streams are, by definition, dynamically packaged but not all dynamically packaged content is live.

15.3.2. Segment-based validation

Live media, or dynamically packaged, streams require the validation of each segment of the stream independently of other segments. As such, the hash of each segment's content needs to be provided as part of that segment. Two different methods for signing that information are described, enabling a given claim generator to choose the most appropriate one for its needs. Either method can be used, and validators are not required to support both.

These methods are:

- "Section 15.4, "Per-segment C2PA Manifest Box method"" which uses a standard C2PA Manifest Box in each segment.
- "Section 15.5, "Verifiable Segment Info method"" which uses a `verifiable-segment-info`, carried in an `emsg` box (for BMFF/CMAF content) or in an E-RTMP `VideoPacketType.Metadata` or

`AudioPacketType.Metadata` message (for content contributed over Enhanced RTMP (V2)), along with session keys.

Other C2PA-defined live media binding schemes (for example, per-segment manifest box-based schemes that derive per-segment validation values by chaining across segments) may also be used, provided that they (a) compute segment hashes using a standard `bmff-hash-map` as defined by this specification and (b) embed or reference a C2PA Manifest Store using a `uuid` box as described in Section A.5.1, "The `'uuid'` Box for C2PA".

15.3.3. C2PA Manifest binding in an initialization segment

For those use cases that require (or desire) the presence of an initialization segment, it shall contain a `uuid` box, whose `bmff-hash-map` includes the `moov` box to establish a cryptographic relationship between the initialization segment and the media segments. For the subset of live media streams that are in a broadcast 'push' streaming scenario (e.g., over-the-air broadcast television without a pull mechanism), the initialization segment may be repeated periodically, and the `uuid` box need not be included in each initialization segment, see here.

> NOTE
>
> The reason for this binding is to ensure that the C2PA Manifest corresponds to the specific media content, which prevents replay attacks and content substitution.

15.4. Per-segment C2PA Manifest Box method

15.4.1. Description

A live media stream may use per-segment C2PA Manifest Boxes to provide per-segment validation. When using this approach, each media segment shall contain a C2PA Manifest Box carried in a `uuid` box as defined in Section A.5.1, "The `'uuid'` Box for C2PA". The active C2PA Manifest for each segment shall contain a BMFF hash assertion (`c2pa.hash.bmff.v3`) with a `hash` field that covers the bytes of that segment and with no `merkle` field, a Section 15.4.2, "Live Media Segment Assertion", and may also contain any other assertions relevant to that segment.

Claim generators may derive additional per-segment validation values (e.g., anchor-point-based chaining values) from segment hashes and embed them in the Manifest.

> NOTE
>
> This method does not require session keys, `verifiable-segment-info-map`, or `emsg` boxes, and can be used independently of the Section 15.5, "Verifiable Segment Info

```
livevideo-segment-map = {
    ; monotonically increasing sequence number for each segment
    "sequenceNumber" => uint,
    "streamId" => tstr, ; A unique "streamId" identifying the live media stream

    ; method used to ensure continuity between segments
    "continuityMethod" => tstr,
    * tstr => any ; additional fields as needed
}


; when continuityMethod == c2pa.manifestId, additional fields include:
livevideo-segment-map-manifestId = {
    ; <<manifest-identifier>> of the previous segment's C2PA Manifest
    "previousManifestId" => tstr,
}
```

An example in CBOR diagnostic notation (RFC 8949, clause 8) is shown below:

```
{
  "sequenceNumber": 175,
  "streamId": "123e4567-e89b-12d3-a456-426614174000",
  "continuityMethod": "c2pa.manifestId",
  "previousManifestId": "urn:c2pa:F9168C5E-CEB2-4FAA-B6BF-329BF39FA1E4"
}
```

15.5. Verifiable Segment Info method

15.5.1. Description

The Verifiable Segment Info Map (`verifiable-segment-info`) is a structure designed to provide concise per-segment information for a live media stream allowing for each segment to be validated independently. It enables the validation of each segment's integrity and authenticity during playback, ensuring that the content has not been tampered with. It is used in workflows which are using Section 15.5.5, "Session Keys" to sign the segment information.

A `verifiable-segment-info` is a `COSE_Sign1_Tagged` structure where the payload field is the CBOR serialized `segment-info-map`. The payload shall be present and shall not be detached. The signature is created by a private session key. The protected header shall carry the identifier of the algorithm used for the signature (`alg` field) and optionally, a timestamp representing "claimed time of signing" for the segment (`iat` field, RFC 8392). The unprotected header shall carry the identifier of the session key (`kid` field).

A `segment-info-map` shall contain the following fields:

`sequenceNumber`

A unique identifier for the segment that is used to ensure the content order. This value shall either match the `mfhd.sequence_number` field defined in ISO/IEC 14496-12, ISO BMFF or shall follow the segment-indexing rules defined in ISO/IEC 23009-9, Section 6.2 (conditions (a) and (f)). When used in conjunction with the per-segment C2PA Manifest Box method, the `sequenceNumber` in the `verifiable-segment-info-map` shall match the `sequenceNumber` value in the Section 15.4.2, "Live Media Segment Assertion".

> NOTE
> These both provide for the sequence number to increase by 1 for each subsequent segment, while not decreasing or repeating.

`manifestId`

A unique identifier for the manifest as described in Section 9.1, "Uniquely Identifying C2PA Manifests and Assets". This is either the identifier of the manifest included in the current segment (via `uuid` box), or the identifier of the manifest referenced by `manifestUrl`, or the identifier of the manifest included in a previous segment (including an init segment) that is still the active manifest.

`bmffHash`

A C2PA `bmff-hash-map` that shall contain the hash of the segment (as described in Section 16.3.2.2, "Hash Computation"), excluding all the C2PA `emsg` boxes (i.e., all `emsg` boxes with the `scheme_id_uri` described above). The `merkle` field shall be absent.

In order to exclude the C2PA `emsg` boxes, `bmffHash` shall at least contain one `exclusion-map` with:

- `xpath` set to `"/emsg"`.
- `data` set with `offset` field equal to `0` and the `value` field set to `urn:c2pa:verifiable-segment-info`.

A `segment-info-map` may also contain the following optional field:

`manifestUri` (optional)

A `hashed-ext-uri` that contains a URL from which a C2PA Manifest can be retrieved.

15.5.2. Use in an `emsg` box

When using the `verifiable-segment-info` in a live media workflow, a `verifiable-segment-info` shall be present in the `emsg` box of each segment of a live media stream to verify the integrity of the content

contributed over Enhanced RTMP (V2), `rtmp-verifiable-segment-info` is carried in an E-RTMP `VideoPacketType.Metadata` message (for the video track) or `AudioPacketType.Metadata` message (for an audio track) instead of an `emsg` box, and its payload is an `rtmp-segment-info-map` instead of a `segment-info-map`. Its `sequenceNumber` and `manifestId` fields have the same meaning as the corresponding fields described above in Section 15.5, "Verifiable Segment Info method"; `segmentHash` replaces `bmffHash` and, instead of a `bmff-hash-map`, contains an `rtmp-hash-map` as described below.

> NOTE
>
> `AudioPacketType.Metadata` is not yet defined by Enhanced RTMP (V2). Its use here assumes that it has been added to the E-RTMP specification.

15.5.3.1. Logical segment

Content contributed over Enhanced RTMP (V2) may include multiple tracks of the same media type (for example, multiple audio tracks), identified by `trackId` when Enhanced RTMP (V2) multitrack support is used. For the purposes of this clause, a track is identified by its media type together with its `trackId`; audio and video `trackId` values are independent of each other, and an audio or video message that does not use Enhanced RTMP (V2) multitrack belongs to track 0 of its media type. A segment is scoped to a single track of the RTMP message stream (as defined by Adobe's Real Time Messaging Protocol) on which its `rtmp-verifiable-segment-info` is carried, identified by the `mediaType` and `trackId` fields of the `rtmp-segment-info-map`; an absent `trackId` is equivalent to 0. The `Metadata` message carrying a `rtmp-verifiable-segment-info` shall belong to the track it identifies.

A video segment shall begin at the `VideoTagHeader` of the first video frame following the previous video segment of that track (or, for a track's first segment, at the start of that track) and shall consist of one or more complete video frames, followed by one or more E-RTMP `VideoPacketType.Metadata` messages, each carrying a `rtmp-verifiable-segment-info` whose payload is an `rtmp-segment-info-map` describing those frames.

An audio segment shall begin at the `AudioTagHeader` of the first audio frame following the previous audio segment of that track (or, for a track's first segment, at the start of that track) and shall consist of one or more complete audio frames, followed by one or more E-RTMP `AudioPacketType.Metadata` messages, each carrying a `rtmp-verifiable-segment-info` whose payload is an `rtmp-segment-info-map` describing those frames.

A publisher shall send each `VideoPacketType.Metadata` message on the same chunk stream (that is, with the same Chunk Stream ID, as defined by Adobe's Real Time Messaging Protocol) as the video messages carrying the coded frames of the track it describes. Likewise, a publisher shall send each

`AudioPacketType.Metadata` message on the same RTMP chunk stream as the audio messages carrying the coded frames of the track it describes. The RTMP message timestamp of a `Metadata` message carrying a `rtmp-verifiable-segment-info` shall be equal to that of the last frame of the segment it closes.

> NOTE
>
> Because RTMP messages sent on different chunk streams can be interleaved, carrying a track's `Metadata` messages on the same chunk stream as that track's coded frames ensures that a receiver observes them in the order in which they were sent, and therefore that the frames closed by each `c2paSegmentInfo` are unambiguous.

Consecutive `Metadata` messages that carry a `rtmp-verifiable-segment-info` for the same track and are not separated by any frame of that track belong to the segment they close, not to a new, empty segment. A segment boundary shall occur only at the completion of a frame; it shall not split a frame across two segments. As with the `segment-info-map` used for BMFF content, the `sequenceNumber` shall increase monotonically by 1 for each subsequent segment of a given `mediaType` and `trackId`, independently of any other track. A given session key shall be used to sign the `rtmp-verifiable-segment-info` of only one track (that is, of one combination of `mediaType` and `trackId`).

15.5.3.2. Segment hash computation

The `segmentHash` field of an `rtmp-segment-info-map` shall be computed, using the algorithm identified in its `alg` field, over the concatenation, in transmission order, of the following fields of each coded-frame packet of the segment:

1. the RTMP message timestamp of the message carrying the packet, as a 32-bit unsigned big-endian integer;
2. the composition time offset of the packet, as a 24-bit signed big-endian integer, or 0 if the packet carries none (for example, a `CodedFramesX` packet or an audio packet);
3. the coded media data of the packet (for example, `[AvcCodedData]`, `[HevcCodedData]` or `[AacCodedData]`, as defined by Enhanced RTMP (V2)).

From a Multitrack message, only the packet of the segment's track shall be included, using the RTMP message timestamp of the Multitrack message. The audio and video messages contained in an RTMP Aggregate Message (message type 22) shall be treated as if they had been transmitted individually, with their normalized timestamps as defined by Adobe's Real Time Messaging Protocol. No other bytes shall be included in the hash.

> NOTE
>
> Consequently, RTMP protocol control messages, RTMP Data Messages (such as

onMetaData), codec configuration (SequenceStart) packets (which are instead bound as described in Section 15.5.3.3, “Initialization data binding”), and all Metadata messages, including those carrying colorInfo, c2paSegmentInfo or c2paManifest, do not contribute to segmentHash.

NOTE

As with the moof box covered by bmffHash, an intermediary that rewrites RTMP message timestamps or composition time offsets invalidates segmentHash. The frame type is not included, as it is conveyed by the coded media data.

NOTE

The nanosecond timestamp offset that Enhanced RTMP (V2) carries as TimestampOffsetNano ModEx data is not included in segmentHash. An intermediary can therefore change or remove it without invalidating segmentHash, and a validator cannot rely on it being the value sent by the publisher.

15.5.3.3. Initialization data binding

A C2PA Manifest delivered in a c2paManifest [name, value] pair shall contain, as its hard binding, a data hash assertion without an exclusions field, whose hash is computed over the E-RTMP initialization data. The E-RTMP initialization data is the concatenation of the codec configuration payloads (for example, [AVCDecoderConfigurationRecord], [HEVCDecoderConfigurationRecord] or [AacSequenceHeader], as defined by Enhanced RTMP (V2)) carried in the most recent SequenceStart packet (or, for legacy codec signaling, the equivalent sequence header) of each track, ordered with video tracks before audio tracks and, within each media type, by ascending trackId (see Section 15.5.3.1, “Logical segment”). When validating that hard binding, the E-RTMP initialization data shall be treated as the content of the asset.

A publisher shall send the codec configuration of each track before the c2paManifest that binds to it. If codec configuration for any track is sent after a c2paManifest (for example, because of a change of codec configuration), the publisher shall send a new c2paManifest, bound to the updated E-RTMP initialization data, before the first coded frame that depends on that configuration.

A publisher shall send the codec configuration of each track on the same chunk stream as the coded frames of that track. A validator shall validate the hard binding of a C2PA Manifest delivered in a c2paManifest [name, value] pair when it validates the first segment, of any track, whose manifestId identifies that C2PA Manifest, rather than when the c2paManifest is received. That hard binding shall be validated against the E-RTMP initialization data formed from the codec configuration of each track most recently received before the c2paSegmentInfo that closes that segment.

```
rtmp-media-type = "audio" / "video"

; The `verifiable-segment-info` structure when carried over Enhanced RTMP
; (E-RTMP). The payload is the CBOR serialized `rtmp-segment-info-map`.
rtmp-verifiable-segment-info = COSE_Sign1_Tagged

; Payload map for `rtmp-verifiable-segment-info` when carried over Enhanced
; RTMP (E-RTMP), where segments are not packaged as BMFF/CMAF fragments.
rtmp-segment-info-map = {
  "mediaType": rtmp-media-type,
  ? "trackId": uint .le 255, ; E-RTMP trackId of the track; absent is equivalent to 0

  ; sequence number, increasing monotonically by 1 per segment of this mediaType/trackId
  "sequenceNumber": uint,
  "segmentHash": rtmp-hash-map, ; hash of the segment's media payload bytes
  "manifestId": tstr, ; identifier of the active manifest (see <<manifest-identifier>>)

  ; optional hashed-ext-uri-map to where manifest can be retrieved
  ? "manifestUri": $hashed-ext-uri-map
}
```

An example in CBOR diagnostic notation (RFC 8949, clause 8) is shown below:

```
{
  "mediaType": "video",
  "sequenceNumber": 175,
  "segmentHash": {
    "alg": "sha256",
    "hash": h'e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855'
  },
  "manifestId": "urn:c2pa:F9168C5E-CEB2-4FAA-B6BF-329BF39FA1E4"
}
```

15.5.3.5. Capability negotiation

Because E-RTMP is also used by endpoints that do not support this specification, support for C2PA Manifests shall be negotiated during the connection handshake using the `C2PA` bit (`0x10`) of `CapsExMask`, as defined by Enhanced RTMP (V2). A client that supports C2PA Manifests shall set this bit in the `capsEx` property of the Command Object of its `connect` command, and shall set the `objectEncoding` property of that same Command Object to 3. A server that supports C2PA Manifests shall set this bit in the Info Object it returns in the `_result` command. An endpoint shall not send a `c2paSegmentInfo` or `c2paManifest` [name, value] pair unless both endpoints have signaled support in this way, and an endpoint that has not signaled support should not expect to receive either.

An endpoint receiving a `VideoPacketType.Metadata` or `AudioPacketType.Metadata` message

shall ignore any [name, value] pair whose name it does not recognize, and shall continue to process the remaining pairs in that message.

> NOTE
>
> Capability negotiation ensures that a publisher does not depend on how a legacy E-RTMP endpoint handles unrecognized `Metadata` [name, value] pairs.

15.5.3.6. Reconnection

The C2PA state of a stream (the active C2PA Manifest, its session keys and the `sequenceNumber` of each track) is not affected by an Enhanced RTMP (V2) reconnection (for example, in response to a `NetConnection.Connect.ReconnectRequest`): segments sent on the new connection shall continue the `sequenceNumber` of each track. After reconnecting, a publisher shall send the active C2PA Manifest again in a `c2paManifest` [name, value] pair, after the codec configuration of each track and before the first coded frame.

> NOTE
>
> A reconnection can move the publisher to a different server instance, which has not received the C2PA Manifest sent on the previous connection. Sending it again allows segments sent on the new connection to be validated without depending on state from the previous connection.

> NOTE
>
> Because a segment is scoped to the message stream on which it is carried (see Section 15.5.3.1, “Logical segment”), a segment cannot span a reconnection: the first segment of each track on the new connection begins with the first frame of that track sent on that connection, and frames of a segment whose `c2paSegmentInfo` was not sent before the previous connection was lost cannot be validated. In response to a `NetConnection.Connect.ReconnectRequest`, a publisher can avoid this by sending the `c2paSegmentInfo` of the current segment of each track before disconnecting.

15.5.4. C2PA Manifest Updates

When different assertions are required for a new segment (for example, to indicate different provenance or replace expiring session keys), a new C2PA Manifest Store shall be provided, either embedded in the segment via a C2PA Manifest Box or linked externally, or for E-RTMP-delivered segments, via a `c2paManifest` [name, value] pair in a `VideoPacketType.Metadata` or `AudioPacketType.Metadata` message. The new manifest shall be used for all subsequent segments until another manifest is provided.

NOTE

This allows for dynamic updates to the C2PA Manifest information while maintaining the integrity and authenticity of the content.

15.5.5. Session Keys

15.5.5.1. Description

Session keys are asymmetric key pairs. Any of the cryptographic algorithms allowed for claim generator X.509 certificates may be used.

When using session-specific keys, they shall be generated at the start of a live media session, but may also be generated and delivered via updated manifests in later segments (either initialization or other segments). Each session key is used for a limited period, which enables key rotation. Session-specific keys should be updated when the provenance changes.

The public halves of the session keys are delivered inside a C2PA Manifest. The initial set of keys shall be part of the C2PA Manifest included in the initialization segment, or for E-RTMP-delivered segments, via a `c2paManifest` [name, value] pair in a `VideoPacketType.Metadata` or `AudioPacketType.Metadata` message sent before the `c2paSegmentInfo` that references them.

Session keys may also be retrieved from additional initialization segments, from the active C2PA Manifest included in a C2PA Manifest Box, from a `c2paManifest` [name, value] pair (for E-RTMP-delivered segments), or from the `manifestUri` included in a `verifiable-segment-info`. If a receiver finds a `verifiable-segment-info` signed with a session key not yet known to the receiver, the receiver should extract updated session keys from the manifest carried in the C2PA Manifest Box, if present. If no C2PA Manifest Box is present in the segment, then the receiver should attempt to retrieve a C2PA Manifest Store from the URL specified in the `manifestUri` field of the `verifiable-segment-info`, if present (see Section 11.8, “External Manifests”). If no C2PA Manifest is present in the current segment, the receiver shall look in the C2PA Manifest that accompanied the initialization segment. For E-RTMP-delivered segments, which have neither a C2PA Manifest Box nor an initialization segment, the receiver shall instead look in the C2PA Manifest delivered in a `c2paManifest` [name, value] pair whose manifest identifier matches the `manifestId` field of the `rtmp-segment-info-map` and, if no such C2PA Manifest has been received or the session key is not found there, should attempt to retrieve a C2PA Manifest Store from the `manifestUri` field of the `rtmp-segment-info-map`, if present. After retrieving the C2PA Manifest, the receiver shall ensure that its manifest identifier matches the one provided in the `manifestId` field of the `verifiable-segment-info`.

IMPORTANT

When using an externally hosted C2PA Manifest that is provided via a `manifestUri`

> in a `verifiable-segment-info`, that C2PA Manifest needs to exist at that URI for the entire period of time that the content is provided.

15.5.5.2. Session Keys Assertion

A session keys assertion is used to carry one or more public session keys inside a C2PA Manifest. The session keys assertion is described in Session Keys Assertion.

15.6. Live Media Architecture Diagram

Figure 14, “Live Media Architecture” shows the general scheme of all the various data structures that can be used in a live media workflow and how each is connected to the others.

the same way that is described in Section A.5, “Embedding manifests into BMFF-based assets”, but with the `bmff-hash-map` only hashing the bytes of that segment. In addition, a Section 15.4.2, “Live Media Segment Assertion” shall be included in the manifest to provide metadata about the segment.

15.7.2. Verifiable Segment Info method

When using the Verifiable Segment Info method, there are two main steps involved in generating the live media stream.

Session Key Generation

At the start of a live media session, one or more session keys are generated, each with its own `createdAt`, `validityPeriod`, and `minSequenceNumber`. These session keys are included in the C2PA Manifest in the C2PA Manifest Box that shall be present in the initialization segment(s), or in an E-RTMP `VideoPacketType.Metadata` or `AudioPacketType.Metadata` message. The `minSequenceNumber` for each key indicates the earliest segment that should be signed with that key, ensuring a clear transition between keys and supporting overlap for seamless key rotation.

> NOTE
>
> For content contributed over Enhanced RTMP (V2), each track (video, audio, etc.) uses its own session key (see Section 15.5.3.1, “Logical segment”).

> NOTE
>
> For limited-duration live media streams, a single session key could be sufficient. For longer live media streams (e.g., 24/7 streams), session keys are rotated periodically by delivering new session keys in subsequent initialization segments before the old session keys expire.

Signing the Box Hash

For BMFF/CMAF content, each segment of a live media or dynamically packaged stream shall have an `emsg` box containing the `verifiable-segment-info`, which is a `segment-info-map` signed by a session key. The `segment-info-map` carries a `sequenceNumber` to address out-of-order attacks, and carries a `bmff-hash-map` that hashes the content in the segment. When manifests change during the live media stream, the updated manifest may be included in a C2PA Manifest Box in the segment or referenced via the `manifestUri` of the `segment-info-map`.

> NOTE
>
> For content contributed over Enhanced RTMP (V2), a `rtmp-verifiable-segment-info` is instead carried in a `VideoPacketType.Metadata` or `AudioPacketType.Metadata` message and its payload is an `rtmp-segment-info-map`, as described in Section 15.5.3, “Carriage over Enhanced RTMP (E-RTMP)”.

> Session keys are delivered in `c2pa.session-keys` in a `c2paManifest` metadata message.

15.8. Live Media Validation Process

15.8.1. Initial/Common Validation Steps

If the segment being validated is an initialization segment, the validator shall look for the presence of an `mdat` box. If an `mdat` box is found, the validation shall fail with a failure code of `livevideo.init.invalid`.

If a BMFF/CMAF segment contains neither a C2PA Manifest Box nor an `emsg` box that meets the requirements defined above, the validation of that segment shall fail with a failure code of `livevideo.segment.invalid`.

If an E-RTMP-delivered segment is not closed by one or more `Metadata` messages carrying a `c2paSegmentInfo` that meets the requirements defined above, the validation of that segment shall fail with a failure code of `livevideo.segment.invalid`.

If a C2PA Manifest Box is found in a given segment, it shall be validated according to the general C2PA validation rules described in Chapter 14, *Validation*. If the C2PA Manifest fails validation, the validation of that segment shall fail with a failure code of `livevideo.manifest.invalid`.

For content contributed over Enhanced RTMP (V2), when a validator validates a segment, it shall have obtained the C2PA Manifest identified by the `manifestId` field of the segment's `rtmp-segment-info-map`, either from a `c2paManifest` [name, value] pair or from the `manifestUri` field of that `rtmp-segment-info-map` (see Section 15.5.5, "Session Keys"); if it has not, the validation of that segment shall fail with a failure code of `livevideo.init.invalid`. Any C2PA Manifest Store obtained this way shall be validated according to the general C2PA validation rules described in Chapter 14, *Validation*, with its hard binding validated against the E-RTMP initialization data as described in Section 15.5.3.3, "Initialization data binding"; if it fails validation, the validation of that segment shall fail with a failure code of `livevideo.manifest.invalid`.

When the hard binding of a C2PA Manifest delivered in a `c2paManifest` [name, value] pair is validated, the codec configuration of each track contained in the E-RTMP initialization data is bound by that C2PA Manifest. If, for any coded frame of an E-RTMP-delivered segment, the most recent codec configuration of its track received before that frame (carried in a `SequenceStart` packet or, for legacy codec signaling, the equivalent sequence header) is not the codec configuration of that track bound by the C2PA Manifest

identified by the `manifestId` field of the segment's `rtmp-segment-info-map`, the validation of that segment shall fail with a failure code of `livevideo.init.invalid`.

15.8.2. Per-segment C2PA Manifest Box Validation

If the C2PA Manifest present in the C2PA Manifest Box contains a live media segment assertion, the validator shall perform the following tests:

- Unless the segment is the first segment, ensure that the `sequenceNumber` field in the live media segment assertion is greater than the `sequenceNumber` of the previous segment. If it is not, the validation of that segment shall fail with a failure code of `livevideo.assertion.invalid`.
- Ensure that the `streamId` field in the live media segment assertion matches the `streamId` from the previous segment (if any). If they do not match, the validation of that segment shall fail with a failure code of `livevideo.assertion.invalid`.
- Ensure that the `continuityMethod` field is present in the live media segment assertion. If it is not, the validation of that segment shall fail with a failure code of `livevideo.continuityMethod.invalid`.
- If the `continuityMethod` field in the live media segment assertion has a value of "c2pa.manifestId", then the live media segment assertion shall contain a `previousManifestId` field. If it does not, the validation of that segment shall fail with a failure code of `livevideo.continuityMethod.invalid`. If it does, then ensure that its value matches the `manifestId` of the previous segment's C2PA Manifest. If they do not match, the validation of that segment shall fail with a failure code of `livevideo.segment.invalid`.
- If the `continuityMethod` field in the live media segment assertion has a value other than "c2pa.manifestId", and the validator does not recognize or support that continuity method, the validation of that segment shall fail with a failure code of `livevideo.continuityMethod.invalid`.

15.8.3. Verifiable Segment Info Validation

If the C2PA Manifest present in the C2PA Manifest Box contains a session keys assertion, the public session keys shall be extracted and stored for use in validating subsequent segments. If any public session key does not conform to the requirements described in Session Keys Assertion, the validation of that segment shall fail with a failure code of `livevideo.sessionkey.invalid`. Similarly, if signature validation (`signerBinding`) of any public session key structure fails, that session key is invalid and shall not be used to validate any media segment, and validation shall fail with a failure code of `livevideo.sessionkey.invalid`.

If a segment contains an `emsg` box that meets the requirements defined above, and the segment is not an initialization segment, and the validator does not have any valid session keys, the validation of that segment shall fail with a failure code of `livevideo.segment.invalid`.

If there is a valid `emsg` box containing a valid `verifiable-segment-info`, the validator shall extract the value of `kid` from its unprotected header and use that to identify the correct public session key. If such key is not available, and if the `manifestUri` field is present, the validator retrieves and verifies the applicable manifest, extracting the public session key(s) as explained above.

> NOTE
>
> There may not be valid session keys for a variety of reasons including (but not limited to) if the initialization segment containing the session keys assertion was not received, if a C2PA Manifest (containing the session keys) failed validation, or if the keys have expired.

Validation fails if the key cannot be found or if the signature in `verifiable-segment-info` cannot be verified with it, and shall fail with a failure code of `livevideo.segment.invalid`.

Subsequently, when parsing the `segment-info-map`, validation of that segment shall fail with a failure code of `livevideo.segment.invalid` if any of the following errors occurs:

- the segment is not the first segment, and its `sequenceNumber` is lower than or equal to the `sequenceNumber` of the previous segment;
- the segment's `sequenceNumber` is lower than the `minSequenceNumber` (of the matching session key);
- for BMFF/CMAF content, the segment's presentation time is outside the key's validity period;
- the segment's hash does not match the `bmffHash` field (see Section 16.3.2, "BMFF-Based Hash").

To address the risk of the Claim Generator's certificate getting revoked during a very long lived session, the validator should check that the certificate used to sign the C2PA Manifest from which the current session key was retrieved is still valid. If it is not, the validation of that segment shall fail with a failure code of `livevideo.sessionkey.invalid`.

15.8.4. Verifiable Segment Info Validation over Enhanced RTMP

For content contributed over Enhanced RTMP (V2), the validation steps described in Verifiable Segment Info Validation apply, with the `VideoPacketType.Metadata` or `AudioPacketType.Metadata` message carrying `c2paSegmentInfo` used in place of the `emsg` box, and the `rtmp-segment-info-map`'s `segmentHash` field (see Section 15.5.3, "Carriage over Enhanced RTMP (E-RTMP)") used in place of

`bmffHash`. Session keys are delivered in `c2pa.session-keys` in a `c2paManifest` metadata message.

Subsequently, when parsing the `rtmp-segment-info-map`, validation of that segment shall fail with a failure code of `livevideo.segment.invalid` if any of the following errors occurs:

- the segment is not the first segment of its track, and its `sequenceNumber` is lower than or equal to the `sequenceNumber` of the previous segment of the same track;
- the segment's `sequenceNumber` is lower than the `minSequenceNumber` (of the matching session key);
- the time at which the segment is validated is outside the key's validity period, that is, earlier than `createdAt` or later than `createdAt` plus `validityPeriod` seconds;
- the segment's hash does not match the `segmentHash` field;
- the session key identified by `kid` has already been used to sign the `rtmp-verifiable-segment-info` of a different track (that is, of a different combination of `mediaType` and `trackId`);
- the `Metadata` message carrying the `rtmp-verifiable-segment-info` does not belong to the track identified by its `mediaType` and `trackId` fields (see Section 15.5.3.1, "Logical segment").

Chapter 16. C2PA Standard Assertions

16.1. Introduction

This section of the document lists the standard set of assertions for use by C2PA implementations, describing their syntax, usage, etc. To keep things simple, all example JUMBF URIs have been shortened for illustrative purposes - full URIs are necessary in the actual data.

All assertions shall have a label as described in Section 8.2, “Labels” and shall be versioned as described in Chapter 5, *Versioning*.

All C2PA standardized assertions use the JSON JUMBF content type, the CBOR JUMBF content type, or the Embedded File content type from ISO 19566-5:2023. Entity-specific assertions can be any of those, any of the other JUMBF content types from ISO 19566-5:2023, Annex B (such as XML) or may create its own (as per the instructions in ISO 19566-5:2023, Table B.1). The Codestream content type shall not be used for a C2PA assertion.

Unless otherwise mentioned, all assertions documented in this standard set of assertions shall be serialized as CBOR. All assertions that are serialized as CBOR shall comply with the Core Deterministic Encoding Requirements of CBOR (see RFC 8949, clause 4.2.1) with their schemas defined using a CDDL Definition.

For those defined using JSON, their schemas are defined using the latest version of JSON Schema.

To make the standard assertions easier to navigate, they are grouped below into functional clusters rather than being presented as a single flat list. Two structures described in this section are used *by* assertions rather than being assertions in their own right: Metadata About Assertions, which describes optional metadata that may be attached to an assertion, and Regions of Interest, which identifies the spatial, temporal or other regions of an asset that an assertion refers to. Both are defined in Assertion-Supporting Structures at the end of this section.

16.2. Standard C2PA Assertion Summary

The standard C2PA assertions are listed in Table 3, “Standard C2PA assertions”, grouped by their function.

Table 3. Standard C2PA assertions

Function	Type	Assertion	Schema	Serialization
Content binding & integrity	BMFF-based Hash	c2pa.hash.bmff (removed), c2pa.hash.bmff.v2 (deprecated), c2pa.hash.bmff.v3	C2PA	CBOR
	Collection Data Hash	c2pa.hash.collection.data	C2PA	CBOR
	Data Hash	c2pa.hash.data	C2PA	CBOR
	General Box Hash	c2pa.hash.boxes	C2PA	CBOR
	Multi-Asset Hash	c2pa.hash.multi-asset	C2PA	CBOR
	Segment Hash	c2pa.hash.segment	C2PA	CBOR
	Soft Binding	c2pa.soft-binding	C2PA	CBOR
Provenance & workflow	Actions	c2pa.actions, c2pa.actions.v2	C2PA	CBOR
	Asset Reference	c2pa.asset-ref	C2PA	CBOR
	External Reference	c2pa.external-reference	C2PA	CBOR
	Linked Resource Integrity	c2pa.linked-resource-integrity	C2PA	CBOR
	Repository Receipt	c2pa.repository-receipt	C2PA	JSON-LD
Ingredient	Ingredient	c2pa.ingredient, c2pa.ingredient.v2, c2pa.ingredient.v3	C2PA	JUMBF Embedded File

Function	Type	Assertion	Schema	Serialization
Content description & referencing	Alternative Content Representation	c2pa.alternative-content-representation	C2PA	CBOR
	Asset Type	c2pa.asset-type (deprecated), c2pa.asset-type.v2	C2PA	CBOR
	Cloud Data	c2pa.cloud-data	C2PA	CBOR
	Depthmap	c2pa.depthmap.GDepth	https://developers.google.com/depthmap-metadata/reference	CBOR
	Embedded Data	c2pa.embedded-data	C2PA	JUMBF Embedded File
	Font Information	font.info	C2PA	CBOR
	Metadata	c2pa.metadata	C2PA	JSON-LD
	Thumbnail	c2pa.thumbnail.claim (claim creation time), c2pa.thumbnail.ingredient (importing an ingredient)	C2PA	Embedded File
Trust & validation adjuncts	Certificate Status	c2pa.certificate-status	C2PA	CBOR
	Session Keys	c2pa.session-keys	C2PA	CBOR
	Time Stamps	c2pa.time-stamp	C2PA	CBOR
Policy & disclosure	AI Disclosure	c2pa.ai-disclosure	C2PA	CBOR
	Environmental Sustainability	c2pa.environmental-sustainability	C2PA	CBOR

16.3. Content Binding & Integrity

These assertions bind a manifest to the content of an asset and enable a Manifest Consumer to detect tampering, using either cryptographic hashes of the asset's bytes or soft (perceptual) bindings.

16.3.1. Data Hash

16.3.1.1. Description

The most common way to uniquely verify the integrity of portions of a non-BMFF-based asset is via the hard bindings (i.e., cryptographic hash) present in data hash assertions. However, for those formats that are "box like" but not compatible with BMFF, the general box hash assertion is recommended.

The `alg` field, if present, shall be a string that is one of the allowed hash algorithms as defined in Section 12.1, "Hashing", and that algorithm shall be used to compute the hash. If the `alg` field is not present, then the hashing algorithm shall be determined by the value of the `alg` field in the Claim. The hash value shall be present in the `hash` field.

Each data hash assertion defines a specified range of bytes over which the hash has been computed. If only a portion of the asset shall be hashed, then the range(s) to be excluded shall be present in the array value of the `exclusions` field. These excluded ranges shall be ordered by increasing start position and shall not overlap.

For data hash exclusion ranges, the range shall begin and end within the same logical unit (e.g., box, segment, object) and shall not overlap with any header or length field associated with that unit, except for freebox or pad data. It is the responsibility of the claim generator to define exclusion ranges in a way that ensures that whatever data an attacker might place in those ranges cannot materially affect the interpretation of the asset. Furthermore, the claim generator shall ensure the exclusion range only contains content from C2PA Manifest Store, or asset metadata (e.g., EXIF, IPTC metadata). Example metadata that could be skipped can be unverified user name or image rotation information.

A previous version of this specification provided a `url` field to provide a pointer to where the hashed data can be located, but it was never used. This field is now deprecated in favour of the asset reference assertion. Claim generators shall not add this field to a data hash assertion, and consumers shall ignore the field when present, except this shall not affect inclusion of the field as part of the content being validated as described in Section 14.10.4, "Assertion Validation".

A data hash assertion shall have a label of `c2pa.hash.data`.

A data hash assertion shall not appear in a cloud data assertion or an external reference assertion.

```
    {
      "start": 9960,
      "length": 4213
    }
  ],
}
```

Normally, the `start` and `length` values of an `exclusion` shall be written in their preferred serialization (i.e., "as short as possible"). However, when a data hash assertion needs to be created but the `start` and `length` values are not yet known, they shall be created "as large as possible", which would be as a 32-bit integer.

The `pad` value shall be present and shall be a zero-filled byte string. The `pad2` value, if present, shall be a zero-filled byte string.

> NOTE
>
> Section 11.7, “Multiple Step Processing” describes how to fill in the correct values and adjust the padding.

16.3.1.3. Special consideration for JPEG 1

When hashing a JPEG 1 (.jpg) file into which the C2PA Manifest will be embedded, the APP11 marker (`FFEB`) and the segment’s length (`Lp`) of all APP11 segments containing the JUMBF data shall be included in the exclusion range.

> NOTE
>
> All the APP11 segments containing the C2PA Manifest JUMBF are contiguous so that only a single range is required.

16.3.1.4. Special consideration for PNG

When hashing a PNG (.png) file into which the C2PA Manifest will be embedded, it is important that the `Length` and the `'caBX'` (representing the chunk type) of the chunk containing the JUMBF data be included in the exclusion range.

16.3.1.5. Special considerations for TIFF-based

When hashing a TIFF-based asset into which the C2PA Manifest will be embedded, the `count` field of the C2PA Manifest Store’s IFD Entry (representing the length of the JUMBF data) should be included in the exclusion ranges. This is to support Update Manifests, which could change the size of the embedded C2PA Manifest Store.

16.3.1.6. Special considerations for RIFF-based

When hashing a RIFF-based asset into which the C2PA Manifest will be embedded, the size field of the top-level `RIFF` chunk (the four bytes at offset 4) should be included in the exclusion ranges. This is to support Update Manifests, which could change the size of the embedded C2PA Manifest Store; because this size field covers the entire file, any change to the size of the C2PA Manifest Store rewrites it.

16.3.2. BMFF-Based Hash

16.3.2.1. Description

Portion(s) of a BMFF-based asset that a claim generator wishes to uniquely identify with a hard binding (i.e., cryptographic hash) shall be described using BMFF-based hash assertions.

A BMFF-based hash assertion shall have a label of `c2pa.hash.bmff.v3`.

All Merkle trees used in BMFF-based hash assertions shall be balanced binary trees with the minimum depth necessary to accomodate leaf nodes, where NULL padding is applied only to the rightmost positions as needed to maintain the complete binary tree structure. This also requires that each level be more than half full, see Section 16.3.2.6, “Validation”.

NOTE	Earlier versions of this specification also documented `c2pa.hash.bmff` and `c2pa.hash.bmff.v2` assertions.

IMPORTANT	Validators shall ignore any `c2pa.hash.bmff` assertions, processing the manifest as if the assertion were not present.

A BMFF-based hash assertion shall not appear in a cloud data assertion nor an external reference assertion.

A previous version of this specification provided a `url` field to provide a pointer to where the hashed data can be located, but it was never used. This field is now deprecated in favour of the asset reference assertion. Claim generators shall not add this field to a BMFF hash assertion, and consumers shall ignore the field when present, except this shall not affect inclusion of the field as part of the content being validated as described in Section 14.10.4, “Assertion Validation”.

16.3.2.2. Hash Computation

The `alg` field, if present, shall be a string that is one of the allowed hash algorithms as defined in Section 12.1, “Hashing”, and that algorithm shall be used to compute the hash. If the `alg` field is not present, then the

hashing algorithm shall be determined by the value of the `alg` field in the Claim.

For a non-fragmented MP4 asset, the `hash` field, the `merkle` field, or both fields shall be present. For a fragmented MP4 asset validated using a Merkle tree, the `merkle` field shall be present and the `hash` field shall be absent. For live media segments used by any of the methods defined in Chapter 15, *Live Media*, the `hash` field shall be present and the `merkle` field shall be absent.

To compute the hash specified in the value field of a BMFF hash, all bytes of the file are added to the hash excluding those BMFF boxes or subset[s] thereof which match any exclusion entry in the `exclusions` array.

Boxes that are included in their entirety also include their box headers in the input data contributed to the hash. Similarly, boxes that are excluded in their entirety also exclude their box headers from the input data contributed to the hash. When a box is partially excluded from the input data contributed to the hash through the use of a `subset` field in the exclusion specification, the portion(s) of the box to be excluded defined by the relative byte offsets in the `subset` field are offsets from the start of the box including the box headers, not offsets from the start of the box's content. These `subset` ranges shall be ordered by increasing `offset` value and shall not overlap.

In a `c2pa.hash.bmff.v2` (deprecated) and `c2pa.hash.bmff.v3` assertion, for any root box not excluded in its entirety, the input data contributed to the hash for that box is comprised of the concatenation of the binary strings `offset || data`, where `offset` is defined as the absolute file offset of the box as an 8-byte integer in big-endian format, and `data` is defined as the box's contents, including headers, minus any exclusions. In this definition, "||" represents the binary concatenation of the two. The offset shall not be included for Merkle tree hashes when the bmff-hash-map includes both the `hash` and `merkle` fields.

In addition, `c2pa.hash.bmff.v2` (deprecated) and `c2pa.hash.bmff.v3` assertions include the following features:

- The absolute file byte offset is included at the start of the input data contributed to the hash for any root box. This ensures that a root box included in the hash cannot change positions in the file.
- The `mdat` box is no longer excluded in its entirety when the bmff-hash-map includes both the `hash` and `merkle` fields. Instead, a mandatory entry on the exclusion list excludes most of the box.

> NOTE
>
> These two features ensure that the `mdat` cannot change positions in the file while also eliminating the need for the offset for each individual Merkle tree hash when the bmff-hash-map includes both the `hash` and `merkle` fields.

A box matches an exclusion entry in the `exclusions` array if and only if all of the following conditions are

met:

- The box's location in the file matches the `exclusions-map` entry's `xpath` field. For example, exclusion xpath `/foo/bar[2]` would match locations `/foo[3]/bar[2]` and `/foo[2]/bar[2]`, but not `/foo[3]/bar[1]` or `/foo[3]/bar[2]/baz[1]`.
- If `length` is specified in the `exclusions-map` entry, the box's length exactly matches the `exclusions-map` entry's `length` field. Note: The length includes the box headers.
- If `version` is specified in the `exclusions-map` entry, the box is a FullBox and the box's version exactly matches the `exclusions-map` entry's `version` field.
- If `flags` (byte array of exactly 3 bytes) is specified in the `exclusions-map` entry and the box is a FullBox. If `exact` is set to true or not specified, the box's flags (bit(24), i.e., 3 bytes) also exactly matches the `exclusions-map` entry's `flags` field. If `exact` is set to false, the bitwise-and of the box's flags (bit(24), i.e., 3 bytes) with the `exclusions-map` entry's `flags` field exactly matches the `exclusions-map` entry's `flags` field (i.e., the box has at least those bits set but may also have additional bits set).
- If `data` (array of objects) is specified in the `exclusions-map` entry, then for each item in the array, the box's binary data at that item's relative byte `offset` field exactly matches that item's `bytes` field.

The `xpath` field's string syntax shall be limited to the following strict subset.

- Only abbreviated syntax shall be used.
- Only full paths shall be used.
- Only node selection via `node` or `node[integer]` shall be used.
- Descendent syntax, i.e., `//`, shall NOT be used.
- All nodes shall be BMFF `4cc` codes.

Example 2. Complete Syntax for `xpath` Field

```
  xpath = '/' nodes
  nodes = node
        | node '/' nodes
  node = box4cc
       | box4cc '[' integer ']'
Where:
  box4cc is any 4cc allowed by ISO/IEC 14496-12 for a BMFF box.
  integer is any non-zero positive integer with no leading zeros.
```

Any given exclusion entry may match zero or more boxes. It is not required that an exclusion entry match exactly one box.

A non-leaf xpath node shall only point to a container box that has no fields of its own (i.e., contains no data, only child boxes) and that does not inherit from FullBox. This ensures that a C2PA validator does not need to be aware of the syntax and semantics of unusual boxes that contain other boxes. If a child box of such an unusual box needs to be excluded in full or in part, the `exclusions-map` entry's `xpath` field shall point to the unusual box itself and the `subset-map` field shall exclude the byte range(s) containing the excluded child box data. For example, the `'sgpd'` box contains other boxes but is unusual in that it inherits from FullBox; as such, if excluding child box(es), in whole or in part, from `'sgpd'` is required, the assertion shall use an `xpath` field pointing to the `'sgpd'` itself (e.g., `/moof/traf/sgpd`) and shall use the `subset-map` field to exclude the desired bytes.

If the C2PA Manifest is embedded into the file, the box containing it shall be one of the entries in the `exclusions` array. Refer to Section A.5, "Embedding manifests into BMFF-based assets" for more information.

If a non-root excluded box is removed after the C2PA Manifest is created it shall be replaced with a `'free'` box of the same size to ensure that the input data contributed to the hash for other boxes are not invalidated. If C2PA Manifest store size is reduced by using compressed manifest after the C2PA Manifest is created, a `'free'` box shall be inserted in its place to ensure the offsets remain the same. If it is expected that a non-root excluded box may be added after the C2PA Manifest is created, then at manifest creation time, a `'free'` box shall be inserted with sufficient space for the excluded box and that `'free'` box shall also be excluded by an exclusion entry using its full xpath. When the excluded box is added or the C2PA Manifest store size is increased, the `'free'` box shall be shrunk (or removed) to compensate for the added data. However if there is insufficient space in the `'free'` box, a standard manifest shall be used.

Embedding C2PA data into a BMFF-based asset via MP4 boxes changes file offsets in other MP4 boxes as well as the absolute file byte offsets being included in the input data contributed to the hash for any root box. Those boxes and offsets shall be included in the input data contributed to the hash with their post-embed values, not their pre-embed values, or the BMFF-based hash assertion will not validate.

There are three possible ways an implementation can ensure that post-embed values for all file byte offsets are hashed:

1. Use `'free'` boxes.
    a. Determine reasonable maximum size(s) for the C2PA box(es) which will be embedded. All MP4 boxes for C2PA support unused padding bytes at the end, so it is fine to overestimate the size for the `'free'` boxes because any extra bytes will be ignored.

b. Insert `'free'` box(es) of said size(s) into the asset file(s) and update all offsets appropriately.

c. Perform hashing of the asset with "/free" on the exclusion list.

d. Create and sign the manifest. Create the C2PA box(es).

e. Overwrite the `'free'` box(es) with the C2PA box(es).

2. Use a two-pass approach.

a. Compute the exact sizes of the BMFF-based hash assertion and the `merkle` box(es) if any. The latter will require parsing the asset file(s) to determine the size of the Merkle tree.

b. Compute the exact size of the final manifest.

c. Perform hashing of the asset file(s). Update any box that includes any file offsets to correct values before including that box in the input data contributed to the hash. Compute the input data contributed to the hash using `(offset || data)` using the updated absolute file offset as described above. As indicated above, the offset is not included in the data contributed for Merkle tree hashes when the bmff-hash-map includes both the `hash` and `merkle` fields.

d. Create and sign the manifest. Create the C2PA box(es).

e. Insert the C2PA box(es).

3. Place Update Manifests at end of BMFF file.

a. Set original Manifest Store `box_purpose` from `manifest` to `original`.

b. Create and sign the Update Manifest.

c. Create Manifest Store with `box_purpose update`.

d. Insert signed Update Manifest into new Manifest Store with `box_purpose update` or append the Update Manifest if there is an existing Manifest Store with `box_purpose update`.

e. When present, the Manifest Store with box `box_purpose update` shall appear at the end of BMFF file.

f. If a standard manifest is later added when a Manifest Store with box `box_purpose` set to `update` is present, its contents shall be moved to the Manifest Store with box `box_purpose original`.

g. The update manifest store is then removed from the end of the file, allowing backward compatibility with a single manifest for common use-cases.

h. The Manifest Store with box `box_purpose original` shall be changed back to `box_purpose manifest` and the standard manifest shall be added as normal.

NOTE

The `box_purpose` field is not included within the hash and can change without

```
  ? "sequenceNumber": uint,
}

; (optional) CBOR byte string of exactly 3 bytes.
flag-type = bytes

flag-t = flag-type .eq 3

exclusions-map = {
  ; Location of box(es) to exclude from the hash starting from the root node as
  ; an xpath formatted string of version https://www.w3.org/TR/xpath-10/ with
  ; highly constrained syntax.
  "xpath": tstr,

  ; (optional) Length that a leafmost box must have to exclude from the hash.
  ? "length": uint,

  ; (optional) The data in the leafmost box at the specified relative byte
  ; offset must be identical to the specified data for the box to be excluded
  ; from the hash.
  ? "data": [1* data-map],

  ; (optional) Only this portion of the excluded box is excluded from the hash.
  ; Each entry in the array must have a monotonically increasing relative byte
  ; offset. No subset within the array may overlap. The last entry may have a
  ; length of zero; this indicates that the remainder of the box from that
  ; relative byte offset onward is excluded. A relative byte offset or relative
  ; byte offset plus length that exceeds the length of the box is allowed; bytes
  ; beyond the end of the box are never hashed.
  ? "subset":[1* subset-map],

  ; (optional) Version that must be set in a leafmost box for the box to be
  ; excluded from the hash. Only specified for a box that inherits from FullBox.
  ? "version": int,

  ; (optional) byte string of exactly 3 bytes. The 24-bit flags that must be set
  ; in a leafmost box for the box to be excluded from the hash. Only specified
  ; for a box that inherits from FullBox.
  ? "flags": flag-t,

  ; (optional) indicates that flags must be an exact match. If not specified,
  ; defaults to true. Only specified for a box that inherits from FullBox and
  ; when flags is also specified.
  ? "exact": bool,
}

data-map = {
  "offset": uint,
  "value" : bstr,
}
subset-map = {
  "offset": uint,
  "length": uint,
```

```
}

; Each entry in a map is a Merkle tree row and the associated data required to
; enable validation of a single
; 'mdat' box or multiple 'mdat' boxes within the asset.",
merkle-map = {
  ; 1-based unique id used to differentiate across files to determine which
  ; Merkle tree should be used to validate a given 'mdat' box.
  "uniqueId": int,
  "localId": int, ; A local id indicating Merkle tree.

  ; Number of leaf nodes in the Merkle tree. Null nodes are not included in this
  ; count.
  "count": int,

  ; A string identifying the cryptographic hash algorithm used to compute the
  ; hashes in this Merkle tree, taken from the C2PA hash algorithm identifier
  ; list. If this field is absent, the hash algorithm is taken from the `alg`
  ; value of the enclosing structure as defined by that structure. If both are
  ; present, the field in this structure is used. If no value is present in any
  ; of these places, this structure is invalid; there is no default.
  ? "alg": tstr .size (1..max-tstr-length),

  ; For fragmented MP4 assets that are split across multiple files, this field
  ; is required to be present and is the hash of the entire initialization
  ; segment file for chunks hashed by this Merkle tree excluding boxes listed in
  ; the exclusions array. For fragmented MP4 assets that are stored as a single
  ; flat MP4 file, this field is required to be present and is the hash of all
  ; bytes preceding the first 'moof' box excluding boxes listed in the
  ; exclusions array. For non-fragmented MP4, this field is required to be
  ; absent.
  ? "initHash": bstr,

  ; An ordered array representing a single row of the Merkle tree, which may be
  ; the leaf-most row, root row, or any intermediate row. Any null nodes are
  ; excluded. The depth of the row is implied by (is computed from) the number
  ; of items in this array.
  "hashes": [1* bstr],

  ; For non-fragmented MP4 assets where the mdat box is validated piecewise,
  ; this field can be present. This field is the non-negative size in bytes of a
  ; given leaf node in the Merkle tree. For fragmented MP4, this field is not
  ; present.
  ? "fixedBlockSize": uint,

  ; For non-fragmented MP4 assets where the mdat box is validated piecewise,
  ; this field can be present. Each entry in the array corresponds to the
  ; non-negative size in bytes of a given leaf node in the Merkle tree. The
  ; number of elements is equal to `count` and sum of the values is equal to
  ; size of payload of mdat. For fragmented MP4, this field is not present.
  ? "variableBlockSizes": [1* int],
}
```

```
    },
    {
      "data": [
        {
          "value": b64'9Q==',
          "offset": 5
        },
        {
          "value": b64'UAJXD79SlkG9rfnmcsqTUA==',
          "offset": 20
        },
        {
          "value": b64'OxKM',
          "offset": 70
        }
      ],
      "flags": b64'ZDNx',
      "xpath": "/emsg",
      "length": 200,
      "subset": [
        {
          "length": 7,
          "offset": 5
        },
        {
          "length": 28,
          "offset": 20
        },
        {
          "length": 63,
          "offset": 45
        },
        {
          "length": 112,
          "offset": 80
        }
      ],
      "version": 1
    }
  ]
}
```

A pseudo-code implementation of this algorithm is in Example 3, “Pseudo-code for BMFF-based hash

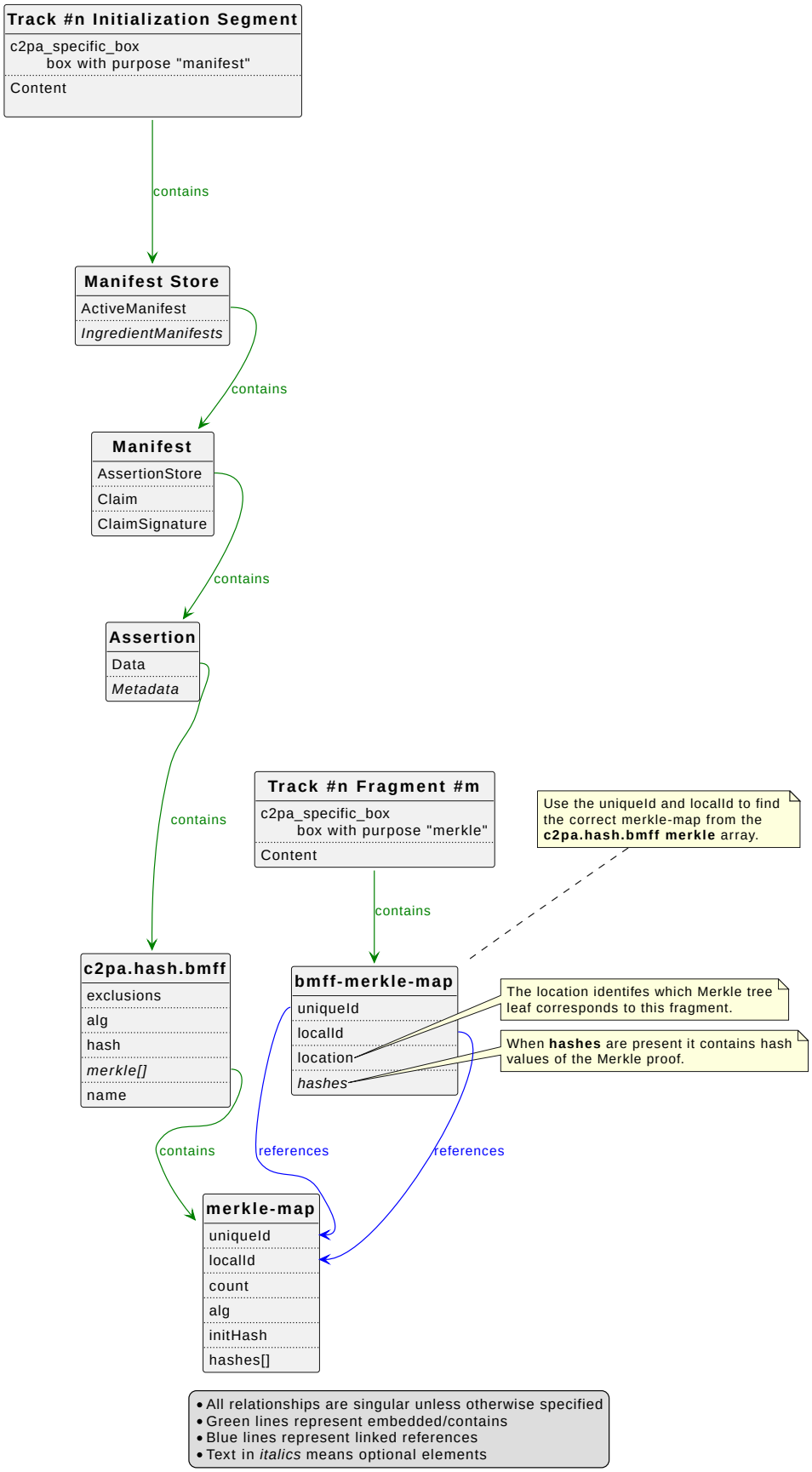


Figure 15. Fragmented BMFF Entity Diagram

16.3.2.6. Validation

Validating a given chunk requires first validating the `merkle-map` field's `initHash` over the corresponding initialization segment and then locating the correct entry in the `merkle-map` field's `hashes` array and validating it against the hash of the chunk's data, and if needed, deriving that hash using the Merkle proof from the `hashes` specified in the chunk's `bmff-merkle-map`.

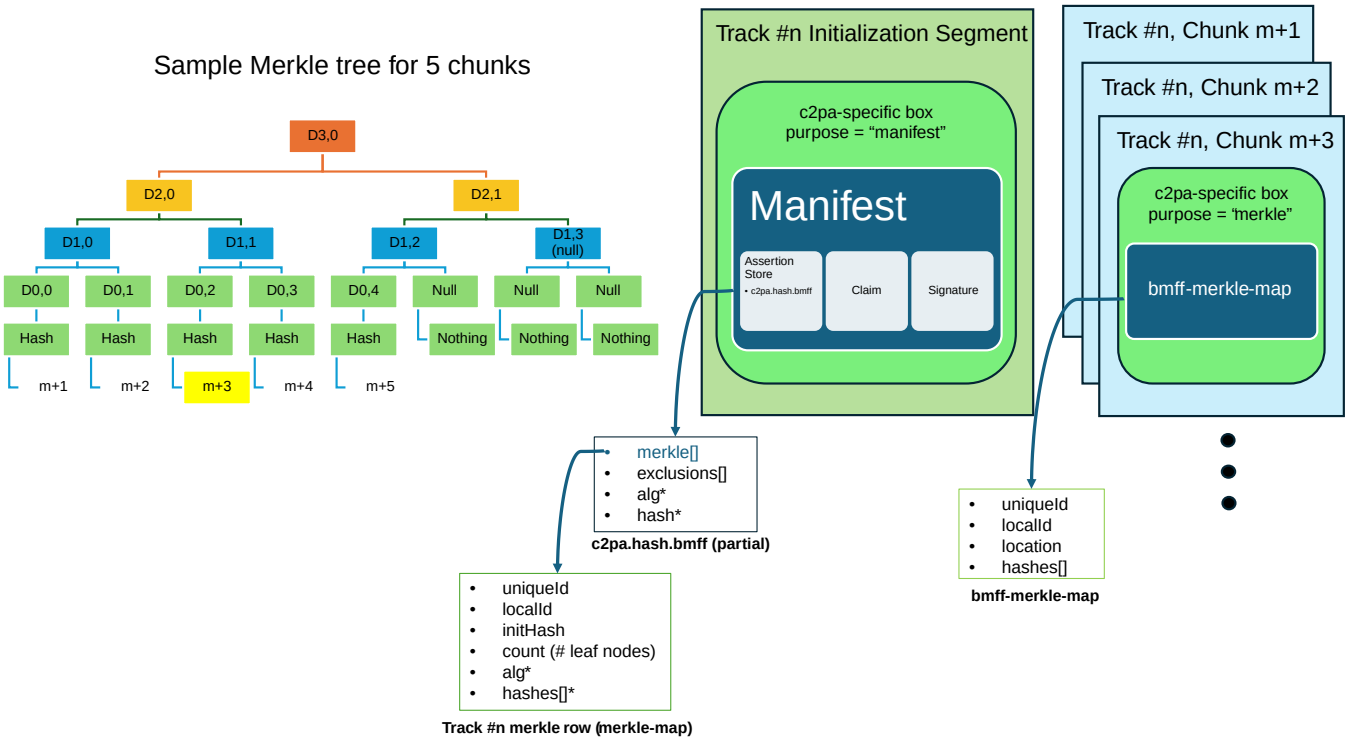


Figure 16. Validating intitialization segment and a chunk's data example

To verify track chunk m+3 you must first verify the corresponding initialization segment. The c2pa-specific manifest box in each Track's initialization segment will contain the Manifest store. If the asset contains multiple initialization segments then the Manifest store must be identical in each. This allows validators to verify a Track belong to the larger set. The active manifest's `c2pa.hash.bmff.v3` assertion will contain a `merkle` field with an array of `merkle-map` objects, one per track.

16.3.2.6.1. Steps

1. From the `bmff-merkle-map` in chunk's c2pa-specific merkle box obtain the `uniqueId` & `localId`. Use the `uniqueId` and `localId` to find a matching `merkle-map` from the `c2pa.hash.bmff.v3` assertion `merkle` array in the init segment.
2. If the hash of the init segment using the `c2pa.hash.bmff.v3` `exclusions` and the `merkle-map` `alg` equals the `initHash` inside the `merkle-map` you just located, the initialization segment is verified.

To complete verification of chunk m+3: We are looking at Track #n's `merkle-map` found in step 1, and in this example it contains row 2 of the Merkle tree - D2,0 and D2,1.

3. Hash chunk m+3 using the `c2pa.hash.bmff.v3` `exclusions` array and the `alg` from the `merkle-map`, yielding D0,2(derived).
4. Chunk m+3's `bmff-merkle-map` `hashes` array (Merkle proof) will contain the hash of chunk m+4 (D0,3) and row one hash value D1,0.
5. Hash D0,2(derived) and D0,3 to yield D1,1(derived). Hash D1,0 with D1,1(derived) to yield D2,0(derived).
6. If D2,0(derived) = D2,0 as stored in the assertion `merkle-map` `hashes` parameter, and the corresponding initialization segment was verified in step 2, then chunk m+3 has been verified.

16.3.3. General Box Hash

16.3.3.1. Description

A claim generator should use a general box hash assertion to verify the integrity, with a hard binding (i.e., cryptographic hash), of assets whose formats use a non-BMFF-based box format such as JPEG 1, PNG, or GIF.

A general box hash assertion shall have a label of `c2pa.hash.boxes`. Such an assertion consists of an array of structures, each one listing one or more boxes (by their name/identifier) and a hash that covers that data of those boxes (and any possible data that may be present in the file between them), along with the algorithm used for hashing. The boxes shall appear in the assertion in the same order that they appear in the asset, including the box containing the C2PA Manifest. If there are any other boxes present in the asset that are not explicitly included in this assertion, or if the boxes appear out of order, the manifest will be rejected during validation as described in Section 14.12.3, "Validating a general box hash".

A box may also have an `excluded` field, which is a boolean value indicating whether a validator can ignore this box (and associated hash) during validation. If this field is absent, or the field is present and its value is `false`, the box shall be hashed and the values compared. For boxes that have an `excluded` field with a value of `true`, the claim generator should include an accurate hash for compatibility with older validators that do not recognize the `excluded` field. If the claim generator is not concerned with backwards compatibility, it should write the binary string `00` (a single byte with a value of 0) for the hash.

A box may also include an `exclusions` field, which is an array of byte ranges to be excluded from the hash calculation. Each exclusion range shall be a map with `start` and `length` fields, indicating the byte offset (from the start of the box) and number of bytes to exclude. When multiple boxes exist in a `box-hash-map` structure, it is necessary to specify which box an exclusion range belongs to by using a `boxIndex` field. The

first `boxIndex` in the `box-hash-map` structure is 0, 1 for the next, and so on. These ranges shall be ordered by increasing start offset and shall not overlap.

For box hash exclusion ranges, the range shall begin and end within one box and shall not overlap with any header or length field associated with that box, except for freebox or pad data. It is the responsibility of the claim generator to define exclusion ranges in a way that ensures that whatever data an attacker might place in those ranges cannot materially affect the interpretation of the asset. Furthermore, the claim generator shall ensure the exclusion range only contains content from C2PA Manifest Store, or asset metadata (e.g., EXIF, IPTC metadata). Example metadata that could be skipped can be unverified user name or image rotation information.

The `excluded` and `exclusions` fields serve different purposes. `excluded` is a boolean flag indicating that the entire box may be ignored during validation. `exclusions` is a list of byte ranges within the box that should be excluded from the hash calculation. If both fields are present, the `excluded` field is set to `true` takes precedence and the `exclusions` field shall be ignored. When the `excluded` field is set to `false` or the `excluded` field is absent, the `exclusions` field shall be applied.

In the case where there are multiple instances of the same box type, such as multiple `APP1` segments in a JPEG 1 file, each instance shall be listed separately in the assertion. JPEG segments that are fragments sharing the same segment identifier are also listed as separate boxes, with the exception of the segments comprising the C2PA Manifest Store (as described below).

The `alg` field in `box-hash-map`, if present, shall be a string that is one of the allowed hash algorithms as defined in Section 12.1, “Hashing”, and that algorithm shall be used to compute the hash. If the `alg` field is not present in `box-hash-map`, then the hashing algorithm shall be determined by the value of the `alg` field in the containing `box-map`, or if that is not present, by the value of the `alg` field in the Claim.

The creation of the hashes is described in Section 12.1, “Hashing”, and the value shall be present in the `hash` field. The hash value for a range of boxes shall be computed from the start of the first box (in the range) until the end of the last box (in the range). This would include any arbitrary bytes that may be present between boxes.

> NOTE
>
> When using a range of boxes, all data between the start of the first box and the end of the last box is included in the hash. However, when listing each box separately, additional data is not included, only data within the listed box.

The box containing the C2PA Manifest Store (e.g. `caBX` for PNG, or `21FF` for GIF) shall also be listed in its own array. In order to clearly identify it as the C2PA Manifest box, it shall have the name `C2PA` and the value of `hash` shall be the binary string `00` (a single byte with a value of 0). The C2PA Manifest Store shall be

represented as a single box, even in the case of a JPEG 1 file where the box is fragmented across multiple APP11 marker segments.

> NOTE
>
> As validators are often used in combination with output of file parsers, it is a security best practice to hash all of the file content outside of the C2PA Manifest Store. This will ensure the integrity of the media and the linked manifest.

The optional pad and pad2 values, if present, shall be zero-filled byte strings.

> NOTE
>
> Section 11.7, "Multiple Step Processing" describes how to fill in the correct values and adjust the padding.

A General Box Hash assertion shall not appear in a Cloud Data assertion or a External Reference assertion.

16.3.3.2. Special handling of multi-part assets

To support file formats that consist of multiple parts (as described in Section 16.3.5, "Multi-Asset Hash"), one additional logical box is defined for cases where the data of one or more parts comes after the box-based data of the primary part. This box shall be labelled `c2pa.after` (for arbitrary data beyond the end of the box structure). The `c2pa.after` box, if present, shall be the last box listed, and its hash shall be computed from the byte following the last box until the end of the physical file.

The hard binding assertion, which covers the whole asset, shall be the only assertion that can include a `c2pa.after` box. A hash assertion for an individual part shall cover only the contents of that part itself, and not any other part.

16.3.3.3. Handling for specific formats

16.3.3.3.1. JPEG 1-specific Handling

When working with JPEG 1, the APP11 box is used for standards other than C2PA (i.e., JPEG 360). In those situations, all non-C2PA APP11 boxes shall be included in the list of hashed boxes. The APP11 boxes containing the C2PA Manifest Store shall be identified by C2PA. All other boxes shall be identified by the symbol found in ISO 10918-1:1994, Table B.1.

The C2PA Manifest Store can be identified by it being a JUMBF superbox with a label of c2pa and a JUMBF type UUID of 63327061-0011-0010-8000-00AA00389B71 as described in Section 11.5, "Serialization".

> NOTE
> The Start of Scan box and Restart boxes, label of `SOS` and `RST[n]`, will include the entropy coded segments following the respective marker.

The CIPA Multi-Picture Format (MPF) extension to JPEG 1 can also be supported using this method by listing all boxes contained in the file as they appear, assuming there is no data between the `EOI` of one Individual Image and the `SOI` of the next. The `boxes` list would enumerate the segments from each Individual Image in the MPF in sequence (`SOI`, …, `EOI`, `SOI`, …, `EOI`, …). However, if the claim generator plans to treat the MPF file as a multi-part asset, then the `c2pa.after` box shall be used to hash the additional parts that follow the `EOI` of the first Individual Image (the primary part).

16.3.3.3.2. PNG-specific Handling

A PNG file always begins with an 8 byte header (`89 50 4E 47 0D 0A 1A 0A`). To include it, use the special value `PNGh` as the first box in the list of boxes and start hashing from the first byte of the image.

16.3.3.3.3. TIFF-specific Handling

> NOTE
> Previous versions of this specification documented use of general box hash with TIFF. However, the use of indirection and other characteristics of the TIFF structure make box hashing awkward, inefficient, and prone to interoperability issues, so it has been decided to remove the general box hash support for TIFF, in favor of the data hash.

16.3.3.3.4. GIF-specific Handling

The hash of a box containing a 'Packed Fields' attribute will also hash the optional data indicated by that attribute. For example, the Image Descriptor will include the Local Color Table block, and the Logical Screen Descriptor will include the Global Color Table block, if they exist.

For all boxes containing a block label, the naming convention shall be as follows: "<Block Label>".

For all extension blocks, the naming convention is as follows: "<Extension Introducer><Extension Label>".

The only other blocks that are not described by the above naming convention are:

- The header will be marked with "GIF89a".
- The Table Based Image Data will be marked with "TBID".
- The Logical Screen Descriptor will be marked with "LSD".

For example:

- Header: "GIF89a".
- Trailer: "3B".
- Image Descriptor: "2C".
- Comment Extension: "21FE".

16.3.3.3.5. RIFF-specific Handling

RIFF file chunks may be nested in a tree structure of arbitrary depth. The root of this structure consists of one or more L0 chunks, each with the chunk identifier of `RIFF`. These `RIFF` chunks are defined with the following structure:

- Bytes 0-3: Chunk identifier, always `RIFF`.
- Bytes 4-7: Chunk length (minus 8 bytes for the chunk identifier and chunk length fields).
- Bytes 8-11: Media type identifier.
- Bytes 12-n: Chunk data (all L1 chunks).

After the media type identifier, the `RIFF` chunk may contain one or more L1 sub-chunks, each with the following structure:

- Bytes 0-3: Chunk identifier.
- Bytes 4-7: Chunk length (minus 8 bytes for the chunk identifier and chunk length fields).
- Bytes 8-n: Chunk data.
- Byte n+1: Padding byte (if necessary).

A special chunk identifier of `LIST` may be used to nest chunks within an L1 chunk. These `LIST` chunks mimic the structure of L0 `RIFF` chunks:

- Bytes 0-3: Chunk identifier, always `LIST`.
- Bytes 4-7: Chunk length (minus 8 bytes for the chunk identifier and chunk length fields).
- Bytes 8-11: List type identifier.
- Bytes 12-n: Chunk data (all L2 chunks).
- Byte n+1: Padding byte (if necessary).

For the purposes of calculating a general box hash, each L0 chunk shall be treated as a single box with a size of exactly 12 bytes, and a box name equal to the media type identifier (bytes 8-11). Each non-`LIST` L1 chunk shall be treated as a box with a name equal to the chunk identifier (bytes 0-3) and content extending from the beginning of the chunk identifier (byte 0) to the padding byte, if any, inclusive. Each `LIST` L1 chunk shall be treated as a box with a name equal to the list type identifier (bytes 8-11) and content extending from the beginning of the chunk identifier (byte 0) to the padding byte, if any, inclusive. All chunks nested within a `LIST` L1 chunk (L2 and higher) shall be treated as a part of the `LIST` L1 chunk's content and hashed as a single box.

In all cases, padding bytes shall be treated as part of the preceding chunk's content, and shall be included in the hash for that box.

16.3.3.3.6. Font-specific Handling

The tables of a font correspond directly to the hash boxes, including the `C2PA` table.

Tables are always enumerated in the order they appear in the font's table directory.

Note that the table directory itself is not part of the hashed content, and therefore not covered by any box.

The `checkSumAdjustment` value shall be treated as zero (`0`) when computing the hash for the box containing the `head` table.

The grouping, or lack thereof, of Font tables in the general box hash assertion is up to the claim generator.

Note: Fonts created for wide distribution may benefit from assigning each table to an individual box; in this way, if the font is re-packaged in another format, its hash will continue to validate correctly. By contrast, systems which generate large numbers of fonts automatically, such as a subsetter, may choose to combine tables into fewer boxes to streamline processing. In this case, the box hash(es) may not validate following a format transformation, due to the inclusion of inter-table padding.

Because font consumers shall not react to tables they do not recognize, existing font-handling infrastructure will expect that the `head` table's `checkSumAdjustment` value incorporate the final settled content of the `C2PA` table itself, including any local manifest in its entirety.

16.3.3.3.7. Ogg-specific Handling

In an Ogg file, the structure consists of one or more logical bitstreams, and the data for each stream is held in Ogg Pages. For the purpose of hashing, each logical bitstream is treated as a single 'box', which includes every Page, in order, with the same bitstream_serial_number. The C2PA Manifest Store 'box' is the logical bitstream

```
  ; An array of hash exclusion ranges. Ranges have monotonically increasing
  ; `start` values, and no two ranges may overlap.
  ? "exclusions": [1* box-exclusions-map],
  ? "pad": bstr, ; zero-filled byte string used for filling up space
  ? "pad2": bstr, ; zero-filled byte string used for filling up space
}

box-name /= tstr .size (1..10)

box-exclusions-map = {
  "start": uint, ; Starting byte offset (from the start of the box) of the range
  "length": uint, ; Number of bytes of data to exclude

  ; 0-based index into the names array in the box-hash-map. Can be omitted if
  ; the number of box is one.
  ? "boxIndex": int,
}
```

Five examples in CBOR diagnostic notation (RFC 8949, clause 8) are shown in Example Box Hash:

1. JPEG 1;
2. PNG;
3. GIF;
4. TTF;
5. JXL.

Example Box Hash

```
// JPEG 1 Example //
// with the first box containing exclusions //
{
    "alg" : "sha256",
    "boxes": [
        {
            "names" : ["SOI", "APP0", "APP2"],
            "hash" : b64'...',
            "pad" : b64'',
            "exclusions": [
                {
                    "start": 54,
                    "length": 4,
                    "boxIndex": 1,
                },
                {
                    "start": 1234,
                    "length": 56,
                    "boxIndex": 1,
                },
```

```
                {
                    "start": 54,
                    "length": 4,
                    "boxIndex": 2,
                }
            ],
        },
        {
            "names" : ["C2PA"],
            "hash" : b64'AA==',
            "pad" : b64'',
        },
        {
            "names" : ["DQT", "SOF0", "DHT", "SOS", "RST0", "RST1", "EOI"],
            "hash" : b64'...',
            "pad" : b64'',
        }
    ]
}

// PNG Example //
// with the XMP box excluded //
{
    "alg" : "sha256",
    "boxes": [
        {
            "names" : ["PNGh", "IHDR"],
            "hash" : b64'...',
            "pad" : b64'',
        },
        {
            "names" : ["C2PA"],
            "hash" : b64'AA==',
            "pad" : b64'',
        },
        {
            "names" : ["sBIT"],
            "hash" : b64'...',
            "pad" : b64'',
        },
        {
            "names" : ["iTXt"],
            "hash" : b64'...',
            "excluded": true,
            "pad" : b64'',
        },
        {
            "names" : ["IDAT", "IEND"],
            "hash" : b64'...',
            "pad" : b64'',
        }
    ]
}
```

```
// GIF Example //
{
    "alg" : "sha256",
    "boxes": [
        {
            "names" : ["GIF89a", "LSD"]
            "hash" : b64'...',
            "pad" : b64'',
        },
        {
            "names" : ["2C", "TBID", "2C", "TBID"],
            "hash" : b64'...',
            "pad" : b64'',
        },
        {
            "names" : ["21FE"],
            "hash" : b64'...',
            "pad" : b64'',
        },
        {
            "names" : ["21F9"],
            "hash" : b64'...',
            "pad" : b64'',
        },
        {
            "names" : ["3B"],
            "hash" : b64'...',
            "pad" : b64'',
        },
    ]
}

// TTF Example //
{
    "alg" : "sha256",
    "boxes": [
        {
            "names" : ["C2PA"],
            "hash" : b64'AA==',
            "pad" : b64'',
        },
        {
            "names" : ["PCLT"],
            "hash" : b64'...',
            "pad" : b64'',
        },
        {
            "names" : ["cmap"],
            "hash" : b64'...',
            "pad" : b64'',
        },
        {
```

```
            "names" : ["cvt"],
            "hash" : b64'...',
            "pad" : b64'',
        },
        {
            "names" : ["fpgm"],
            "hash" : b64'...',
            "pad" : b64'',
        },
        {
            "names" : ["gasp"],
            "hash" : b64'...',
            "pad" : b64'',
        },
        {
            "names" : ["glyf"],
            "hash" : b64'...',
            "pad" : b64'',
        },
        {
            "names" : ["head"],
            "hash" : b64'...',
            "pad" : b64'',
        },
        {
            "names" : ["hhea"],
            "hash" : b64'...',
            "pad" : b64'',
        },
        {
            "names" : ["hmtx"],
            "hash" : b64'...',
            "pad" : b64'',
        },
        {
            "names" : ["loca"],
            "hash" : b64'...',
            "pad" : b64'',
        },
        {
            "names" : ["maxp"],
            "hash" : b64'...',
            "pad" : b64'',
        },
        {
            "names" : ["name"],
            "hash" : b64'...',
            "pad" : b64'',
        },
        {
            "names" : ["post"],
            "hash" : b64'...',
            "pad" : b64'',
```

```
        },
        {
            "names" : ["prep"],
            "hash" : b64'...',
            "pad" : b64'',
        }
    ]
}

// JPEG-XL Example //
// showing extra jumb box not containing "C2PA"
{
    "alg" : "sha256",
    "boxes": [
        {
            "names" : ["JXL", "ftyp", "jumb"],
            "hash" : b64'...',
            "pad" : b64'',
        },
        {
            "names" : ["C2PA"],
            "hash" : b64'AA==',
            "pad" : b64'',
        },
        {
            "names" : ["jxlp"],
            "hash" : b64'...',
            "pad" : b64'',
        },
        {
            "names" : ["brob", "brob"],
            "hash" : b64'...',
            "excluded": true,
            "pad" : b64'',
        },
        {
            "names" : ["jxlp"],
            "hash" : b64'...',
            "pad" : b64'',
        }
    ]
}
```

Normally, the `start` and `length` values of an `exclusion` shall be written in their preferred serialization (i.e., "as short as possible"). However, when a box hash assertion needs to be created but the `start` and `length` values are not yet known, they shall be created "as large as possible", which would be as a 32-bit integer.

16.3.4. Collection Data Hash

16.3.4.1. Description

In workflows where it is known in advance that the C2PA Manifest will refer to a collection of assets, instead of a single asset, the collection data hash assertion shall be used as the method to specify the hard bindings (i.e., cryptographic hashes) for the assets in the collection.

> NOTE
>
> It is possible to describe each folder of the training data set of an AI/ML model by having each folder be a separate ingredient of the complete training data set's manifest.

A collection data hash assertion shall have a label of `c2pa.hash.collection.data`.

A collection data hash assertion shall not appear in a cloud data assertion.

16.3.4.2. Schema and Example

The schema for this type is defined by the `collection-data-hash-map` rule in the following CDDL Definition (see Section G.4.5, "Collection Data Hash"Section F.15.5, "Collection Data Hash"

```
; An array of URIs and their associated hashes
$collection-data-hash-map /= {
  "uris": [1* $uri-hashed-data-map],

  ; A string identifying the cryptographic hash algorithm used to compute the
  ; hash on each entry of the `uris` array, taken from the C2PA hash algorithm
  ; identifier list.
  "alg": tstr .size (1..max-tstr-length),
  ? "zip_central_directory_hash" : bstr,
}

; URN-based record identifiers for container formats whose members are named by
; URN rather than by path (e.g., WARC-Record-ID in WARC files), per RFC 8141.
relative-url-type  /= tstr .regexp "urn:[a-zA-Z0-9][a-zA-Z0-9-]{0,30}[a-zA-Z0-9]:[a-zA-Z0-
9()+,\\-./:=@;$_!*'&~%#]+"
```

An example in CBOR diagnostic notation (RFC 8949, clause 8) is shown below:

```
// example of a list of remote URLs //
{
    "alg" : "sha256",
    "uris": [
        {
            "uri": "photos/id/870.jpg"
```

```
            "hash": b64'+ddHMTUUEpuSF6dNaHFa9uFc1sSnY+O3l3MMPFvX5Ws=',
            "dc:format": "image/jpeg"
        },
        {
            "url": "deepmind/bigbigan-resnet50/1",
            "hash" : b64'...',
            "dc:format": "application/octet-stream",
            "data_types": [
                {
                    "type": "c2pa.types.generator",
                },
                {
                    "type": "c2pa.types.model.tensorflow",
                    "version":  "1.0.0",
                },
                {
                    "type": "c2pa.types.tensorflow.hubmodule",
                    "version":  "1.0.0",
                }
            ]
        }
    ]
}

// example of a list of (relative) file URIs //
{
    "alg" : "sha256",
    "uris": [
        {
            "uri": "image1.png",
            "hash": b64'U9Gyz05tmpftkoEYP6XYNsMnUbnS/KcktAg2vv7n1n8='
        },
        {
            "uri": "document.pdf",
            "hash": b64'G5hfJwYeWTlflxOhmfCO9xDAK52aKQ+YbKNhRZeq92c='
        },
    ]
}

// example of a list of relative paths inside an EPUB (which is a ZIP) //
{
    "alg" : "sha256",
    "uris": [
        {
            "uri": "mimetype"
            "hash": b64'+ZXhhbXBsZSBvZiBhIGxpc3Qgb2YgcmVsYXRpdmUgc8=',
            "dc:format": "text/text"
        },
        {
            "uri": "META-INF/container.xml"
            "hash": b64'+ddHMTUUEpuSF6dNaHFa9uFc1sSnY+O3l3MMPFvX5Ws=',
            "dc:format": "text/xml"
        },
```

```
        {
            "uri": "cover_page.svg",
            "hash": b64'U9Gyz05tmpftkoEYP6XYNsMnUbnS/KcktAg2vv7n1n8='
        },
        {
            "uri": "chapter1.html",
            "hash": b64'G5hfJwYeWTlflxOhmfCO9xDAK52aKQ+YbKNhRZeq92c='
        },
    ]
}
```

16.3.4.3. Fields

The `uris` field consists of an array of `uri-hashed-data-map` values that represents a collection of assets. The `alg` field, if present, shall be a string that is one of the allowed hash algorithms as defined in Section 12.1, "Hashing", and that algorithm shall be used to compute the hash of each member of the collection. If the `alg` field is not present, then the hashing algorithm shall be determined by the value of the `alg` field in the Claim.

For each `uri-hashed-data-map`, the `uri` field shall be present. Except in the container formats described below, whose members are identified by an absolute URI native to the container format rather than by path, it shall be a valid relative URI. Such relative URIs shall be considered as relative to the location of the manifest, regardless of whether that is local, in a container (e.g., ZIP) or in the cloud. As a relative URI can contain navigation elements (e.g., `../`), it is possible to refer to content items that are not in the same folder as the manifest - which would be a security issue. A claim generator shall validate or sanitize the URIs before use, ensuring that neither `.` nor `..` appear as part of the URI.

The `uri` field shall not contain a query component. A fragment component is permitted. The `uri` names a static content item whose bytes are hashed, not a dynamic retrieval, so a query component would allow the same hashed member to be addressed by more than one value.

For container formats whose members are identified by an absolute URI native to the container format rather than by path (e.g., the `WARC-Record-ID` of a record in a WARC file, which ISO 28500:2017 requires to be a globally unique absolute URI but does not require to be a URN, see Section A.10, "Embedding manifests into WARC files"), the `uri` field shall be that identifier.

The `hash` field is a byte string representing of the valid hash value for the content item, as determined by the `alg` field. The hash shall be over all bytes (from 0 to n) of the content item - no exceptions.

The rest of the fields are identical to those of an ingredient assertion.

16.3.4.4. Hashing the members of the collection

Each file in the collection shall be hashed individually using the specific hash algorithm defined in the `alg` field. The resultant hash value shall be stored in the `hash` field of the `uri-hashed-data-map` associated with the `uri` to the file.

Not all files in a given hierarchy are required to be included in a hashed collection.

> NOTE
>
> While this is useful in cases where there are files present that aren't necessary to hash, it also provides an opening for an adversary to add files without invalidating the binding.

16.3.5. Multi-Asset Hash

16.3.5.1. Description

There exist a number of file formats that are composed of multiple parts, where each part is itself a valid file format, such as when multiple individual images are aggregated into a single file. Some examples include:

- CIPA Multi-Picture Format (MPF)
- Android Ultra HDR format (which uses MPF)
- ISO 21496 HDR (which uses MPF)
- Android Motion Photo format (which doesn't use MPF, but can exist alongside MPF in the same file)

In some cases, it may be desirable or even required to verify the integrity of each individual part of the file, rather than just the file as a whole. Accordingly, the current set of hard binding assertions are not sufficient to separately verify the integrity of each part. Additionally, the individual parts may have their own C2PA Manifests that need to be recorded. The multi-asset hash assertion is used to provide this functionality.

One additional unique case is where an individual part is optional - meaning that it is possible that it can/will be removed as part of a workflow that does not involve a trusted signer - but the ability to verify the integrity of the rest of the file is still desired.

16.3.5.2. Details

A multi-asset hash assertion shall have a label of `c2pa.hash.multi-asset`. Although it contains hashes and modifies the handling of the hard binding, it is not considered a hard binding.

A multi-asset hash assertion shall not appear in a cloud data assertion nor an external reference assertion.

A multi-asset hash assertion should not be used with a compressed manifest.

> NOTE
> It is not clear if there exists a technical incompatibility between the two, so it is recommended to avoid using them together until further evaluation is complete.

Each part, including the primary part, shall be represented as a `part-hash-map` object within the `parts` array. The `location` field shall contain a `locator` object that describes the location of the part within the file. The `locator` object shall contain either a `bmffBox` field or `byteOffset` and `length` fields. The `byteOffset` field shall contain the byte offset (from the physical start of the file) of the part within the file, and the `length` shall contain the length of the part in bytes. The `bmffBox` field shall contain the BMFF box of the part, when the part is contained with the primary part but as a specific BMFF box (e.g., `mpvd` as used by Motion Photo). For a part described by a `bmffBox` field, the content of the part shall be considered the payload of that box only, excluding the box header.

The parts within the `parts` array shall be listed in the order in which they appear in the file, and the parts shall be contiguous, non-overlapping, and cover every byte of the asset.

> NOTE
> Appearance in the file is defined as their sequential order as they would be located if starting from byte 0 and scanning through to the last byte of the file.

The `hashAssertion` field shall contain a hashed URI to the hash assertion for the part. A part's hash assertion shall be a standard hard binding assertion (e.g., `c2pa.hash.data`), but the label shall have the string `.part` and any multiple instance identifier appended. For example, `c2pa.hash.data.part__2`. Any uses of absolute byte offsets in a part's hash assertion (e.g., the `start` value in a data hash exclusion range) shall be computed relative to the beginning of that part.

> NOTE
> Adding these label suffixes makes it clear that hard binding assertions for parts are not considered standard hard binding assertions and thus there can exist multiple instances of them within a C2PA Manifest.

The `optional` field shall be a boolean indicating if the presence of the part is optional - the default is `false` if not present.

If a part has its own C2PA Manifest, which is not self-contained within that part (e.g., individual frames in a multi-frame asset), then it is recommended to store that C2PA Manifest into the asset's Manifest Store and create a `componentOf` ingredient to reference it.

```
      "hashAssertion": "self#jumbf=c2pa.assertions/c2pa.hash.data.part"
    }
  ]
}

// c2pa.hash.boxes.part - box hash for the first part of the asset //
{
  "alg" : "sha256",
  "boxes": [
    {
      "names" : ["SOI", "APP0", "APP2"],
      "hash" : b64'...',
      "pad" : b64'',
    },
    {
      "names" : ["C2PA"],
      "hash" : b64'AA==',
      "pad" : b64'',
    },
    {
      "names" : ["DQT", "SOF0", "DHT", "SOS", "RST0", "RST1", "EOI"],
      "hash" : b64'...',
      "pad" : b64'',
    }
  ]
}

// c2pa.hash.data.part - data hash for the second part of the asset //
{
  "alg" : "sha256",
  "pad" : '0000',
  "hash" : b64'...',
}

// c2pa.hash.boxes - overall asset hash, covering the whole two-part asset //
{
  "alg" : "sha256",
  "boxes": [
    {
      "names" : ["SOI", "APP0", "APP2"],
      "hash" : b64'...',
      "pad" : b64''
    },
    {
      "names" : ["C2PA"],
      "hash" : b64'AA==',
      "pad" : b64''
    },
    {
      "names" : ["DQT", "SOF0", "DHT", "SOS", "RST0", "RST1", "EOI"],
      "hash" : b64'...',
      "pad" : b64''
    },
```

```
    {
      "names" : ["c2pa.after"],
      "hash" : b64'...',
      "pad" : b64''
    }
  ]
}
```

Such a sample multi-asset hash assertion might be included in an image, as shown below.

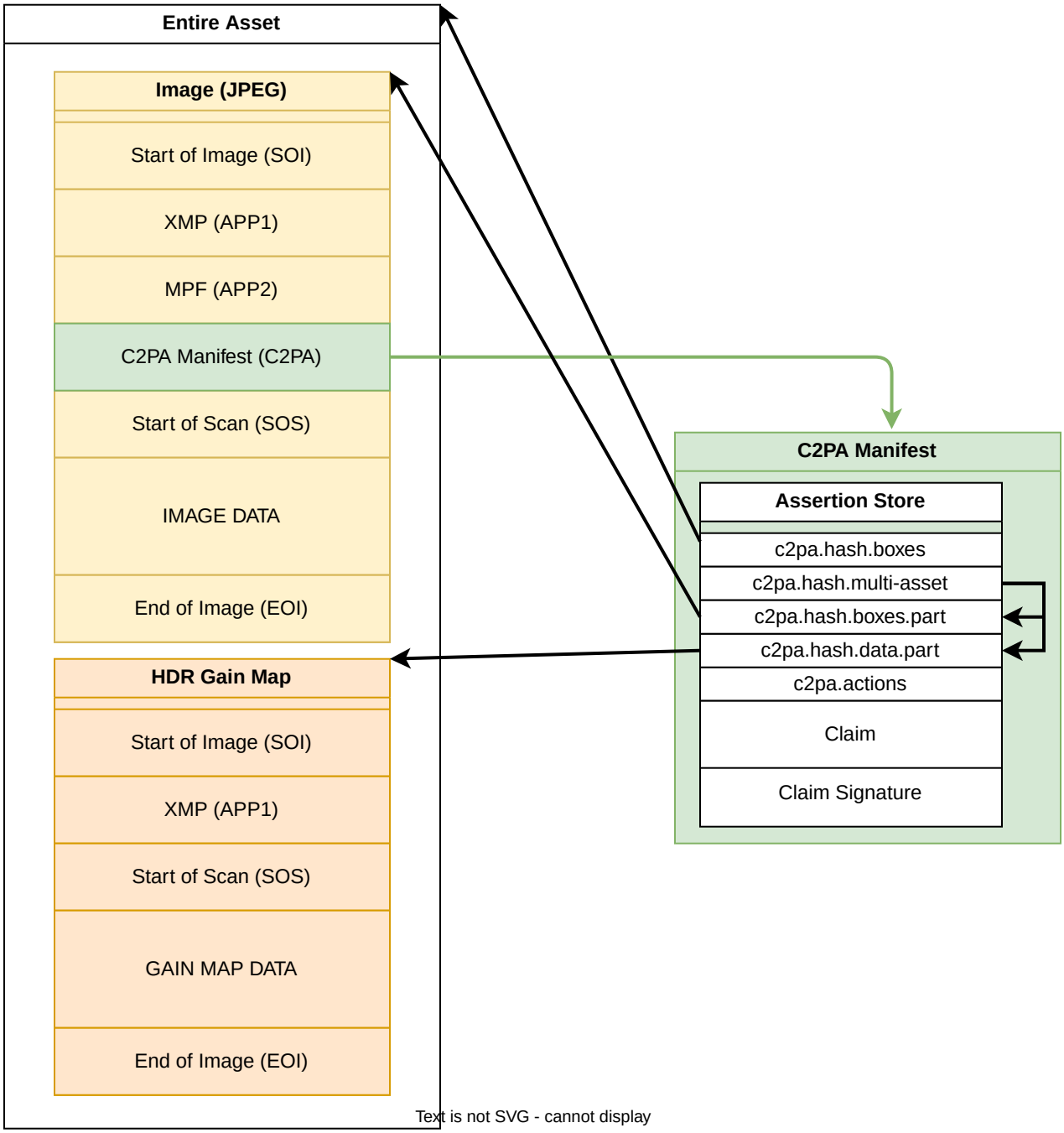


Figure 17. Example of a multi-asset hash assertion used for an HDR gain map

16.3.6. Segment Hash

16.3.6.1. Description

Files that are written only by appending complete lines to the end of the file, such as application logs, event logs and JSON Lines streams, grow continuously and cannot be re-signed as a whole each time content is added. For these assets a single hard binding over the complete file would be invalidated by the next line written.

A segment hash assertion binds a manifest to one bounded segment of such a file — the bytes added since the previous checkpoint — rather than to the file as a whole. Each segment hash assertion also records the position and length of the sentinel line that carries its own manifest, and a digest of the sentinel line of the immediately preceding checkpoint, so that the history of the file cannot be rewritten without invalidating the chain. The way in which checkpoints are written into an asset is defined in Section A.3.17, “Embedding manifests into append-only text files”.

16.3.6.2. Details

A segment hash assertion shall have a label of `c2pa.hash.segment`. It is a hard binding, and shall be used only for the assets described in Section A.3.17, “Embedding manifests into append-only text files”.

The `version` field shall be present and shall have a value of `1` for this version of this specification. An assertion that declares a version a validator does not recognise will be rejected during validation as described in Section 14.12.6, “Validating a segment hash”.

The `alg` field, if present, shall be a string that is one of the allowed hash algorithms as defined in Section 12.1, “Hashing”, and that algorithm shall be used to compute both the `hash` and the `predecessorSentinel.hash` values. If the `alg` field is not present, then the hashing algorithm shall be determined by the value of the `alg` field in the Claim.

The `segmentStart` field shall contain the byte offset of the first byte of the segment, and the `segmentEnd` field shall contain the byte offset of the first byte of the sentinel line, which is the exclusive end of the segment. For the first checkpoint in a file, `segmentStart` shall be 0. The `hash` field shall contain the hash of the bytes `[segmentStart, segmentEnd)`.

The `sentinelLength` field shall contain the byte length of the sentinel line that carries this manifest, including its terminating line ending. It shall be present when the sentinel line references the C2PA Manifest Store by an external URI, and shall not be present when the C2PA Manifest Store is embedded inline in the sentinel line.

```
  ; Byte length of the sentinel line, including its terminating line
  ; ending (LF or CRLF). Present when the sentinel references an external
  ; URI; absent when the manifest is embedded inline in the sentinel.
  ? "sentinelLength": uint,

  ; Digest of the immediately preceding sentinel line's bytes.
  ; Required for every checkpoint after the first; absent for the first.
  ? "predecessorSentinel": predecessor-sentinel-map,

  ? "metadata": $assertion-metadata-map,
}

; Locates and authenticates the immediately preceding sentinel line.
; All three fields are required together; none may appear without the others.
predecessor-sentinel-map = {
  ; Byte offset of the first byte of the preceding sentinel line.
  "start": uint,

  ; Byte length of the preceding sentinel line including its line ending.
  "length": uint,

  ; Hash of bytes [start, start + length), using the alg of the enclosing
  ; segment-hash-map.
  "hash": bstr,
}
```

An example in CBOR diagnostic notation (RFC 8949, clause 8) is shown below. The byte offsets and hash values are those of the second checkpoint in Section A.3.17.6, “Test vectors”:

```
{
  "version": 1,
  "alg": "sha256",
  "hash": b64'axfDsyQ12xTN4nWbwIPzKx/rxSLSC1UQhasygsqD4Ls=',
  "segmentStart": 127,
  "segmentEnd": 143,
  "sentinelLength": 113,
  "predecessorSentinel": {
    "start": 14,
    "length": 113,
    "hash": b64'52VROEjEl9C7I15jXoXVjuO9LzZvsBVf04bXeubHuM8='
  }
}
```

16.3.7. Soft Binding

16.3.7.1. Description

If a claim generator will be providing a soft binding for the asset’s content, it shall be described using a soft

```
  ? "alg-params": bstr,

  ; (optional) Additional metadata of the soft binding. Useful for
  ; binding-specific information
  ? "bindingMetadata": soft-binding-metadata-map,
  ? "url": uri, ; Unused and deprecated.

  ; additional information about the assertion
  ? "metadata": $assertion-metadata-map,
}

soft-binding-block-map = {
  "scope": soft-binding-scope-map,

  ; CBOR byte string describing, in algorithm specific format, the value of the
  ; soft binding computed over this block of digital content"
  "value": bstr,
}

soft-binding-scope-map = {
  ; deprecated, CBOR byte string describing, in algorithm specific format, the
  ; part of the digital content over which the soft binding value has been
  ; computed"
  ? "extent": bstr,
  ? "timespan":soft-binding-timespan-map,
  ? "region": region-map, ; CBOR object defined in regions-of-interest.cddl
}

soft-binding-timespan-map = {
  ; Start of the time range (as milliseconds from media start) over which the
  ; soft binding value has been computed.
  "start": uint,

  ; End of the time range (as milliseconds from media start) over which the soft
  ; binding value has been computed.
  "end": uint,
}

soft-binding-metadata-map = {
  ; Additional description of the implementation or author of the binding or the
  ; algorithm
  ? "description": tstr .size (1..max-tstr-length)

  ; Contact information for the implementation or author of the binding or the
  ; algorithm,
  ? "contact": tstr .size (1..max-tstr-length)

  ; A web page containing more details about the implementation or author of the
  ; binding or the algorithm.
  ? "informationalUrl": tstr .size (1..max-tstr-length)
* tstr => any
}
```

sentence. These IANA top-level types and subtypes are listed at https://www.iana.org/assignments/media-types/media-types.xhtml

Additional information shall accompany each entry in the soft binding algorithm list, within the `entryMetadata` field. These are a human readable description of the algorithm (`description`), and the date it was proposed for entry into the soft binding algorithm list (`dateEntered`).

The contact details of the owner of the entry shall be provided as an email address (`contact`, required). An informational URL (`informationalUrl`, required) shall be provided that references a human readable page describing characteristics of the soft binding algorithm. The information at that page is unconstrained but might include details such as how to interpret the `value` field in the soft binding registry, which is encoded in an algorithm specific form.

16.3.7.5. Soft Binding Resolution API

The soft binding resolution API is a Web API providing a standard way of retrieving C2PA Manifest stores from a soft binding resolution API endpoint given a soft binding value, a manifest identifier, or an asset. The soft binding algorithm list entry may contain a list of URIs of soft binding resolution APIs in the `softBindingResolutionApis` field. If several URIs are given then any may be used for a soft binding resolution.

The API specification and documentation is available here.

16.3.7.5.1. Validating Soft Binding Matches

A common use for soft bindings is to discover the active manifest, from a manifest repository, for an asset whose C2PA Manifest is absent or invalid.

Discovery of the C2PA Manifest shall be performed using one, or a combination of, algorithms identified by the `alg` field within the C2PA Soft Binding Algorithm List. The list is maintained as a JSON document by the C2PA at the following location: https://spec.c2pa.org/softbinding-alg-list

If a C2PA Manifest is found in a manifest repository, and that manifest contains one or more soft binding assertions, then the matcher shall ensure that all soft binding assertions in the located manifest match the soft bindings used to perform the discovery.

A soft binding assertion shall be considered a match if both the algorithm identifier (`alg`) and the value (`value`) described within the assertion match the algorithm identifier (`alg`) and value (`value`) used to perform the match. Matching is performed in the manner prescribed by the specified algorithm.

```
    }
    ]
  }
}
```

16.5.7. Description field

An ingredient may include a free-text description, in the `description` field, of what an the ingredient is or is used for. If present, the content of this field should be produced by the claim generator, and should not contain user-provided information.

> NOTE
> A user-provided ingredient description can be specified using a custom assertion.

16.5.8. Ingredient Data

16.5.8.1. Standard Usage

In certain use cases, such as Generative AI, it may be important to have ingredients where the data of the ingredient is provided - either embedded into the C2PA Manifest or via a URL that references the data. This is accomplished through the `data` field in the ingredient, which uses a `hashed-uri` to point to an embedded data assertion or a `hashed-ext-uri-map` to point to an external reference.

> NOTE
> Previous versions of this specification allowed the `hashed_uri` to point to a data box.

Using an embedded data assertion implies that its content will be embedded in this C2PA Manifest and in any future C2PA Manifest (unless redacted) that contains this asset as an ingredient. Claim generators should take the size of this field into consideration when choosing whether to embed data.

Example 9. Example of ingredients with data

An example of some ingredients with data, in CBOR diagnostic notation (RFC 8949, clause 8):

```
// ingredient (prompt) //
{
  "dc:format": "text/plain",
  "relationship": "inputTo",
  "dataTypes": [{
    "type": "c2pa.types.generator.prompt",
  }],
  "data": {
```

assertion and referenced herein via a hashed-uri reference. This assertion shall also be added to the `relatedAssertions` field.

Manifest Consumers should also support the data box approach recommended by earlier versions of this specification.

16.5.11. Related Assertions

In cases where a claim generator has an ingredient that does not have an associated C2PA Manifest, but there are other assertions in the active manifest that should be understood as being about that ingredient rather than about the current asset, the claim generator may reference those assertions using the `relatedAssertions` field of the ingredient assertion. The value of this field shall be an array of one or more hashed JUMBF URIs, each referencing an assertion, in the same C2PA Manifest, that is neither an ingredient nor an action assertion.

An ingredient assertion shall not contain both an `activeManifest` and a `relatedAssertions` key: when an ingredient has its own active manifest, any assertions about that ingredient belong in that manifest instead.

Each assertion referenced by `relatedAssertions` shall have a label ending in `.ingredient`, following the same naming convention used for ingredient thumbnails (see Section 8.2, “Labels”). For example, a `c2pa.ai-disclosure` assertion that is about an ingredient, rather than about the current asset, would have the label `c2pa.ai-disclosure.ingredient`. This also applies to entity-specific namespaces, such as `com.litware.metadata.ingredient` would be valid. This label component shall follow the usual rules for assertion labels with respect to multiple instances (e.g., `c2pa.ai-disclosure.ingredient__1` or `com.litware.metadata.ingredient__1` for a second such instance).

NOTE	One use case is when the ingredient comes from a tool that is not C2PA-compliant but the claim generator needs to assert some information about that ingredient, such as that found in a `c2pa.ai-disclosure` assertion.

NOTE	When the multiple instance naming occurs, it is not expected that the numbering for assertions match the numbering for a given assertion, such as `c2pa.thumbnail.ingredient__4` could be for `c2pa.ingredient__1`.

Example 11. Example of ingredient with a related assertion

An example of an ingredient with a `relatedAssertions` field referencing a `c2pa.ai-disclosure.ingredient` assertion, in CBOR diagnostic notation (RFC 8949, clause 8):

```
// ingredient v3 (inputTo, no activeManifest, with a relatedAssertions entry) //




                    "identifier": "track_id",
                    "value": "3"
    {           }
      "url":}"self#jumbf=c2pa.assertions/c2pa.ai-disclosure.ingredient",
      "a]g": "sha256",
      "hash": b64'UjRAYWiAq4lfCRDmksWAlDJN/XtHHFFwMWymsZsm3j8='
  } }
  "dc:format": "video/mp4",
```

16.5.12. Existing manifests

If the ingredient has an existing C2PA Manifest Store, then all C2PA Manifests in the ingredient's C2PA Manifest Store that have undergone validation, and that do not already exist in the asset's C2PA Manifest Store, shall be copied by the claim generator into the asset's C2PA Manifest Store, except as outlined in Section 16.5.13, "Copying existing manifests" or when directed not to do so (for example via user input or via configuration).

The claim generator should also copy into the asset's C2PA Manifest Store any additional C2PA Manifests that were not validated, as well as any additional JUMBF boxes and superboxes appearing in the C2PA Manifest Store that are not recognized as C2PA Manifests.

> NOTE
>
> Copying these additional elements supports compatibility with custom assertions and future constructs that may reference elements of the C2PA Manifest Store in ways that the claim generator does not recognize.

16.5.13. Copying existing manifests

16.5.13.1. Determining the need

To determine whether or not an existing manifest from the ingredient's C2PA Manifest Store needs to be copied into the asset's C2PA Manifest Store, the claim generator shall:

1. Validate the ingredient per the process described in Section 16.5.13.4, "Ingredient validation". In case of validation failures, the claim generator may skip the rest of these steps if directed to do so (for example, via user input or via configuration).
2. For each manifest in the ingredient's C2PA Manifest Store, compare its URN identifier with the URN identifiers of each C2PA Manifest already present in the asset's C2PA Manifest Store.
    a. If a match is found, compute and compare the hash of the manifest box from ingredient's C2PA Manifest Store to the hash of the matching manifest box from the asset's C2PA Manifest Store
        i. If the hashes match, then the claim generator shall not copy the manifest from the ingredient's C2PA Manifest Store to the asset's C2PA Manifest Store.
        ii. If the hashes do not match:
            A. The claim generator shall check if any assertions from either manifest were redacted (optionally utilizing the list of redactions compiled in the Performing explicit validation process).
                I. If the validator is able to determine that the hashes differ only due to redaction, then:
                    1. If all redactions were applied against the manifest already present in the asset's C2PA Manifest Store, then the claim generator shall not copy the manifest from the ingredient's C2PA Manifest Store into the asset's C2PA Manifest Store.
                    2. If all redactions were applied against the manifest from the ingredient's Manifest Store, then the claim generator shall replace the manifest in the asset's C2PA Manifest Store with the manifest from the ingredient's C2PA Manifest Store.
                    3. If different redactions were applied against both the C2PA Manifest from the ingredient's C2PA Manifest Store and the asset's C2PA Manifest Store, then the claim generator shall redact as many assertions as needed from the existing manifest in the asset's C2PA Manifest Store to result in a union of the two sets of redactions.
                II. In all other cases, then the claim generator shall copy the manifest from the ingredient's C2PA Manifest Store, re-label it with an updated URN per the process described in Unique Identifiers, and insert the re-labeled version into the asset's C2PA Manifest Store.

NOTE The process for determining whether the hashes differ only due to redaction is left up to the

validator.

16.5.13.2. Examples

EXAMPLE: Consider a claim generator D that is importing ingredients. It begins by importing ingredient B, which itself has an ingredient Manifest A. After validating both manifests, claim generator D copies Manifests B and A into Asset D 's C2PA Manifest Store. Then it imports ingredient C, which also includes a redacted version of Manifest A. After validating Manifest C and the redacted Manifest A, it compares the hashes of both versions of Manifest A. Knowing that the version of Manifest A in ingredient C was redacted, claim generator D over-writes the version of Manifest A already present in asset D 's C2PA Manifest Store with the redacted version of Manifest A from ingredient C.

EXAMPLE: Consider the same scenario as above, except that the version of Manifest A in ingredient C failed validation because one of its assertions failed a hash comparison. In this situation, Claim generator D copies Manifest A from ingredient C, re-labels it with a new URN, and places the re-labeled copy in asset D 's C2PA Manifest Store.

NOTE

A C2PA Manifest Store can contain JUMBF boxes or superboxes that are not C2PA Manifests. They need not be copied as part of this process.

16.5.13.3. Adding manifest references to the ingredient assertion

If the active manifest of the ingredient has been copied into the asset's C2PA Manifest Store, then a URI reference to the ingredient's active C2PA Manifest box shall be stored as the value of the `activeManifest` field in the ingredient assertion, and an additional URI reference to the active Manifest's C2PA Claim Signature box shall be stored as the value of `claimSignature`.

For a C2PA Manifest present in the C2PA Manifest Store, `hashed_uri`s shall be used as the values for both of the ingredient assertion's `activeManifest` and `claimSignature` fields.

NOTE

Providing both values enables efficient ingredient validation and also supports validation if one of the ingredient's assertions were redacted.

In the case where an ingredient has an associated active manifest, it can be used to identify the

Example 13. Example of an ingredient assertion whose active manifest could not be established

An example of an ingredient assertion recording that its remote manifest was inaccessible, in CBOR diagnostic notation (RFC 8949, clause 8):

```
// ingredient v3 (parentOf, remote active manifest could not be retrieved) //
{
  "dc:format": "image/jpeg",
  "relationship": "parentOf",
  "validationResults": {
    "activeManifest": {
      "success": [],
      "informational": [],
      "failure": [{
        "code": "manifest.inaccessible"
      }]
    }
  }
}
```

Each validation result (as described using a `status-codes-map`), consists of an array of `success`, `informational`, and `failure` codes. Each code is represented as a `status-map` object which shall contain a `code` field with the status code. In addition, it may contain a `url` field with a JUMBF URI reference to the specific element in the manifest about which the status refers, and an optional `explanation` field with a human-readable explanation of the status. Status codes are defined in Appendix D, *Standard Validation Status Codes*.

Custom status codes are permitted when a claim generator has a need to record some process-specific status information. The code shall conform to the same syntax as entity-specific namespaces (e.g. `com.litware.malformedFrobber`).

Additionally, a `validation-results-map` may contain a SemVer formatted `specVersion` field which represents the version of this specification that was used as the basis for the validation procedure. It may also contain either a `trustListUri` field (DEPRECATED), which is a single URI or a `trustListUris` field, which is an array of objects. In either case, they identify the trust list(s) that were used to validate the claim signature.

EXAMPLE: For C2PA 2.5, the value of `specVersion` would be `2.5.0`.

If present, the `trustListUris` field shall contain one or more objects, each of which identifies a trust list on which a trust anchor match occurred during the validation of the claim signature or a TSA time-stamp. Each `trust-list-uri-map` object in the `trustListUris` array shall contain a `uri` field, which

identifies the trust list, and a `type` field, which identifies the type of trust list.

> **NOTE** While other trust lists may have been used in the validation process, only those trust lists that contained the matching trust anchor(s) are included in the `trustListUris` field.

When it is necessary to identify that the C2PA Trust List was used, then the `uri` for the C2PA Trust List is `https://c2pa.org/trustlist`. Similarly, identifying that the C2PA TSA Trust List was used, then the `uri` for the C2PA TSA Trust List is `https://c2pa.org/trustlist-tsa`.

> **NOTE** A `uri` value in a `trustListUris` object is a URI and not a URL, meaning that the elements of the array need not resolve to an actual machine-readable list, but instead they are just unique identifiers for a specific trust list. If an official URI is not known, a descriptive unique value in the form of a URI can be used.

The `type` field shall conform to the same syntax as entity-specific namespaces (e.g. `c2pa.trustList` or `com.litware.trustList`). Two values for the `type` field are defined in this specification: `c2pa.trustList` and `c2pa.tsaTrustList`. The `c2pa.trustList` type is used for trust lists that contain claim-signing credentials, while the `c2pa.tsaTrustList` type is used for trust lists that contain TSA credentials. Other types may be defined by other specifications or by individual entities.

A `trust-list-uri-map` object may also contain an optional `identifier` field, which is a unique identifier for a specific instance of the trust list. For trust lists formatted per ETSI TS 119 602 or ETSI TS 119 612 (including the C2PA Trust List and C2PA TSA Trust List), if the `identifier` field is present, it shall contain the decimal string representation of the sequence number (`ListSequenceNumber`, e.g. `"1"`, `"42"`) from the trust list's `ListAndSchemeInformation` header. For all other trust lists, this may be a version number, a commit hash, a release date, or another unique identifier defined by the maintaining authority.

An example `validation-results-map` referencing the C2PA Trust List and C2PA TSA Trust List with illustrative sequence numbers is shown in Validation results with trust list references in CBOR diagnostic notation (RFC 8949, clause 8).

Validation results with trust list references in CBOR diagnostic notation (RFC 8949, clause 8)

```
{
  "specVersion": "2.5.0",
  "trustListUris": [
    {
      "uri": "https://c2pa.org/trustlist",
      "type": "c2pa.trustList",
      "identifier": "42"
    },
```

```
    {
      "uri": "https://c2pa.org/trustlist-tsa",
      "type": "c2pa.tsaTrustList",
      "identifier": "17"
    }
  ]
}
```

16.5.14. Ingredient Metadata

As described in assertion metadata, the `metadata` field of the ingredient assertion may contain metadata about the ingredient, such as the date and time when it was generated or other data that may help Manifest Consumers to make informed decisions about the provenance or veracity of the assertion data.

One common use for the `metadata` field is when only a portion of an ingredient is used in the creation or editing of an asset. In such cases, the `metadata` field should contain a `regionOfInterest` field (as described in Section 16.9.1.5, “Region of Interest”) which describes the relevant portions of the ingredient that were used. An example of this can be found in Example 14, “Example of ingredient with metadata containing regions”.

NOTE

Although the field contains only a single region of interest, the `region-map` object can specify multiple regions as the values of its `region` field. This would be useful when multiple parts of a single ingredient are involved.

Example 14. Example of ingredient with metadata containing regions

An example of an ingredient containing a region of interest in its metadata, in CBOR diagnostic notation (RFC 8949, clause 8):

```
{
  "metadata": {
    "regionOfInterest" : {
        "description": "10 seconds of audio",
        "region": [
            {
                "type": "temporal",
                "time": {
                    "type": "npt",
                    "start": "10",
                    "end": "20"
                }
            },
            {
```

```
                "type": "identified",
                "item": {
                    "identifier": "track_id",
                    "value": "3"
                }
            }
        ]
    }
  }
  "dc:format": "video/mp4",
  "relationship": "componentOf",
  "activeManifest" : {
      "url": "self#jumbf=/c2pa/urn:c2pa:98782815-5116-4d78-93de-3f5d8b4f4615",
      "hash": b64'TEWww2UCIR/e8mmR0XvzkFVZYTJ59Q8Ip4nkYxrS/Ys='
  },
  "claimSignature" : {
      "url": "self#jumbf=/c2pa/urn:c2pa:98782815-5116-4d78-93de-
3f5d8b4f4615/c2pa.signature",
      "hash": b64'ICJkYzpmb3JtYXQiOiAiaW1hZ2UvanBlZyIsCiAgImR='
  },
  "validationResults": { ... }
}
```

16.5.15. Soft Bindings

An active manifest may include a C2PA Manifest as an ingredient (via a `parentOf` relationship) that was discovered using a soft binding lookup. If the Claim Generator does include such a C2PA Manifest, then it shall include a `softBindingsMatched` field indicating true, and a `softBindingAlgorithmsMatched` field containing an array of strings (of soft binding algorithm names that were used to discover the ingredient C2PA Manifest). The algorithm names shall be listed with the C2PA Soft Binding Algorithm List as identified within the `alg` field of entries in that list.

16.6. Content Description & Enrichment

These assertions carry additional information about an asset or alternative representations of its content, such as thumbnails, metadata, format-specific details, and externally stored data.

16.6.1. Thumbnail

16.6.1.1. Description

A thumbnail assertion provides an approximate visual representation of the asset at a specific event in the lifecycle of an asset. There are currently two specific events:

16.6.2.1. General

This assertion is used to refer to an alternative digital content that is different from the primary asset, and to provide functionality depending on the referenced digital content. It shall have a label of `c2pa.alternative-content-representation`.

This assertion has the parameters depending on the file format specification. Individual specifications are specified in the following sections.

16.6.2.2. Original Preservation Image

16.6.2.2.1. Description

This type is used to reference "Original Preservation Image(OPI)" in Exif. The OPI is introduced for recording authenticity of the image at the time of capture in which subsequent post image processing is not applied, along with metadata that describes image properties. This enables users to recognize the difference between the image at the time of capture and the main image developed through the image processing by camera systems when the main image has noticeable differences from the real scene. This assertion enables applications to reference the OPI through a Manifest as an element of provenance data.

Considering that OPI is normally recorded at the time of capture as an individual image in MPF, this assertion provides both

- 1) the method to refer to OPI stored in the Asset (including validation) at capture time and
- 2) the method to move and record the OPI into the manifest

Moving OPI metadata typically occurs during editing, particularly under the following conditions

- when the file format will be changed from Exif and / or
- when the file is used as an ingredient.

When referencing the OPI in MPF, the location shall be specified in the alternative content representation assertion by the index number of the item from the `parts` array described in the multi-asset hash assertion in the same C2PA Manifest. If the asset is subsequently edited and the OPI is preserved, a new alternative content representation assertion should be included in the new C2PA Manifest, referencing the OPI location in the new multi-asset hash assertion.

> NOTE
>
> An OPI referenced using a multi-asset part index can only be validated when the alternative content representation assertion exists in the active manifest, or in the presence of update

```
generic-representation = {
  "type": tstr .regexp "([\\da-zA-Z_-]+\\.)+[\\da-zA-Z_-]+",
  "parameters": { * tstr => any }

  ; additional information about the assertion
  ? "metadata": $assertion-metadata-map,
}
```

The schema for the `c2pa.alternative-content-representation` assertion is defined by the following abstract rule, which permits the inclusion of either an `original-preservation-image-representation` or a `generic-representation structure`. The `type` field within the alternative content representation assertion shall be set to `exif.originalPreservationImage`. The `parameters` field shall be the `original-preservation-image-params` structure. The `original-preservation-image-params` structure shall use either the `multiAssetPartIndex` field to reference OPI data located within the asset in MPF, or the `embeddedOriginalPreservationImage` field to record the OPI inside the manifest, but shall not use both.

An example in CBOR Diagnostic Format (`.cbordiag`) is shown below:

```
// The case for reference Original Preservation Image(OPI) data located within the asset
in MPF. //
{
  "type": "exif.originalPreservationImage",
  "parameters": {
    "multiAssetPartIndex": 1
  }
}

// The case for recording the OPI inside the manifest. //
{
  "type": "exif.originalPreservationImage",
  "parameters": {
    "embeddedOriginalPreservationImage" : {
       "url" : "self#jumbf=c2pa.assertions/c2pa.embedded-data",
       "hash": b64'Hf0IgeqbL0m+FTTLpUWwsDGR8pvhUR1AlwvaXjQ0qGY='
   }
  }
}
```

16.6.3. Asset Type

16.6.3.1. Description

The asset type assertion provides a way to more completely describe an asset, specifically additional context on how to parse or otherwise process it. This assertion allows for specifying an IANA Media Type value and/or

```
  ; additional information about the assertion
```

6.6.4.2. Schema and Example

The schema for this type is defined by the `font-info-map` rule in the following CDDL Definition (see Section F.16.18, “Font Information” for the complete normative schema):

```
  "fullName": tstr, ; The full name of the font.
  ; A version in the semantic versioning (semver) format.
  ? "version": semver-string, ; The version of the font in SemVer format.

  ; A URL to the release notes associated with this version of the font.
  ? "versionUrl": ext-url-type,

  ; The date this version of the font was released or published.
  ? "releaseDate": tdate,
  "familyName": tstr, ; The Font Family.
  "style": $font-style, ; The style of the font, e.g. italic or regular.
  "weight": font-weight-map, ; The weight of the font with name and value.
  ; The PostScript name, ID 6, from the font 'name' table.

  ; Characters from ASCII 33-126 except the following: [](){}<>/%
  "postScriptName": tstr .regexp "^(?!.*[\\[\\]\\(\\)\\{\\}<>\\/%])[!-~]{1,63}$",
  "format": $font-format-choice, ; The format of this font.
  "copyrightNotice": tstr, ; The copyright associated with this font.

  ; The entity that holds the copyright to the font.
  ? "copyrightHolder": font-entity-map,

  ; The years for which the holder asserts copyright.
  ? "copyrightYears": [1* font-copyright-year-range],

  ; The individuals that designed the font.
  ? "designers": [1* font-designer-map],
  ? "designFoundry": font-entity-map, ; The foundry that designed the font.
  ? "sourceFoundry": font-entity-map, ; The foundry that distributes the font.
  ? "identifier": tstr, ; Internal identier of font for foundry or vendor use.

  ; additional information about the assertion
  ? "metadata": $assertion-metadata-map,
}

; Font Formats
$font-format-choice /= "TrueType"
$font-format-choice /= "OpenType"

; Copyright year range
font-copyright-year-range = 1..9999

; Font weight range
font-weight-range /= 1..1000
```

```
; The font style
$font-style /= "Normal"
$font-style /= "Italic"
$font-style /= "Oblique"

; Font format
$font-format /= "Roman"
$font-format /= "Regular"
$font-format-choice /= "OpenType"

; Data for a font weight
font-weight-map = {
  ; The type name of the weight class, e.g. bold or thin.
  "class": $font-weight-class,
  ; The value of the weight.
  "value": font-weight-range,
}
font-weight-range /= 1..1000

; Data for an entity with a name and credentials
font-entity-map = {
  ; The name of the person or foundry.
  "name": tstr,
  ; A URL for additional information about this person or foundry.
  ? "url": ext-url-type,
}
; Data for a font designer
font-designer-map = {
  ; The descriptive name of the weight class, e.g. bold or thin.
  "person": $font-weight-class; The person who designed the font.
  "value": font-weight-range, ; The value of the weight.
} ; The name of the foundry with which the designer was associated when
  ; contributing to the font design.
; Data for an entity with a name and credentials
font-entity-map = {
  ; A description of what the designer contributed to the font. For example,
  ; 'All the Latin and Arabic characters'.
  ? "contribution": tstr,
  ? "metadata": $assertion-metadata-map,

  ; When the designer started to contribute to the font design.
  ? "startDate": tdate,

  ; When the designer ended contributions to the font design.
  ? "endDate": tdate,
}
```

A basic example in CBOR diagnostic notation (RFC 8949, clause 8), containing only required fields, is shown below:

```
{
  "style": "Italic",
  "weight": {
    "class": "Regular",
    "value": 400
  }
}
```

`minSequenceNumber`

An unsigned integer representing the earliest sequence number that can be signed with this key.

`createdAt`

The timestamp indicating when the session key was created formatted as CBOR tag 0 (RFC 3339 date-time).

`validityPeriod`

An unsigned integer representing the validity period of the session keys in seconds, starting from the `createdAt` timestamp

`signerBinding`

A detached signature of the Signer's end-entity certificate, signed using the (private key) of the session key `key`, enabling verification that this session key used to protect the media is uniquely associated with the Signer. The payload of `Sig_structure` shall be the Signer's end-entity certificate (the first certificate in the x5chain header parameter, as defined in Section 13.5, "X.509 Certificates") encoded as a CBOR byte string.

Session keys are valid from their individual `createdAt` until `createdAt + validityPeriod`, and shall only be used within their validity period. The `minSequenceNumber` field indicates the earliest segment sequence number that can be signed with a given key, supporting key rotation and preventing replay attacks.

When providing an new C2PA Manifest (via a C2PA Manifest Box), new session keys may be included in that C2PA Manifest. The new session keys may have overlapping validity periods with previous session keys, but the `minSequenceNumber` for the new keys shall be greater than or equal to the current sequence number. This ensures a clear transition between keys and supports overlap for seamless key rotation.

16.7.3.3. Schema and Example

The schema for this type is defined by the `session-keys-data` rule in the following CDDL Definition (see Section F.17.2, "Session Keys" for the complete normative schema):

```
; Data structure to supply session keys used for live-media validation
session-keys-data = {
    "keys" => [1* session-key], ; one or more session keys
}

session-key = {
    ; session key used to sign the `verifiable-segment-info-map`, note:
    ; semantics need to require kid in COSE_Key
    "key" => COSE_Key,
```

16.9.1.10. Localization

16.9.1.10.1. General

It is important that consumers of C2PA manifests be able to understand the information in their native language, when possible. To this end, it is possible to add localization information for an assertion with a dictionary that is included in the assertion's metadata.

16.9.1.10.2. Localization Dictionary

A localization dictionary shall consist of a single object, where each of its keys represents the translations using the language indexing technique from JSON-LD. If the value that requires translation is not associated with a top-level key, then "dot notation" (`.`) shall be used to reference keys nested in objects. Array indexing notation (`[n]`, `n>=0`) shall be used where a specific element in an array needs to be traversed. When the value requiring translation is itself an array, a specific element may be referenced. Some examples are shown in Example 17, "Examples of Localization Dictionaries":

Example 17. Examples of Localization Dictionaries

```
{
  "dc:title": {
    "en-US": "Kevin's Five Cats",
    "en-GB": "Lord Kevin's Five Cats",
    "es-MX": "Los Cinco Gatos de Kevin",
    "es-ES": "Los Thinco Gatos de Kevin",
    "fr": "Les Cinq Chats de Kevin",
    "ja": ""#$%&'(&)"
  }
}
```

```
{
  "actions[0].softwareAgent": {
    "en-US": "Joe's Photo Editor",
    "en-GB": "Joe's Photo Editor",
    "es": "Editor de fotos de Joe",
    "fr": "L'éditeur de photos de Joe",
    "ja": "*+,&-./01"
  }
}
```

Any such 3rd party keys or values are required to be namespaced in the same way as Section 8.2.1, "Namespacing", e.g. `com.litware`. In order for a Manifest Consumer to display human-readable information about these keys and values, the claim generator should provide the strings via this localization

approach.

Localized Actions shows its use in localizing custom actions, by using it in the assertion metadata of a `c2pa.actions` assertion.

Localized Actions

```
{
  "com.litware.blur": {
    "en-US": "Blur",
    "fr-FR": "Brouiller",
  },
  "com.litware.filter": {
    "en-US": "Filter",
    "es-ES": "Filtrar",
    "ja-JP": "2$34,"
  }
}
```

16.9.2. Regions of Interest

16.9.2.1. Description

In some use cases, a given assertion, such as an actions assertion, may only be relevant to a specific portion of an asset as opposed to the entire asset. In those cases, it is necessary to have a way to describe that region - whether it be temporal, spatial, textual or a combination of them. A `region` definition serves that purpose.

16.9.2.2. Common

The most important part of the `region` definition is the `range` field which is used to describe a temporal range, a spatial range, a frame range, a textual range or a combination of them, for the region.

> NOTE
>
> While the specification allows for specifying a combination of ranges, it is not defined how a Manifest Consumer will use them. It is expected that the C2PA's User Experience Task Force will take this up in the future.

A `region` may also contain one or more common fields:

`name`

a free-text string representing a human-readable name for the region which might be used in a user interface.

`identifier`

a free-text string representing a machine-readable, unique to this assertion, identifier for the region.

`type`

a value from a controlled vocabulary such as https://cv.iptc.org/newscodes/imageregiontype/ or an entity-specific value (e.g., `com.litware.newType`) that represents the type of thing(s) depicted by a region.

`description`

a free-text string.

Older versions of this specification included a `role` field. This field has been deprecated and shall no longer be included when generating a region of interest.

16.9.2.2.1. Ranges

All ranges consist of a `type` field whose value is either "spatial", "temporal", "frame", "textual" or "identified". In addition, it shall contain one of the following fields whose data is an object consisting of the specific data for that range:

- `shape` (for spatial);
- `time` (for temporal);
- `frame` (for temporal or textual);
- `text` (for textual);
- `item` (for specifically identified items).

16.9.2.2.2. Spatial

Spatial ranges are described using a `shape` object. A `shape` can be used to represent a rectangle, a circle or a polygon. It is modelled on the Region Boundary Structure from the IPTC.

16.9.2.2.3. Temporal

Temporal ranges are described using a `time` object, which represents a range from a starting time to an ending time. Times are normally described using a value that is relative to the start of the media. This is commonly referred to as "Normal Play Time" (`npt`) as described in Section 4.4.2 of RFC 7826 (as recommended in W3C Media Fragments specification). Alternatively, it is possible to refer to an absolute time (e.g., "10:00am"). Absolute times, sometimes referred to as "wall clock time", shall be specified using the

Internet profile of ISO 8601 as described in RFC 3339.

> NOTE
> Absolute, or wall clock time, is useful in scenarios where the media asset represents activity that took place during a specific date and time period, such as a news broadcast or a live event.

If no `type` field is provided, the range shall be in `npt` format. If no `start` field is provided, the range shall start at the beginning of the asset. If no `end` field is provided, the range shall end at the end of the asset.

All `start` times are inclusive of that moment in time, and all `end` times are, by default, exclusive of it. The `endInclusivity` field may be set to `inclusive` to make the `end` time inclusive instead, which should be used where the range runs to the end of the asset.

For `npt` time ranges (expressed via a `npt-time-map`): If neither the `start` nor the `end` fields are provided, the range shall represent the entire asset.

For `wallClock` time ranges (expressed via a `wall-clock-time-map`): The `end` time shall not be earlier than the `start` time. At least one `start` or `end` field shall be provided. If one field is not provided, and the duration of the asset is available, the missing field is calculated by adding or subtracting the duration of the asset to/from the provided field. If the total asset duration is not available (such as for live streams), the time range extends forward or backward from the provided field, as appropriate, until the region is updated or the end of the asset is reached.

Where temporal ranges use fractions of a second, and not a sample or frame count, implementations should use the smallest number of decimal places possible to accurately represent the point in their media timeline. Where the fractions of a second do not align to a sample or frame boundary, and that is important for the validator or its user interface, implementations should choose the nearest sample or frame whose time is less than or equal to the specified time.

16.9.2.2.4. Frames

A `frame` object defines a range using the starting and ending frames or pages (inclusive). If no `start` is provided, the range shall start at the beginning of the asset. If no `end` is provided, the range shall end at the end of the asset. If neither is provided, the range shall represent the entire asset.

Frames are represented as a single ordinal numbers, where `0` is the first frame.

While frames are typically used to represent page numbers of a document, such as PDF, they may have uses in other media types, such as animation, video and audio. It is recommended that where possible, media types

dealing with regions of interest over time use `temporal` ranges instead.

16.9.2.2.5. Textual

A `text` object defines a range using a one or more URL fragment identifiers, as defined by the W3C Web Annotation fragment selector. It may also refine the range using offsets to the starting and ending characters (inclusive). If no `start` is provided, the range shall start at the beginning of the fragment. If no `end` is provided, the range shall end at the end of the fragment. If neither is provided, the range shall represent the entire fragment.

When used singularly, the `fragment` entry of the `text-selector-map` represents the entirety of the specified textual range. However, the `text-selector-range-map` supports a pair of `text-selector-map` objects. The value of `selector` is the start of the range (or its entirely, if no `end` entry is present) and the value of `end` (if present) represents the end of a contiguous range. In addition, multiple pairs may be used to represent a range that is not contiguous.

16.9.2.2.6. Identified

An `item` object defines a media track, media item, or other discrete content item in the asset, allowing the claim generator to indicate assertions that apply to only a subset of the content carried in the asset's file container. For example, it could be used to indicate that only the audio track of a video file is relevant.

The media or content item is identified by an `identifier` string whose value should match the typical item identification naming scheme in that specific container format. For example, the value of `identifier` should be `track_id` for MP4 files, and `item_ID` for HEIF files. The value of the `identifier` is then provided in the `value` field. For example, a `value` of `2` with an `identifier` of `track_id` in an MP4 video file container would indicate an assertion related to the second media track in the file (which could be the audio track).

Another use for identified ranges is to indicate a specific region by a known semantic value. For example, the Foundational Model of Anatomy could be used to identify a specific region of a human body. In such a case, the `identifier` shall be the URL or URI to where to locate the schema (though not necessary directly to a machine readable one).

16.9.2.3. Schema

The schema for this type is defined by the `region-map` rule in the following CDDL Definition (see Section F.16.10, "Regions of Interest" for the complete normative schema):

```
region-map = {
```

```
    ; definition of the range, one or more ranges
    "region":        [1* range-map],

    ; a free-text string representing a human-readable name for the region which
    ; could be used in a user interface.
    ? "name": tstr .size (1..max-tstr-length),

    ; a free-text string representing a machine-readable, unique to this
    ; assertion, identifier for the region.
    ? "identifier": tstr .size (1..max-tstr-length),
    ? "type":     tstr .size (1..max-tstr-length), ; from a controlled list
    ? "role":     $role-choice, ; DEPRECATED

    ; human readable description of the region
    ? "description": tstr .size (1..max-tstr-length),

    ; additional information about the assertion
    ? "metadata": $assertion-metadata-map,
}

$range-choice /= "spatial" ; a range identified by physical area
$range-choice /= "temporal" ; a range identified by a time period
$range-choice /= "frame" ; a range identified by a series of frames or pages
$range-choice /= "textual" ; a range identified by a range of text

; a range identified by a specific identifier and value
$range-choice /= "identified"

range-map = {
    ; either "spatial", "temporal", "frame", "textual" or "identified"
    "type":     $range-choice,
    ? "shape":     shape-map, ; description of the shape of a spatial range

    ; description of the time boundaries of a temporal range
    ? "time":     time-map,

    ; description of the frame boundaries of a temporal range
    ? "frame":     frame-map,
    ? "text":     text-map, ; description of the boundaries of a textual range

    ; description of the boundaries of an identified range
    ? "item":     item-map,
}

coordinate-map = {
    "x": float, ; coordinate along the x-axis
    "y": float, ; coordinate along the y-axis
}

$shape-choice /= "rectangle" ; a rectangular shape
$shape-choice /= "circle" ; a circular shape
$shape-choice /= "polygon" ; a polygonal shape
```

```
; this can be used for either frames of a video or pages of a document
frame-map = {
    ? "start":     int, ; start frame (or beginning of asset if not present).
    ? "end":     int ; end frame (or end of asset if not present).
}

; this is modelled after the W3C Web Annotation selector model
text-selector-map = {
    ; fragment identifier, as per RFC3023 or ISO 32000-2, Annex O
    "fragment":     tstr,

    ; start character offset (or beginning of fragment if not present).
    ? "start":         int,

    ; end character offset (or end of fragment if not present).
    ? "end":         int
}

; one or two text-selector-maps used to identify the range
text-selector-range-map = {
    "selector":     text-selector-map, ; start (or only) text selector

    ; if present, represents the end of the text range
    ? "end":         text-selector-map
}

text-map = {
    ; array of (possibly discontinuous) ranges of text
    "selectors": [1* text-selector-range-map]
}

item-map = {
    ; the container-specific term used to identify items, such as "track_id" for
    ; MP4 or "item_ID" for HEIF
    "identifier":    tstr .size (1..max-tstr-length),

    ; the value of the identifier, e.g. a value of "2" for an identifier of
    ; "track_id" would imply track 2 of the asset
    "value":        tstr .size (1..max-tstr-length),
}
```

16.9.2.4. Examples

A series of examples in CBOR diagnostic notation (RFC 8949, clause 8) are shown below:

```
// example of a combined temporal and spatial range in a video //
{
  "region": [
    {
      "type": "temporal",
```

```
      "time": {
        "type": "npt",
        "start": "0",
        "end": "5.2"
      }
    },
    {
      "type": "spatial",
      "shape": {
        "type": "rectangle",
        "unit": "pixel",
        "origin": {
          "x": 10.0,
          "y": 10.0
        },
        "width": 200.0,
        "height": 112.0
      }
    }
  ],
  "name": "Animated Logo",
  "identifier": "logo-clip",
  "description": "5.2 seconds of the opening animated logo, in a rectangle 10 pixels down
from the top and left, 200px by 112px"
}

// example of a textual range in a Word/DOCX file //
{
  "region": [
    {
      "type": "textual",
      "text" : {
        "selectors" : [
          [
            {
              "fragment" : "xpointer(/w:document/w:body/w:p/)"
            }
          ]
        ]
      }
    },
  ],
  "description": "AI assistant edited content"
}

// example of a textual range in a tagged PDF file //
{
  "region": [
    {
      "type": "textual",
      "text" : {
        "selectors" : [
          [
```

```
            {
              "selector" : {
                "fragment" : "path=/Document/Sect[0]/P[3]",
                "start" : 10,
                "end" : 20
              }
            }
          ]
        ]
      }
    },
  ],
  "description": "Redaction performed as per FOIA request"
}

// example of a textual range in a non-tagged PDF file //
//  in this case, we can only specify a page & rectangular area //
{
  "region": [
    {
      "type": "textual",
      "text" : {
        "selectors" : [
          [
            {
              "selector" : {
                "fragment" : "page=1,rect=10,10,450,500",
                "start" : 10,
                "end" : 20
              }
            }
          ]
        ]
      }
    },
  ],
  "description": "Redaction performed as per FOIA request"
}

// example of deletion of some pages from a PDF //
{
  "region": [
    {
      "type": "frame",
      "frame" : {
        "start" : 27,
        "end" : 30
      }
    },
  ],
  "description": "unnecessary pages removed before distribution"
}
```

```
// example of a range of cells in Excel //
{
  "region": [
    {
      "type": "textual",
      "text" : {
        "selectors" : [
          [
            {
              "selector" : {
                "fragment" : "xpointer(Sheet1!A5:A10)",
              }
            }
          ],
          [
            {
              "selector" : {
                "fragment" : "xpointer(Sheet1!B5:B10)",
              }
            }
          ]
        ]
      }
    },
  ],
  "description": "applied some styling to a range of cells in Excel"
}

// example of a contiguous range of table cells //
{
  "region": [
    {
      "type": "textual",
      "text" : {
        "selectors" : [
          [
            {
              "selector" : {
                "fragment" : "xpointer(//table[1]/tr[1]/td[2])",
              },
              "end" : {
                "fragment" : "xpointer(//table[1]/tr[1]/td[4])",
              }
            }
          ]
            ]
      }
    },
  ],
  "description": "cleared some table cells"
}

// example of a range of a specific track of a video //
```

```
{
  "region": [
    {
      "type": "temporal",
      "time": {
        "type": "npt",
        "start": "0",
        "end": "5.2"
      }
    },
    {
      "type": "identified",
      "item": {
        "identifier": "track_id",
        "value": "2"
      }
    }
  ],
  "description": "enhanced some of the audio track"
}


// example of specifying that the eyes were changed throughout the entire asset //
{
  "region" : [
    {
      "type" : "temporal",
      "time" : {},
    },
    {
      "type" : "identified",
      "item" : {
        "identifier" : "https://bioportal.bioontology.org/ontologies/FMA",
        "value" : "set of eyeballs"
      },
    }
  ]
  "description": "made sure he looks like he was sleeping during the entire meeting"
}
```